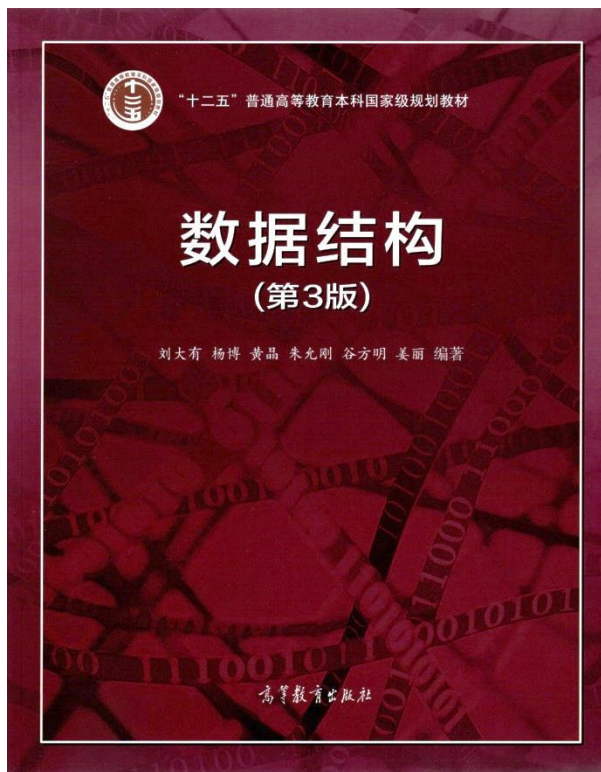




最小支撑树及图应用

- 最小支撑树的概念
- Prim算法
- Kruskal算法
- 强连通分量
- 图的其他应用举例

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn



戴文渊

上海交通大学本科硕士

香港科技大学博士

ACM/ICPC全球总决赛冠军

百度主任架构师、华为主任科学家

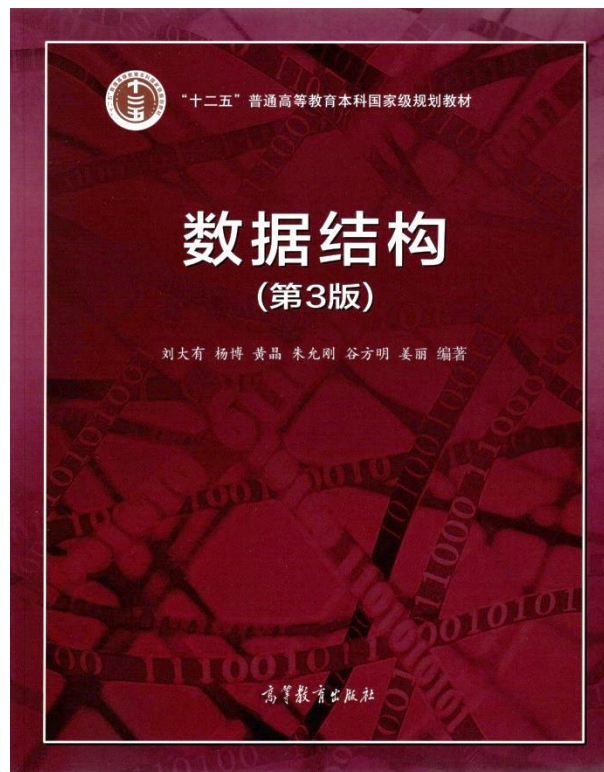
第四范式创始人兼CEO

身价85亿元人民币（据胡润财富榜）

学习达不到自己预期时，不用慌。每个人的预期总会或多或少的带点理想色彩，所以实际的效果一般来说总是不如预期的。有时甚至会出现只有预期的三分之一到二分之一之一的情况。

但是，只要你坚持不懈地去做，不动摇自己的信念，最后的成绩一定不会差。

记住，你遇到的困难，你的对手也会遇到的。



最小支撑树及图应用

- **最小支撑树概念**
- Prim算法
- Kruskal算法
- 强连通分量
- 图的其他应用举例

数据之美
结构之美
算法之道

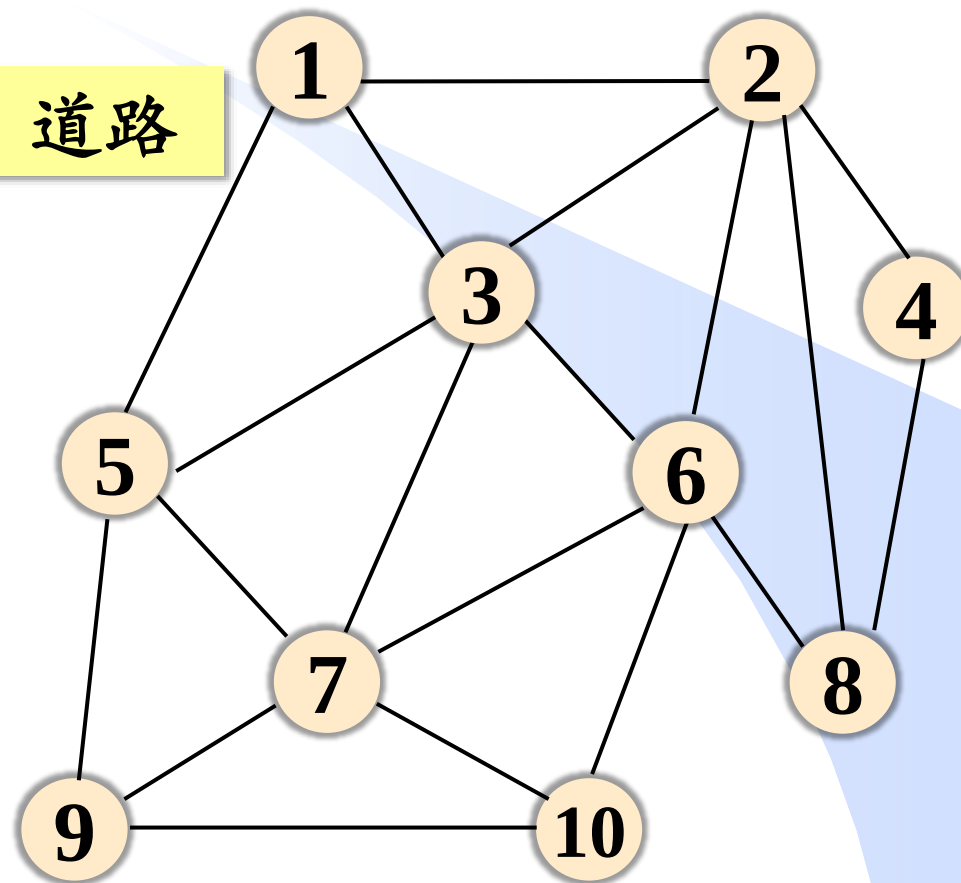
zhuyungang@jlu.edu.cn

问题引入

在 n 个地点间修缮造价最低的连通道路。



边：道路

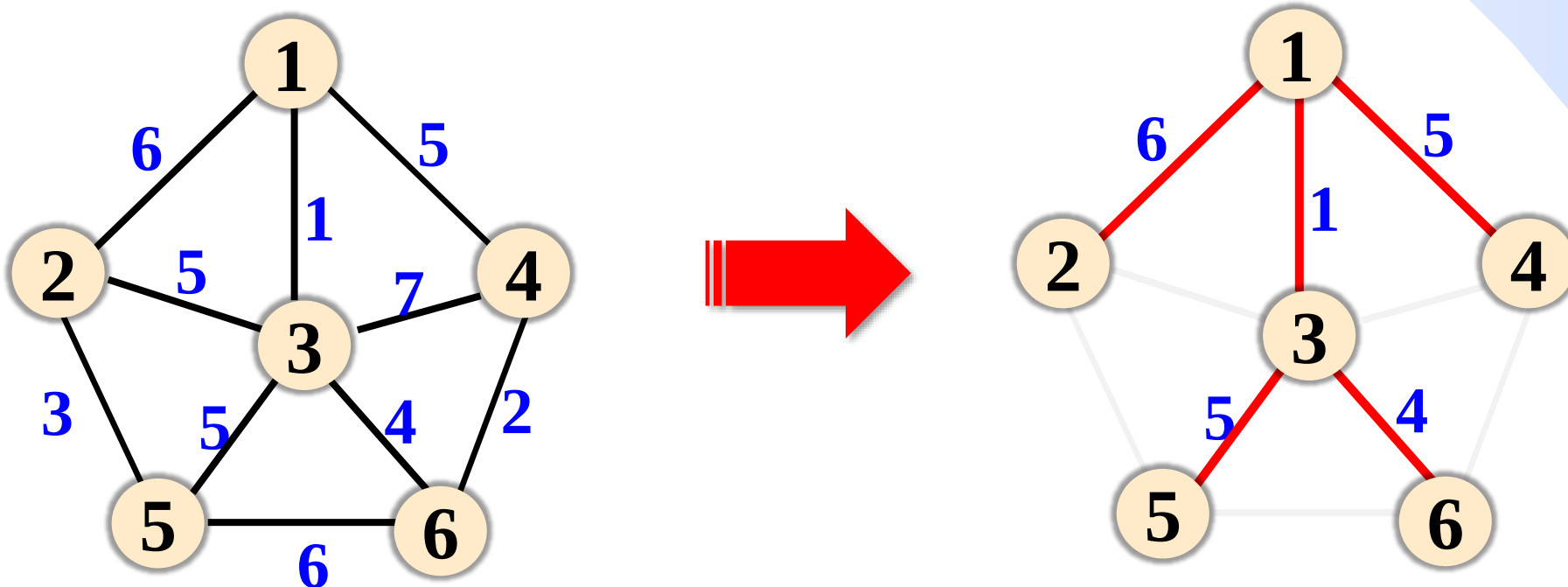


边的权值：修缮道路的费用

支撑树 (Spanning Tree)

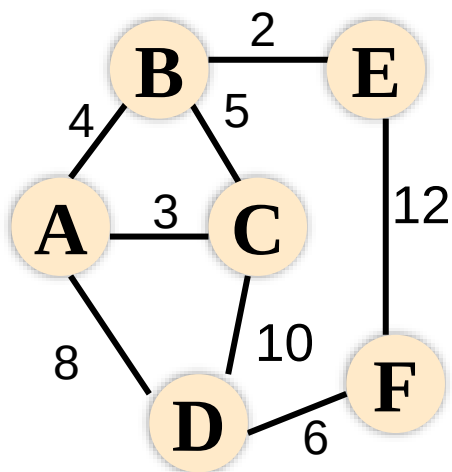
- 给定包含 n 个顶点的无向带权连通图 G ，由 G 中 n 个顶点和 $n-1$ 条边构成的连通子图，称为 G 的一棵支撑树，亦称生成树。
- 支撑树包含图 G 的全部顶点，且包含使子图连通所需的最少的边（ $n-1$ 条边）。

课下思考：支撑树中有环么？

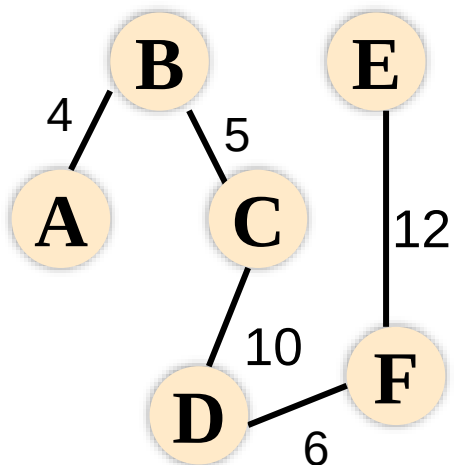


最小支撑树 (Minimum Spanning Tree, MST)

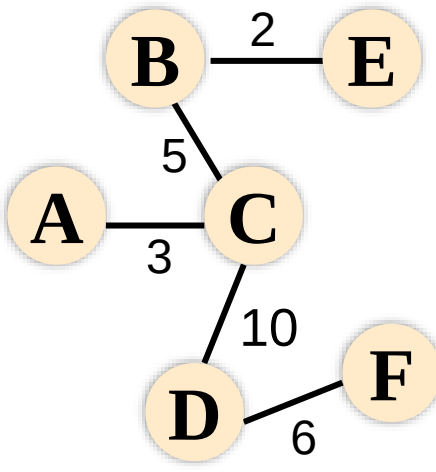
➤ 边权之和最小的支撑树称为G的最小支撑树。



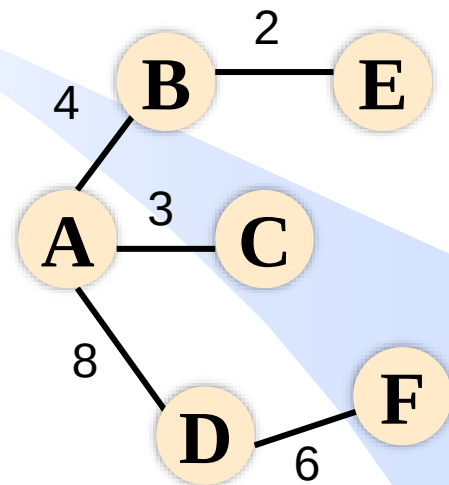
无向带权
连通图G



G的支撑树
边权和 37



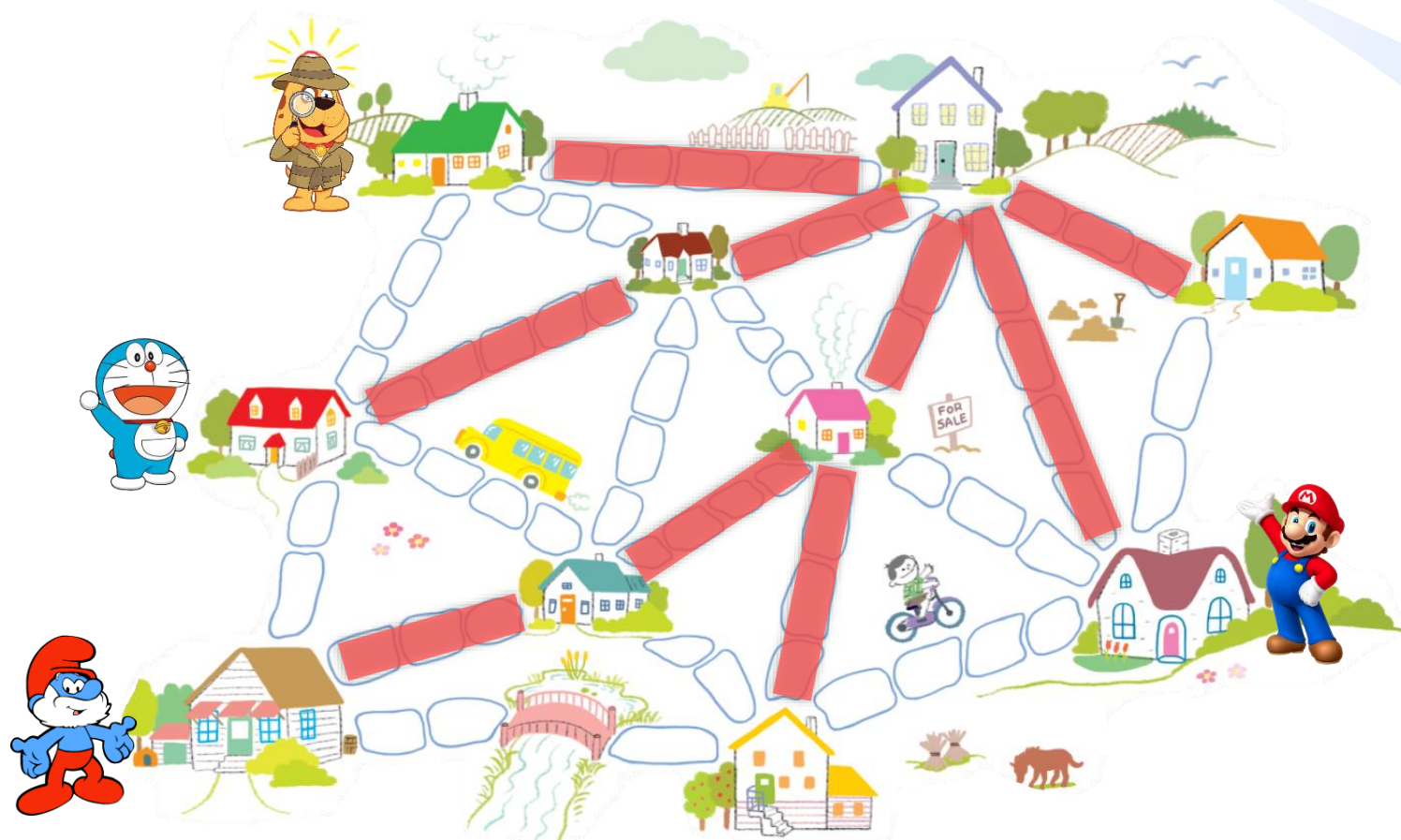
G的支撑树
边权和 26



G的最小支撑树
边权和23

最小支撑树的典型应用

网络设计：如交通网络、通信网络等。



应用广泛



检索范围: 学术期刊 (篇名: 最小生成树)

☐ 全选 已选: 0 清除 批量

☐ 1 基于最小生成树的高分辨率;

☐ 2 基于最小生成树原理的矿井;

☐ 3 基于LINGO的最小支撑树问

☐ 4 一种求解双目标最小生成树;

☐ 5 最小支撑树博弈:重新审视Bi

☐ 6 基于度约束最小生成树的域

☐ 7 基于最小生成树的船舶运输;

☐ 8 基于局部差异最小生成树脑

☐ 9 面向装配序列规划的最小生

☐ 10 一种基于最小生成树的无人

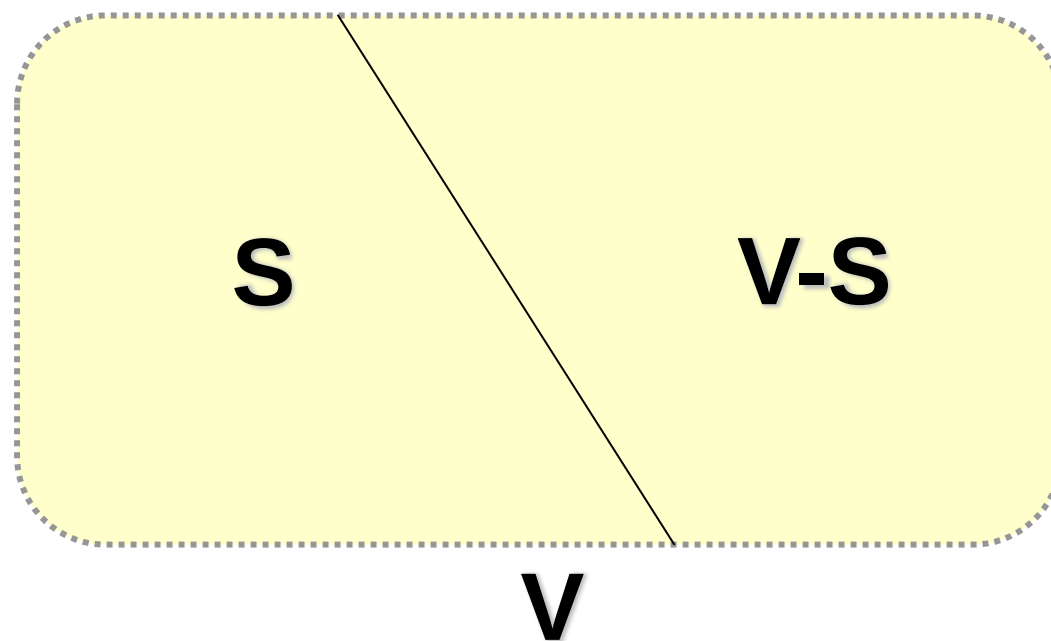
☐ 11 最小生成树算法应用于蛋白

☐ 38 基于互近邻相对距离的最小生成树聚类算法	程汝峰; 刘奕志; 梁永全	郑州大学学报(理学版)	2017-07-24 17:28	2019-09-15
☐ 39 基于聚类-最小生成树的沥青路面裂缝检测方法研究	张栋冰	中山大学学报(自然科学版)	2017-07-15	2019-01-15
☐ 40 一种基于统计学习理论的最小生成树图像分割准则	王平; 魏征; 崔卫红; 林志勇	武汉大学学报(信息科学版)	2017-07-05	2019-01-05 08:50
☐ 41 一种融合超像素与最小生成树的高分辨率遥感影像分割方法	董志鹏; 王密; 李德仁	测绘学报	2017-06-15	2018-12-15
☐ 42 基于广义最小生成树的多微网源荷储恢复顺序优化策略	许志荣; 杨苹; 曾智基; 何婷; 彭嘉俊	电力系统自动化	2017-04-25	2018-12-15
☐ 43 基于自然邻居和最小生成树的原型选择算法	朱庆生; 段浪军; 杨力军	计算机科学	2017-04-15	2018-11-16 13:47
☐ 44 基于GPU的Bor?vka最小生成树改进算法	吴永存; 刘成安; 印茂伟	科技通报	2017-02-28	2018-09-26
☐ 45 基于最小生成树的多视图特征点快速匹配算法	李丹; 朱玲玲; 胡迎松	华中科技大学学报(自然科学版)	2017-01-20 13:53	2018-09-15
☐ 46 基于最小生成树的渠道系统优化布局模型	许自昌	农业工程学报	2017-01-08	2018-07-15
☐ 47 基于最小生成树的含分布式电源的配电网重构算法	侯成滨; 王瑞峰	郑州大学学报(理学版)	2016-12-21 13:58	2018-07-11 17:19
☐ 48 粒子寻优和最小生成树聚类下的WSN能量优化	郑淼; 郑成增	计算机工程与应用	2016-12-16 14:00	2018-06-29 10:33
☐ 49 基于粒子群优化和最小生成树聚类的能耗均衡算法	李凯佳; 袁凌云; 俞锐刚	微电子学与计算机	2016-12-05	2018-06-16
☐ 50 基于直觉模糊集的随机最小支撑树选取	王肖霞; 杨风暴; 袁华	计算机工程	2016-10-15	2018-05-08
				2018-04-11 21:29



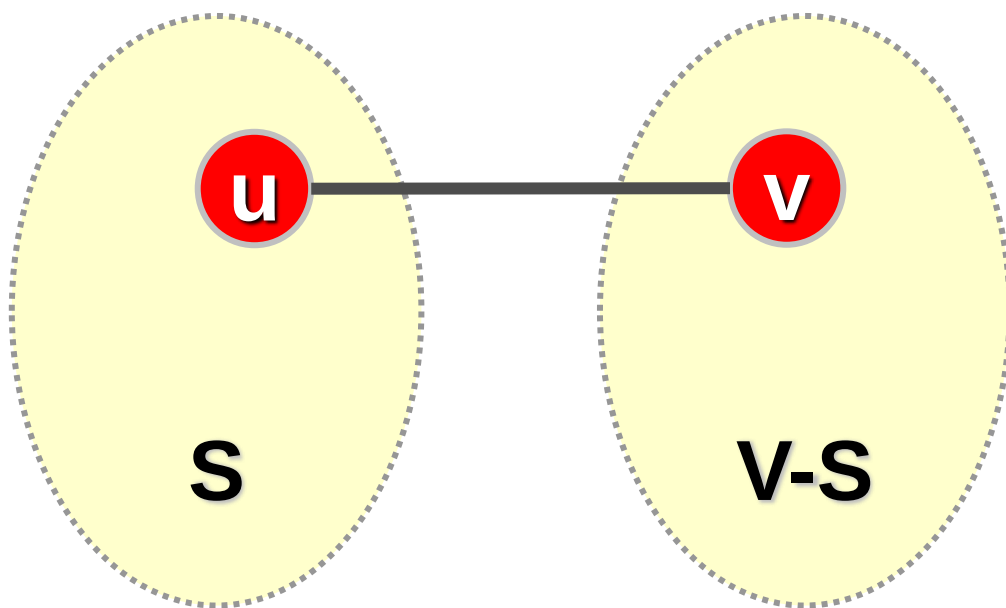
跨集合边

$G=(V,E)$ 是一个连通图， S 是顶点集 V 的一个非空子集，则图 G 的顶点被分为 S 和 $V-S$ 两个集合。



跨集合边

$G=(V,E)$ 是一个连通图， S 是 V 的一个非空子集，若边 (u,v) 满足 $u \in S, v \in V-S$ ，则称边 (u,v) 为跨集合边，简称跨边。



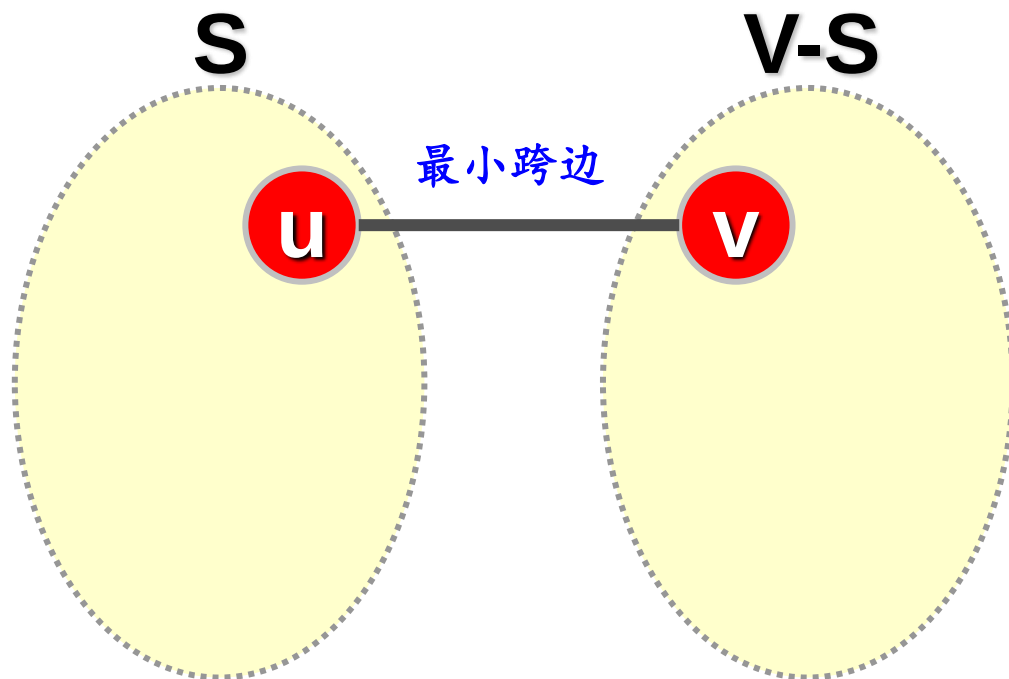
端点分别属于两个
集合的边

跨集合边

跨边

性质：最小跨边一定在某棵最小支撑树里

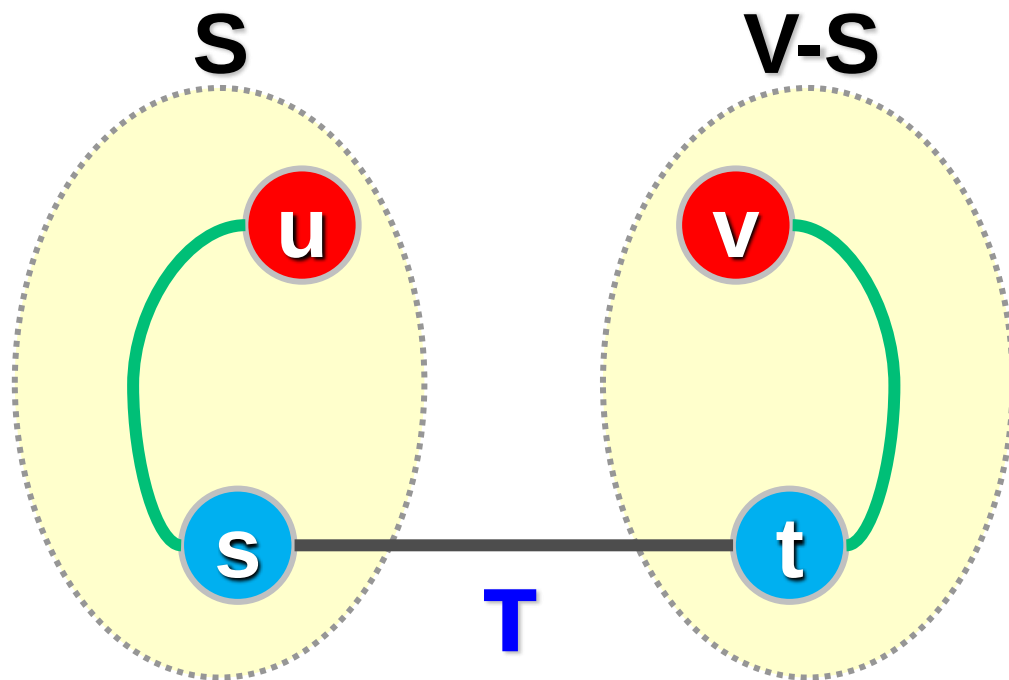
定理：连通图 G 中权值最小的跨集合边（简称**最小跨边**）一定在 G 的一棵**最小支撑树里**。即若边 (u,v) 满足 $\text{weight}(u,v)=\min\{\text{weight}(u_0, v_0) \mid u_0 \in S, v_0 \in V-S\}$ ，则必存在 G 的一棵最小支撑树 T ， T 包含边 (u,v) 。



证明：反证法，假定最小支撑树 T 不包含最小跨边 (u,v) ， T 是连通的，故 u 和 v 之间必有一条路径，该路径必包含一条跨集合边 (s,t) 。

性质：最小跨边一定在某棵最小支撑树里

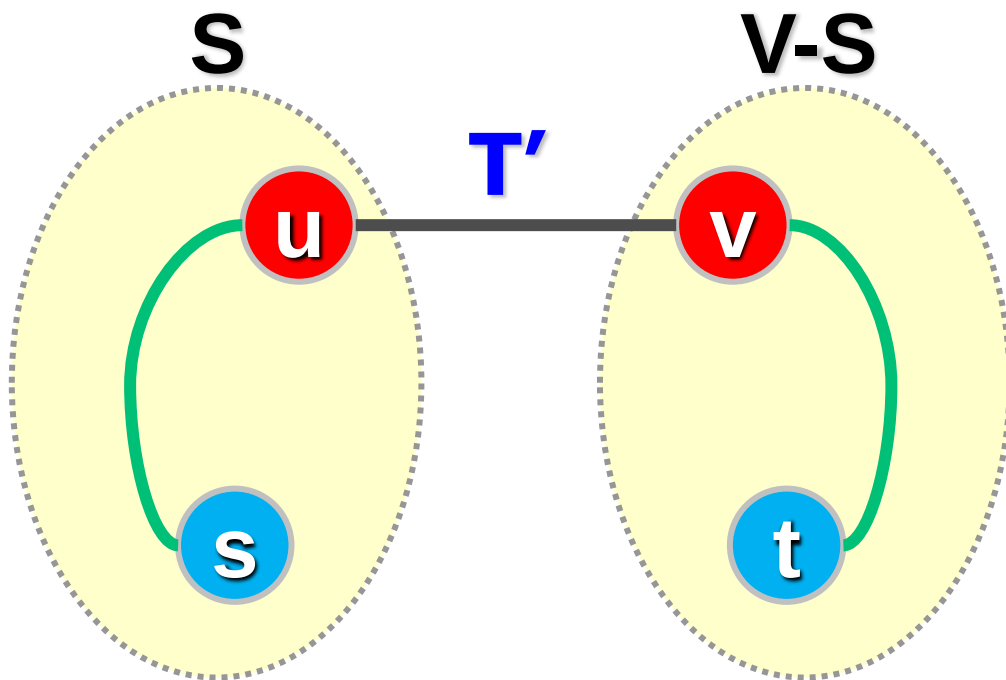
定理：连通图 G 中权值最小的跨集合边（简称**最小跨边**）**一定在 G 的一棵最小支撑树里**。即若边 (u,v) 满足 $\text{weight}(u,v)=\min\{\text{weight}(u_0,v_0) \mid u_0 \in S, v_0 \in V-S\}$ ，则必存在 G 的一棵最小支撑树 T ， T 包含边 (u,v) 。



证明：反证法，假定最小支撑树 T 不包含最小跨边 (u,v) ， T 是连通的，故 u 和 v 之间必有一条路径，该路径必包含一条跨集合边 (s,t) 。

性质：最小跨边一定在某棵最小支撑树里

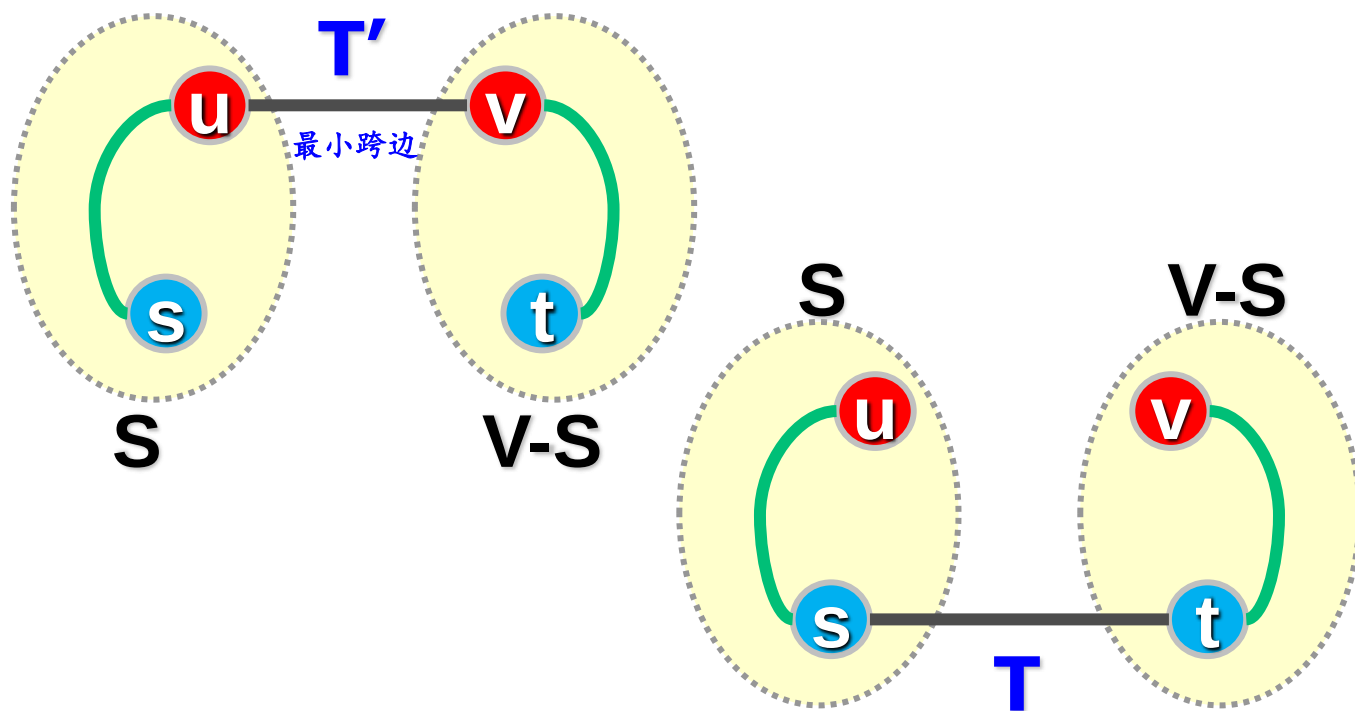
定理：连通图 G 中权值最小的跨集合边（简称**最小跨边**）一定在 G 的一棵**最小支撑树里**。即若边 (u,v) 满足 $\text{weight}(u,v) = \min\{\text{weight}(u_0, v_0) \mid u_0 \in S, v_0 \in V-S\}$ ，则必存在 G 的一棵最小支撑树 T ， T 包含边 (u,v) 。



对 T 删去边 (s, t) ，加入边 (u, v) ，得到一棵新的支撑树 T'

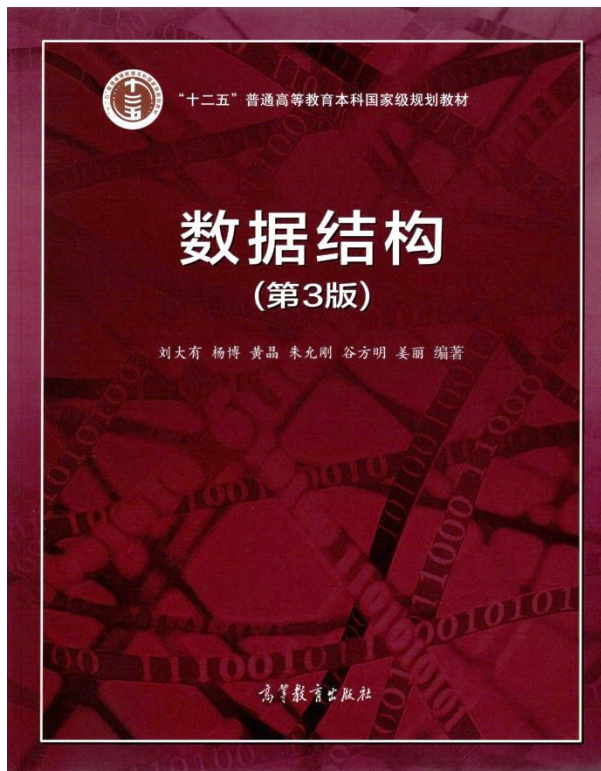
性质：最小跨边一定在某棵最小支撑树里

定理：连通图 G 中权值最小的跨集合边（简称**最小跨边**）**一定在 G 的一棵最小支撑树里**。即若边 (u,v) 满足 $\text{weight}(u,v) = \min\{\text{weight}(u_0, v_0) \mid u_0 \in S, v_0 \in V-S\}$ ，则必存在 G 的一棵最小支撑树 T ， T 包含边 (u,v) 。



由于 $w(u,v) < w(s,t)$ ，故 T' 的边权之和小于 T ，这与 T 是最小支撑树矛盾。

该定理是最小支撑树算法的依据



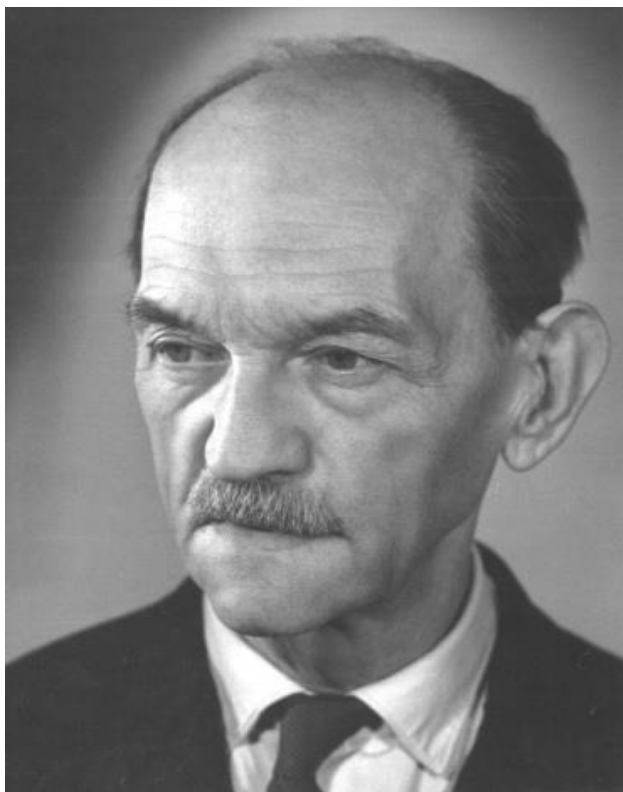
最小支撑树及图应用

- 最小支撑树的概念
- **Prim算法**
- Kruskal算法
- 强连通分量
- 图的其他应用举例

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

Prim算法

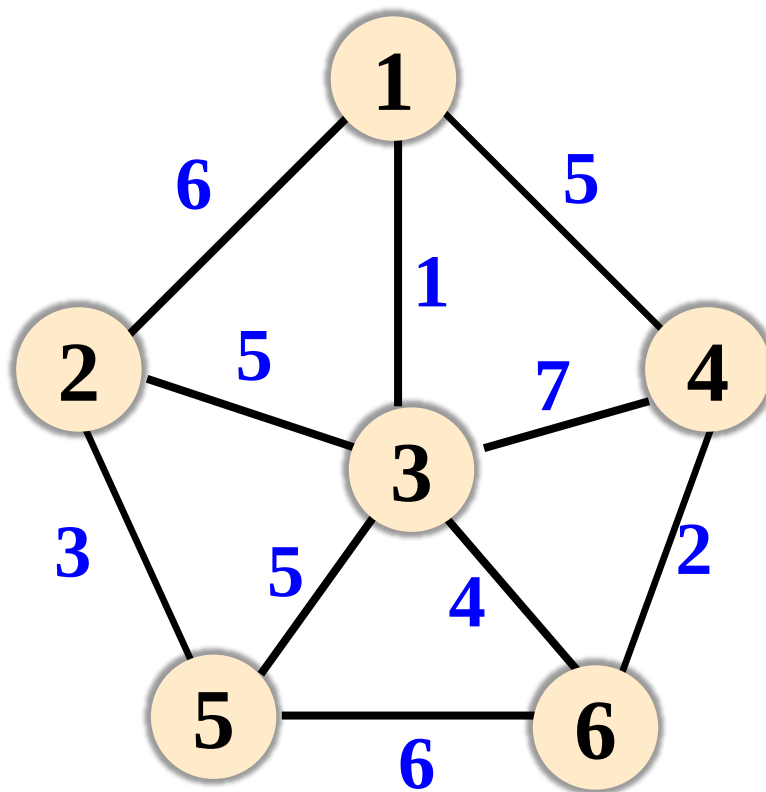


Vojtěch Jarník
布拉格大学教授
捷克科学院院士



Robert Prim
贝尔实验室数学研究中心主任

Prim算法举例



Prim算法举例

集合S:

1

集合V-S:

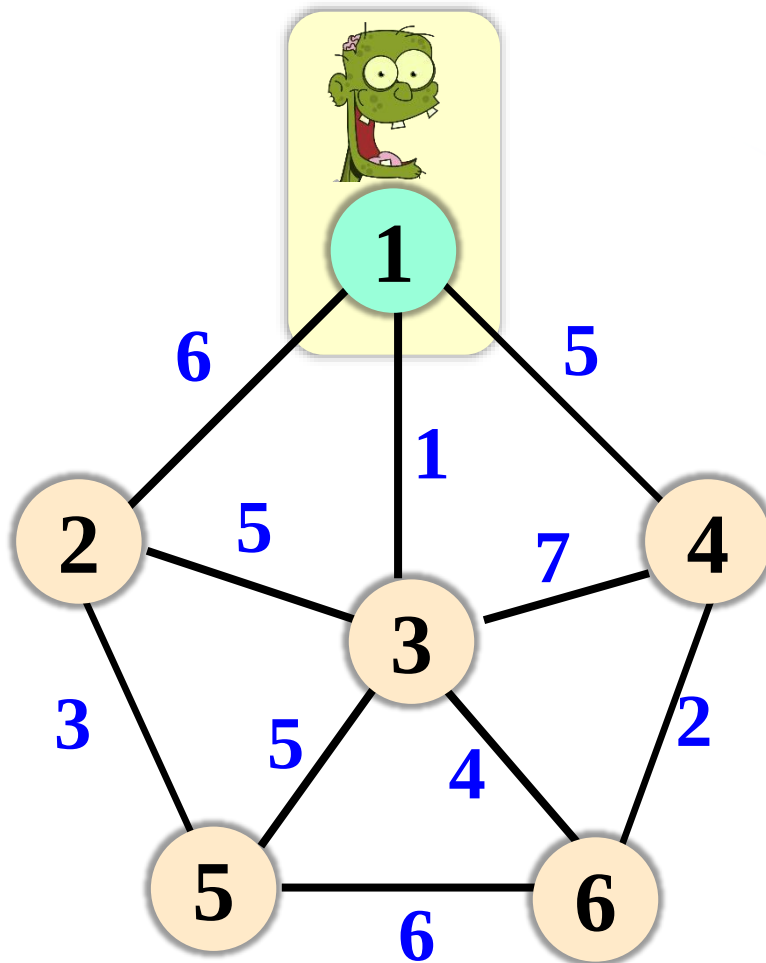
2

3

4

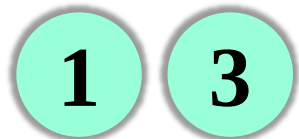
5

6

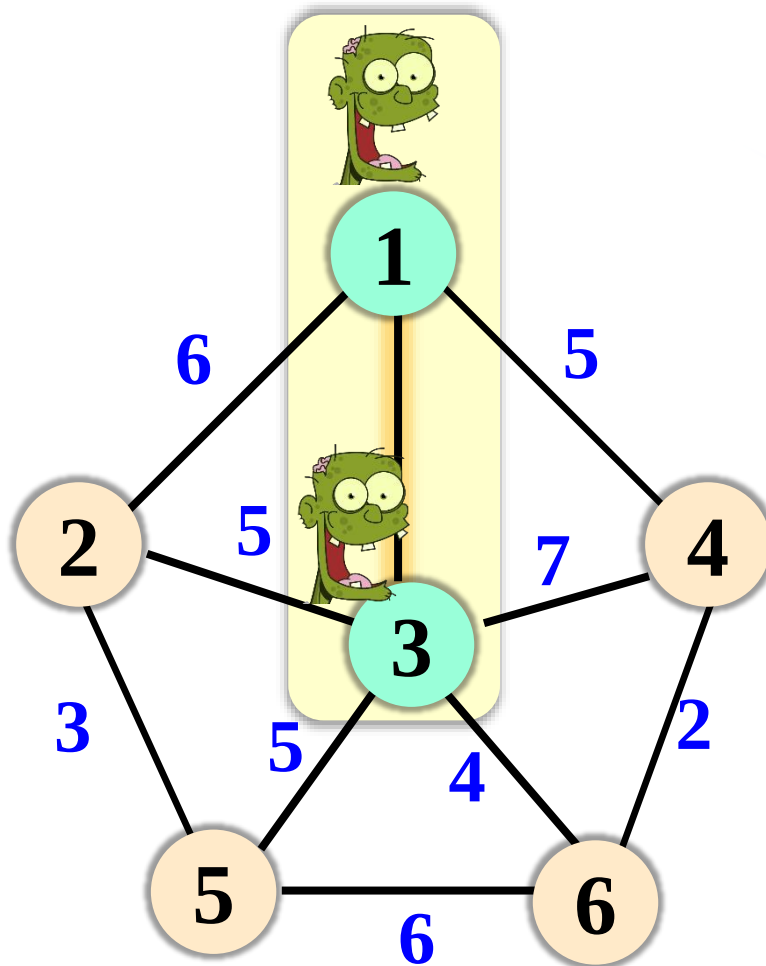
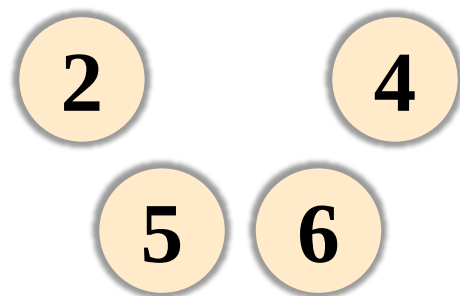


Prim算法举例

集合S:



集合V-S:



最小跨边

1—3

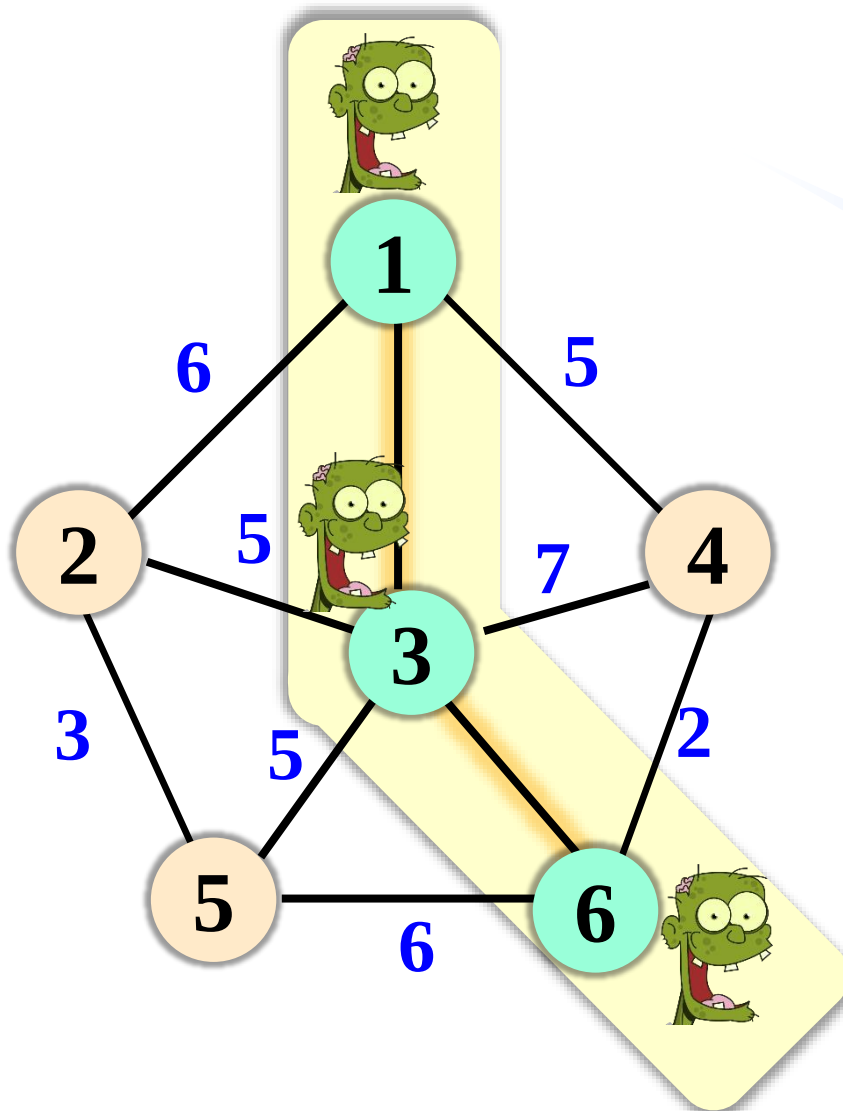
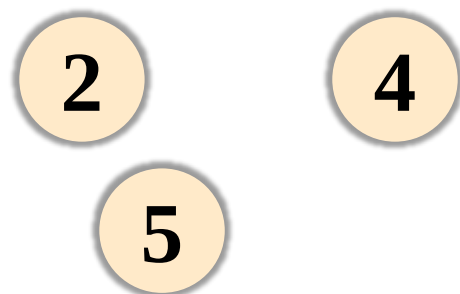
3—6

Prim算法举例

集合S:



集合V-S:



最小跨边

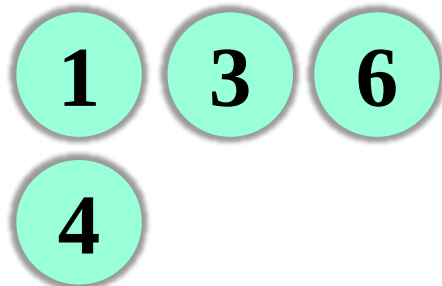
1—3

3—6

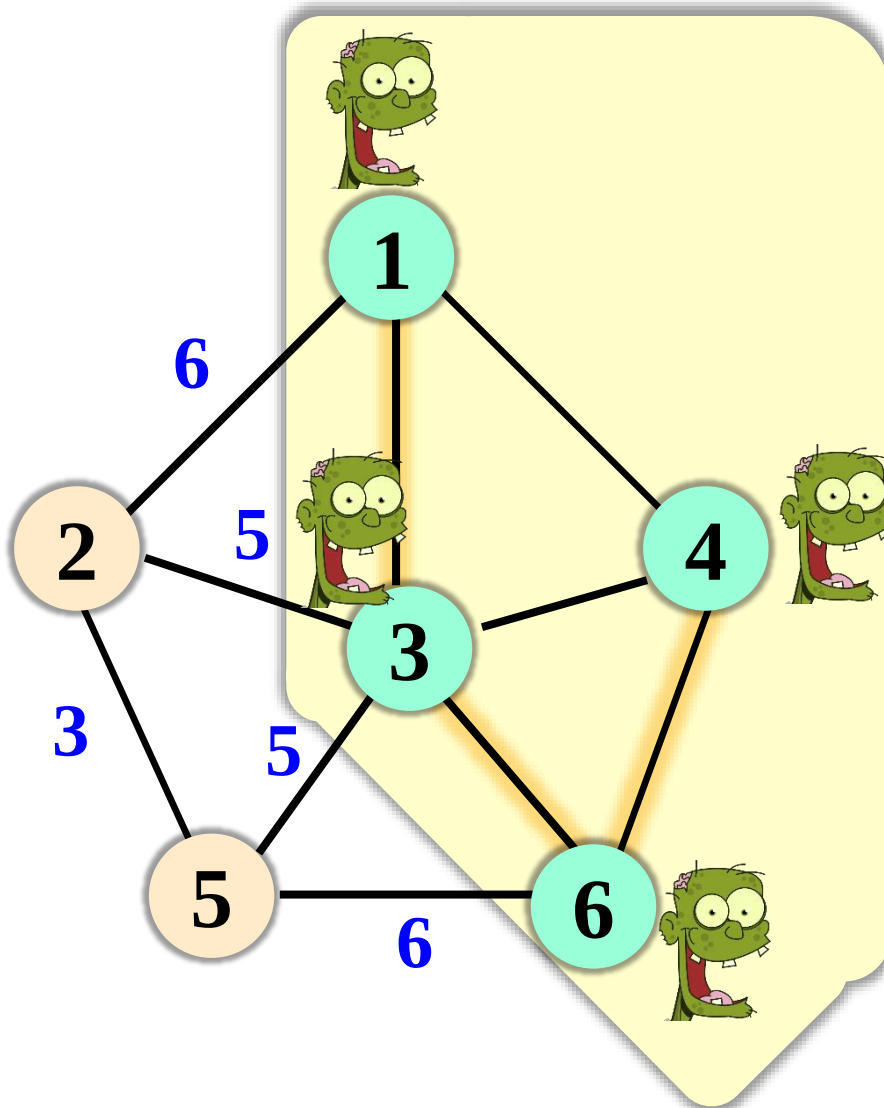
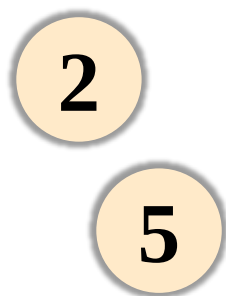
6—4

Prim算法举例

集合S:



集合V-S:



最小跨边

1—3

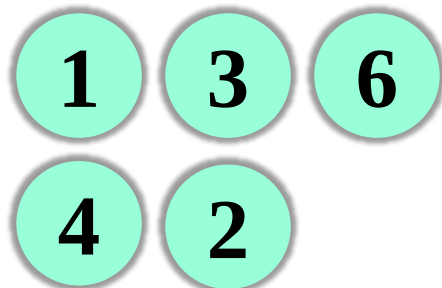
3—6

6—4

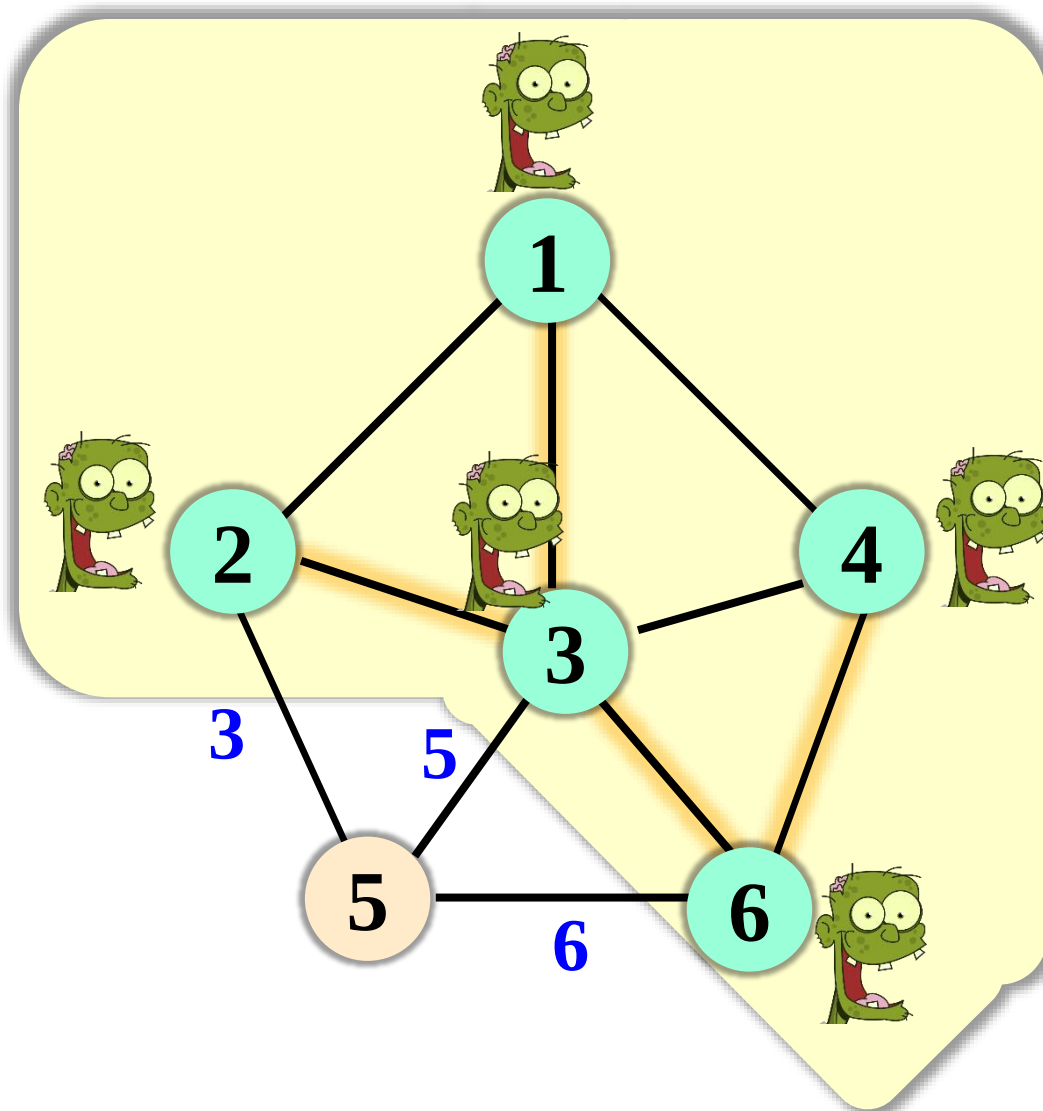
3—2

Prim算法举例

集合S:



集合V-S:



最小跨边

1—3

3—6

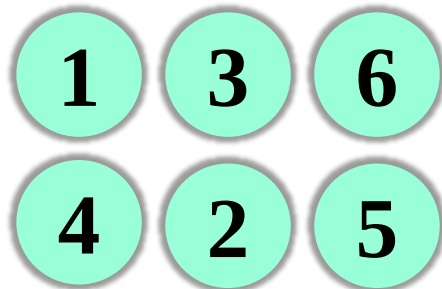
6—4

3—2

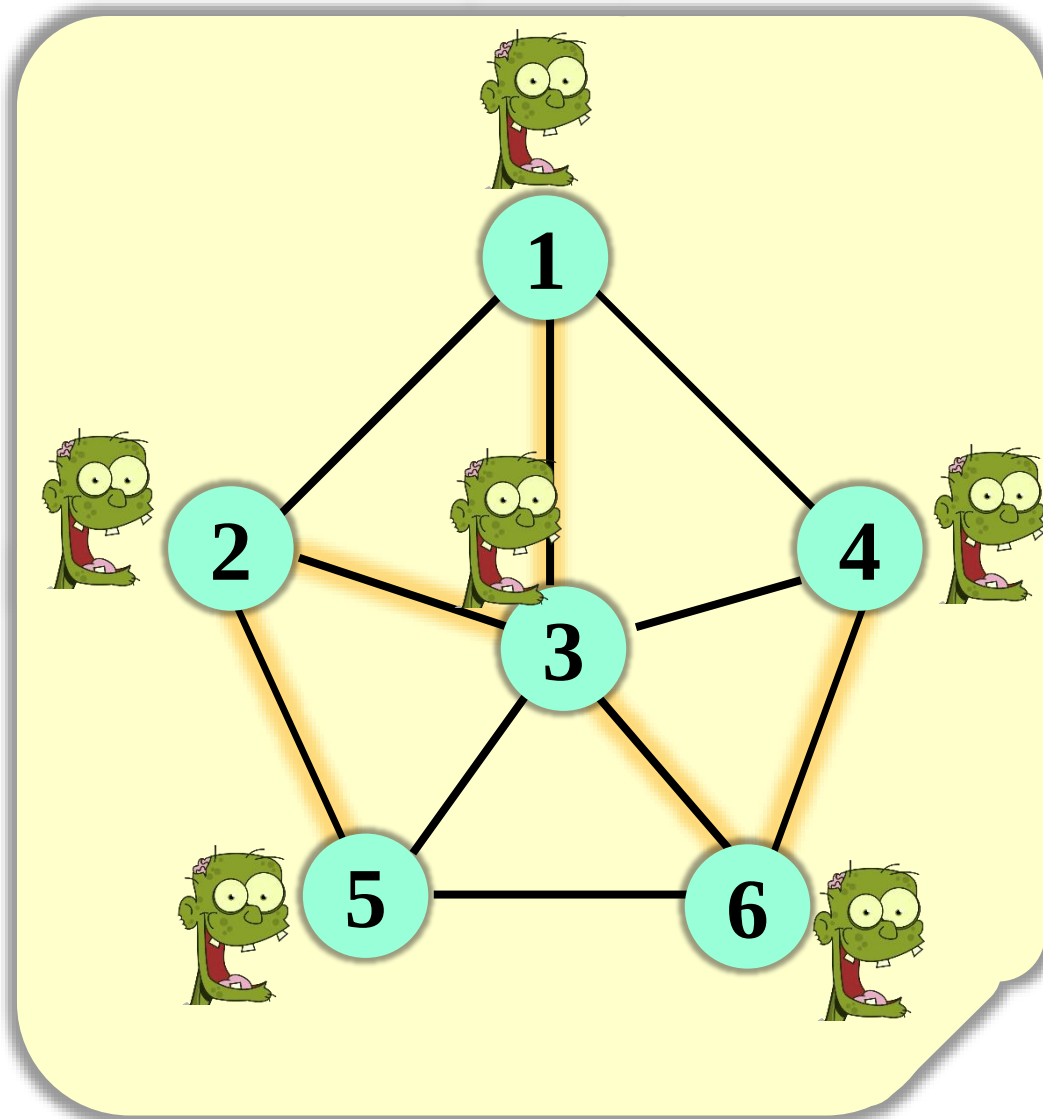
2—5

Prim算法举例

集合S:



集合V-S:



最小跨边

1—3

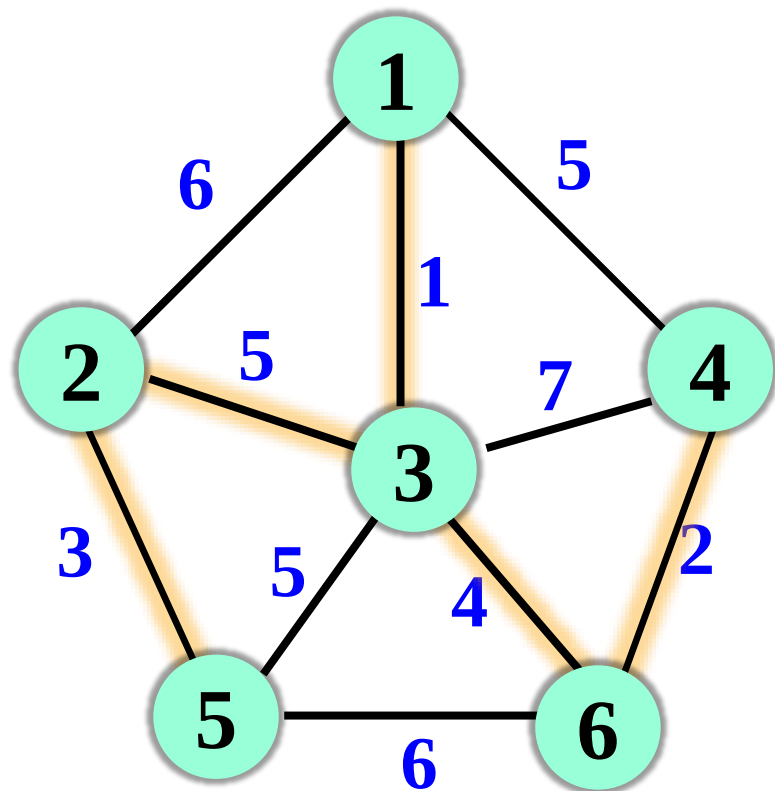
3—6

6—4

3—2

2—5

Prim算法举例



最小跨边

1—3

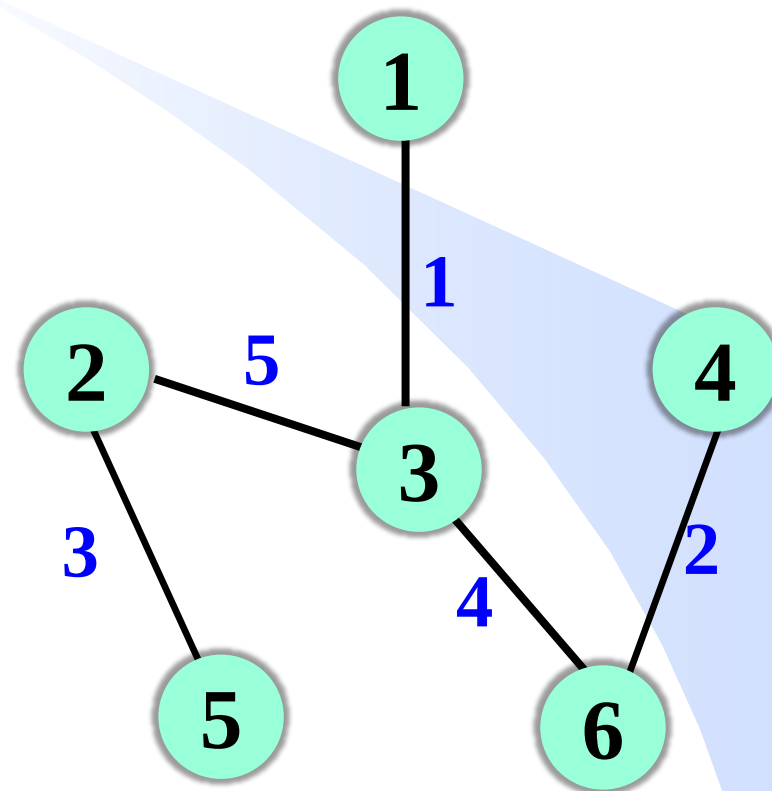
3—6

6—4

3—2

2—5

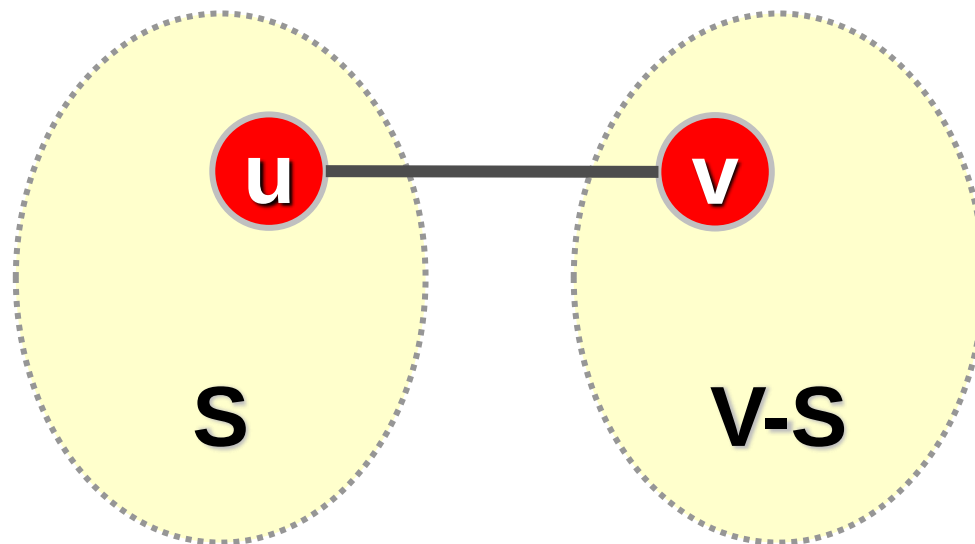
最小支撑树



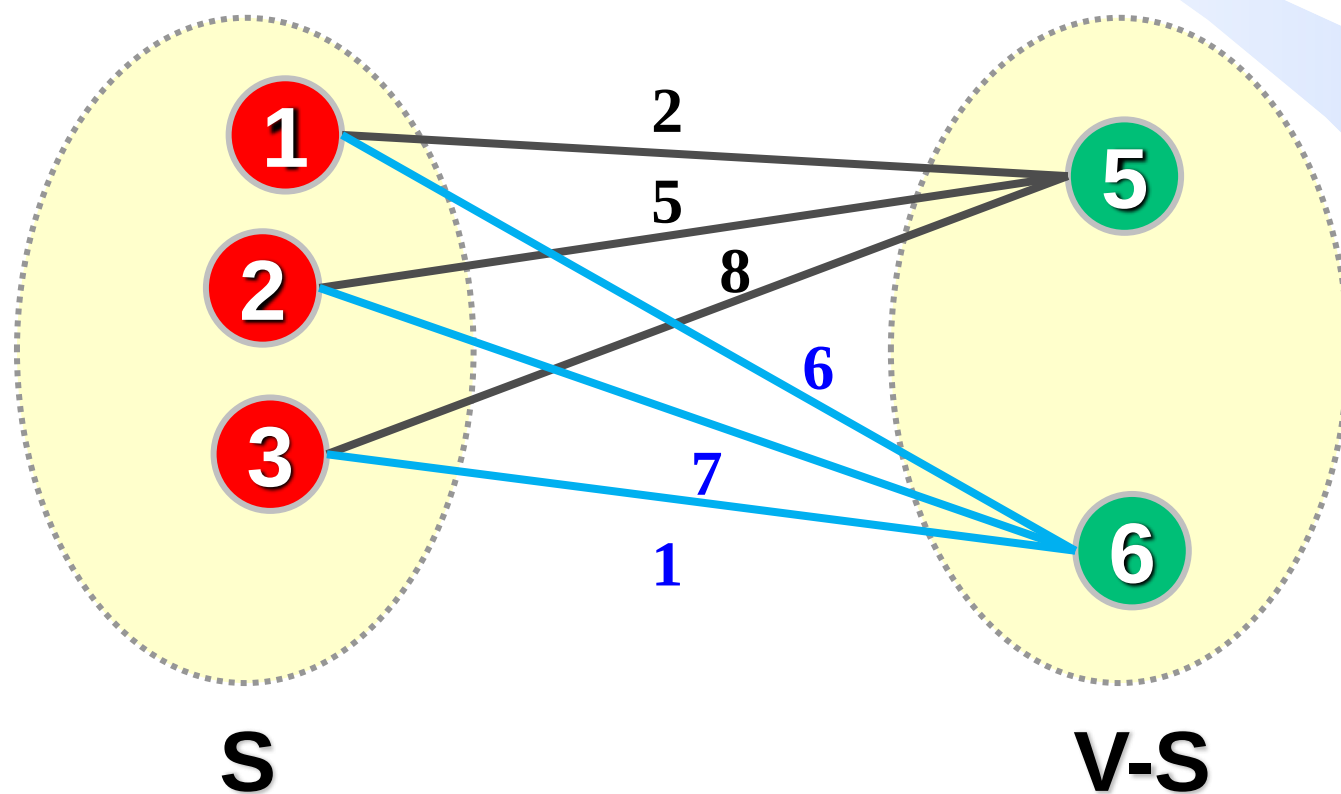
Prim算法

设 $G=(V, E)$ 为连通图， S 为最小支撑树的顶点集。

- ① 选择任一点 u 做为起点，放入集合 S ，即令 $S=\{u\}$ ($u \in V$);
- ② 找最小跨集合边 (u, v) ，即端点分别属于集合 S 和 $V-S$ 且权值最小的边，将该边加入最小支撑树，并将点 v 放入 S ;
- ③ 执行②，直至 $S=V$.

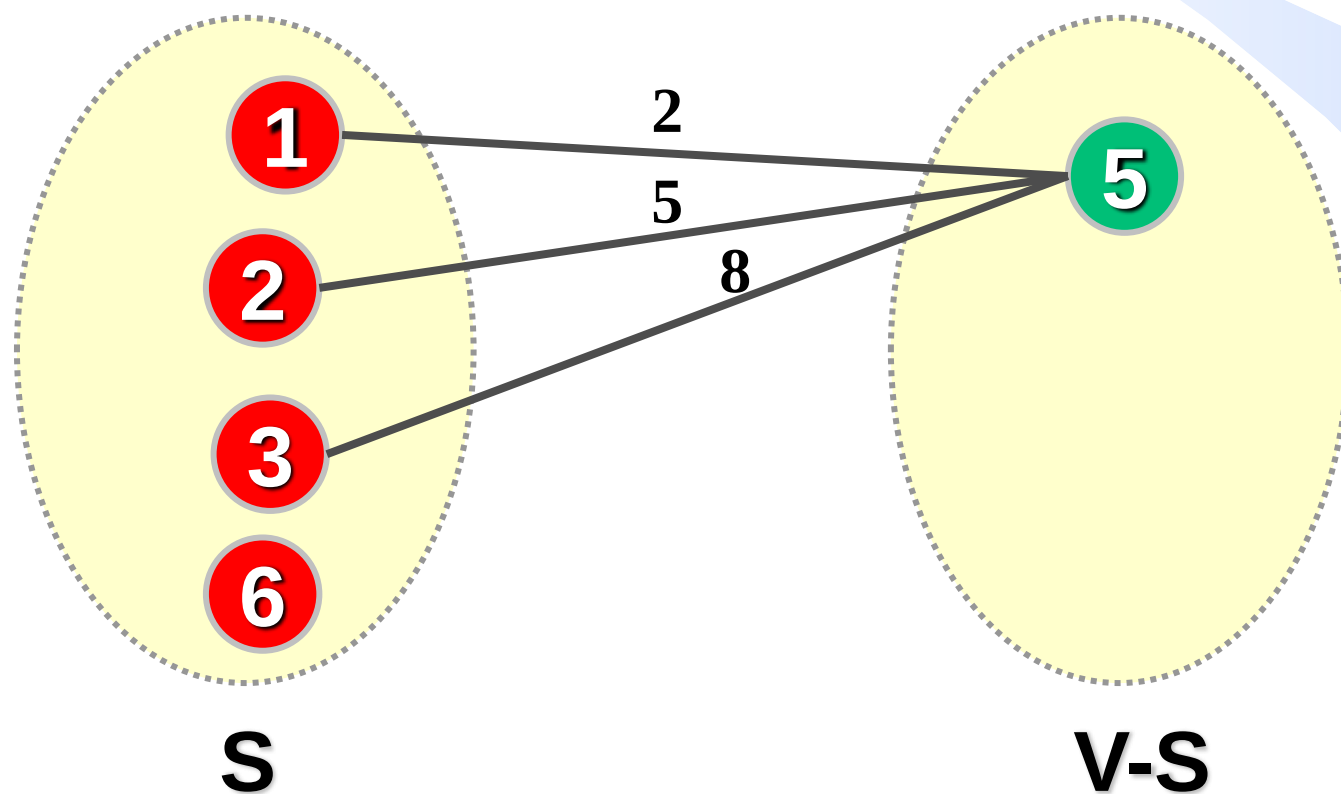


如何找最小跨集合边?



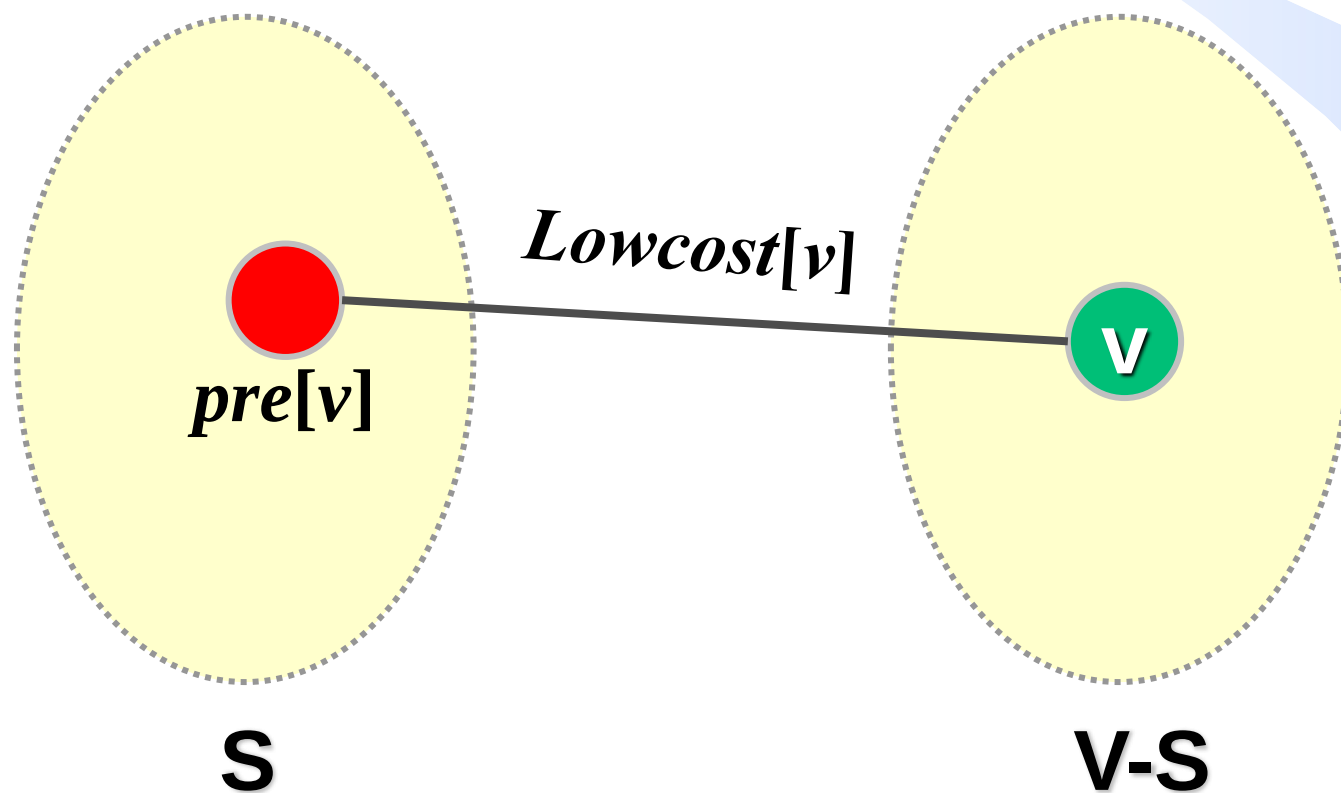
如何找最小跨集合边?

扫描所有跨集合边找权值最小的边，涉及重复运算

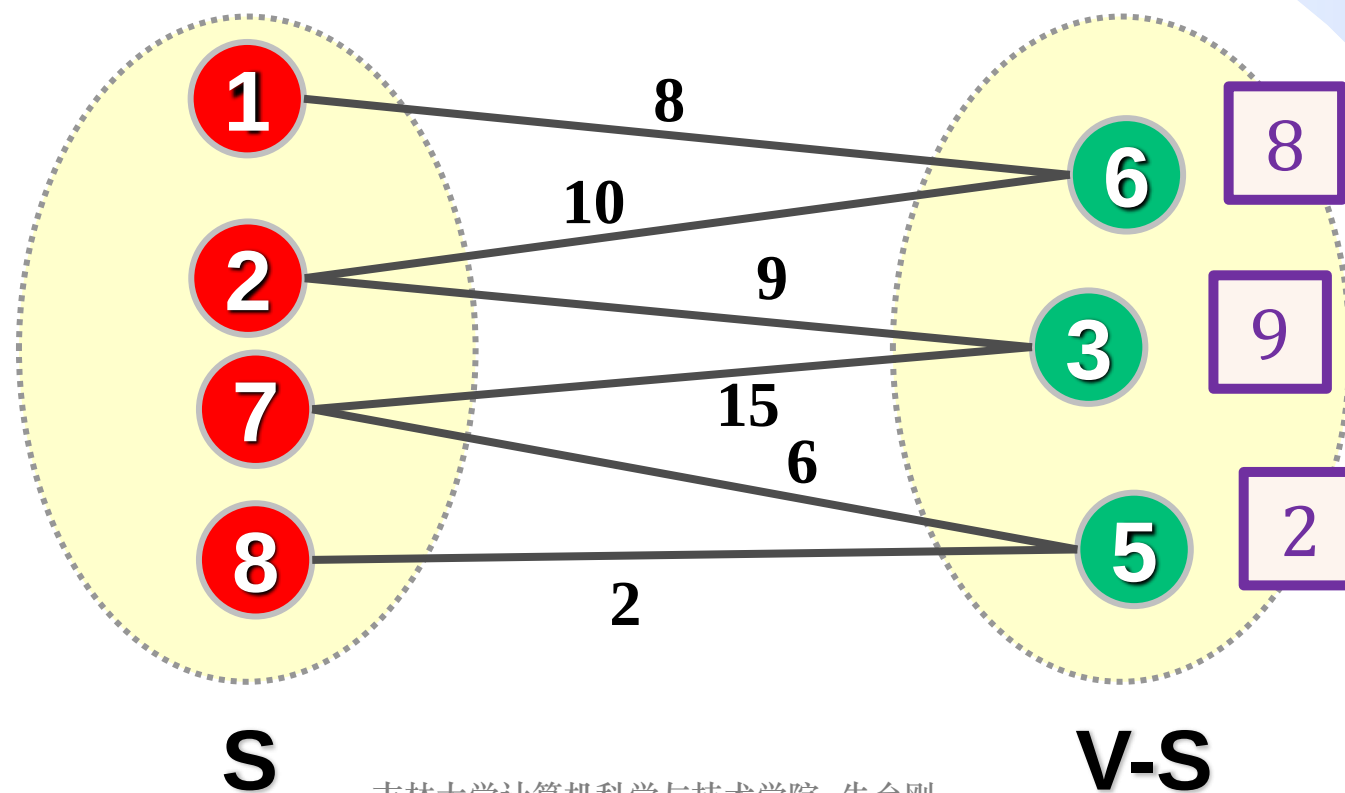


如何找最小跨集合边？

把顶点 v 到集合 S 的最小跨边保存起来，权值存入 $Lowcost[v]$ ，边在集合 S 中的端点存入 $pre[v]$ 。即 $Lowcost[v] = \min\{weight(u, v) | u \in S, v \in V-S\}$, $pre[v] = u, u \in S$ 。



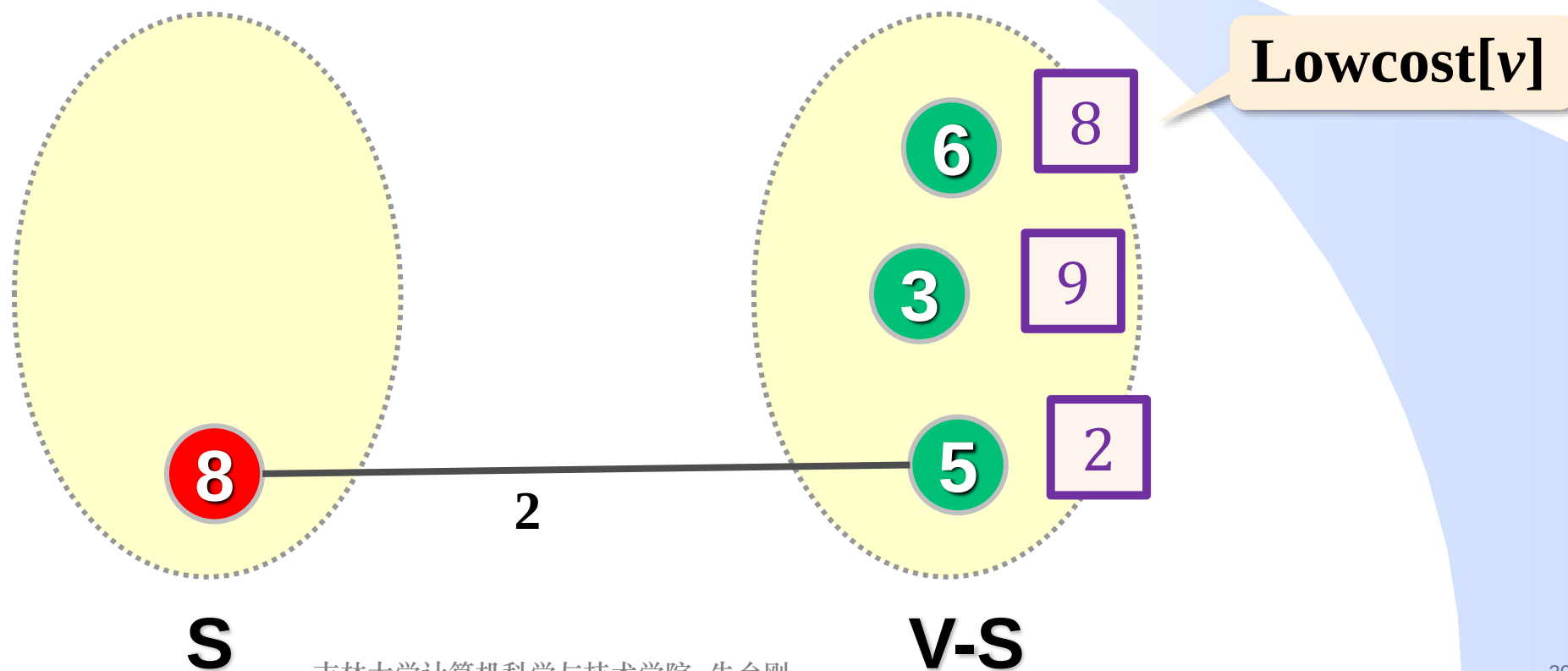
如何找最小跨集合边?



如何找最小跨集合边？

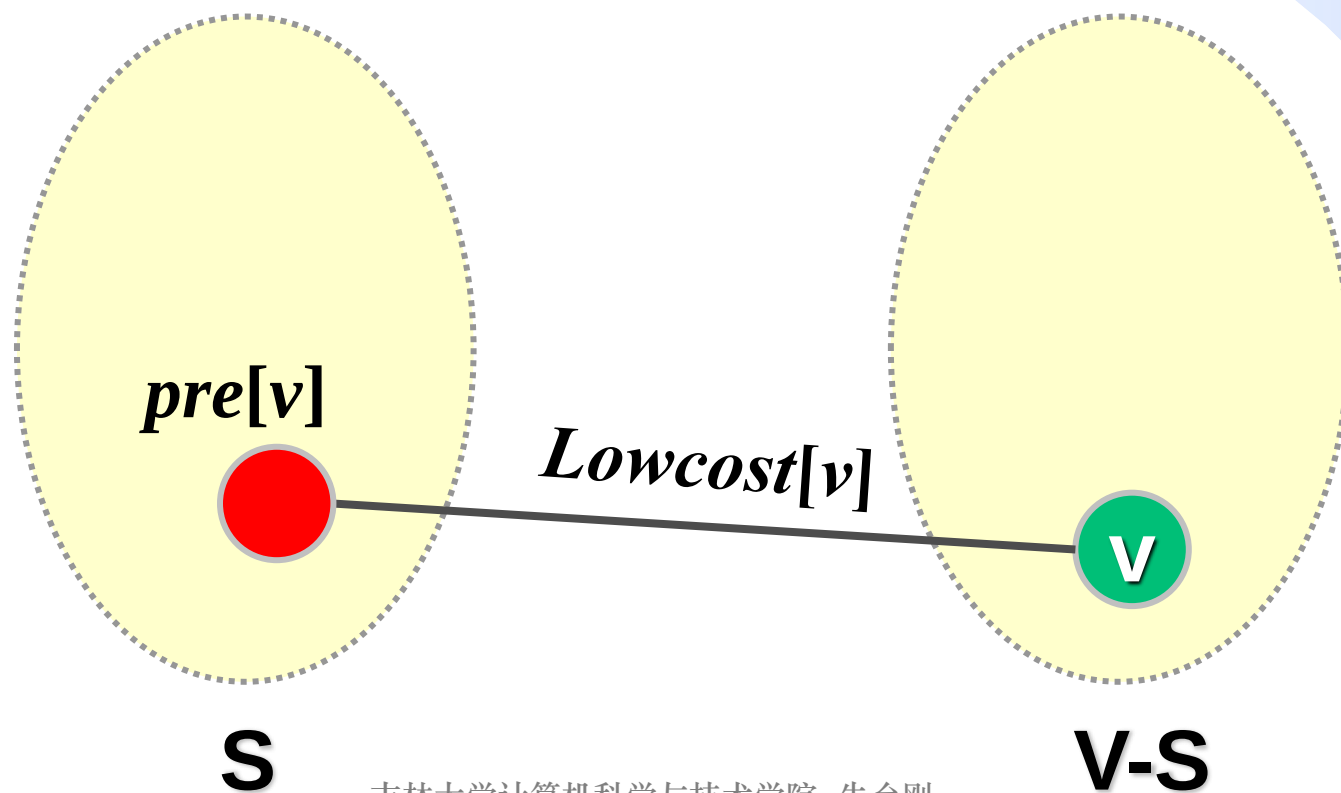
每次选择 **Lowcost** 值最小的顶点 v ，把边 $(pre[v], v)$ 加入最小支撑树，把点 v 放入集合 S 。

整个图的最小跨边的权值： $\min_{i \in V-U} \{ \text{Lowcost}[i] \}$



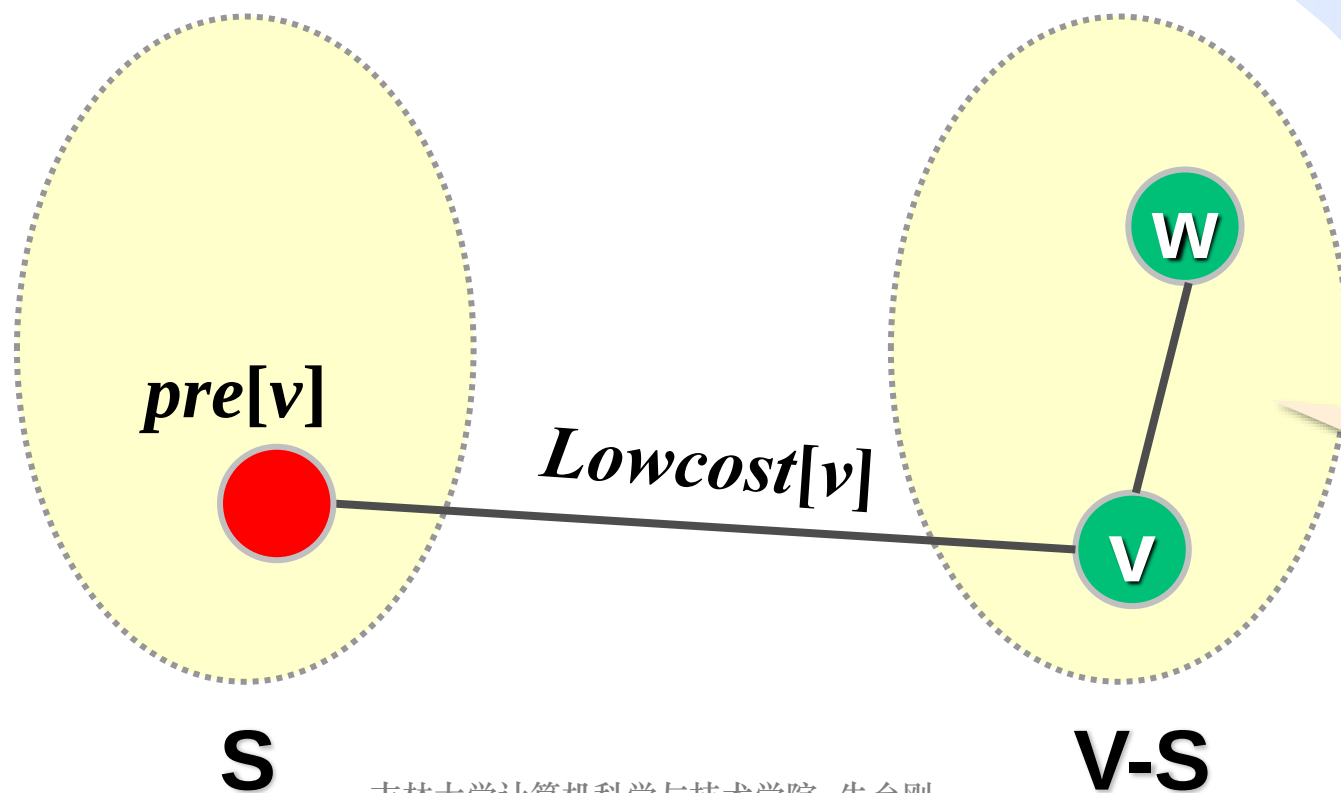
如何找最小跨集合边？

每次选择 Lowcost 值最小的顶点 v ，把边 $(\text{pre}[v], v)$ 加入最小支撑树，把点 v 放入集合 S 。



如何找最小跨集合边？

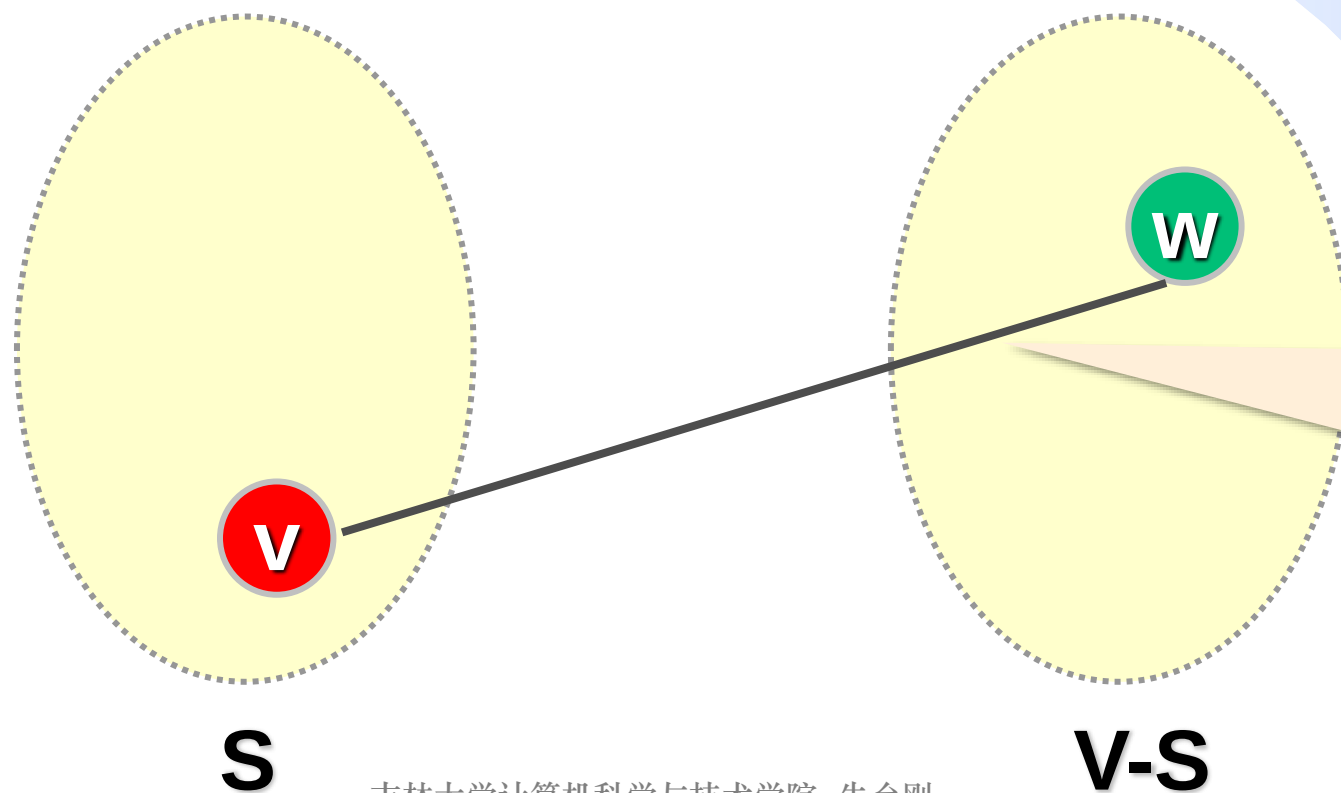
每次选择 Lowcost 值最小的顶点 v ，把边 $(\text{pre}[v], v)$ 加入最小支撑树，把点 v 放入集合 S 。



如果 v 有邻接
顶点 $w...$

如何找最小跨集合边？

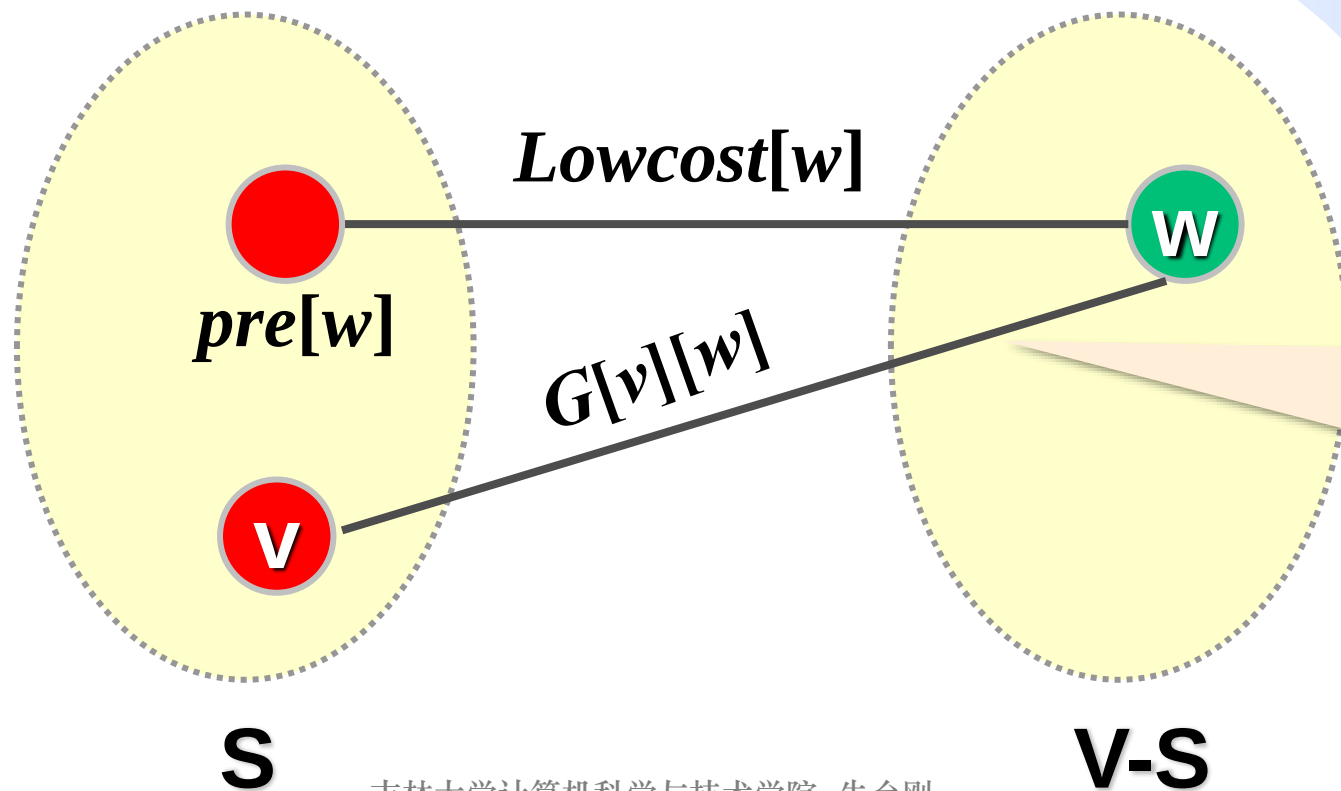
每次选择 $Lowcost$ 值最小的顶点 v ，把边 $(pre[v], v)$ 加入最小支撑树，把点 v 放入集合 S 。



将 v 加入集合 S 后将产生新的跨集合边 (v, w)

如何找最小跨集合边?

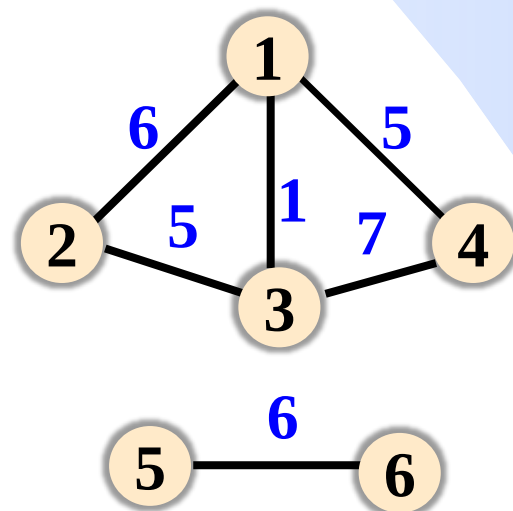
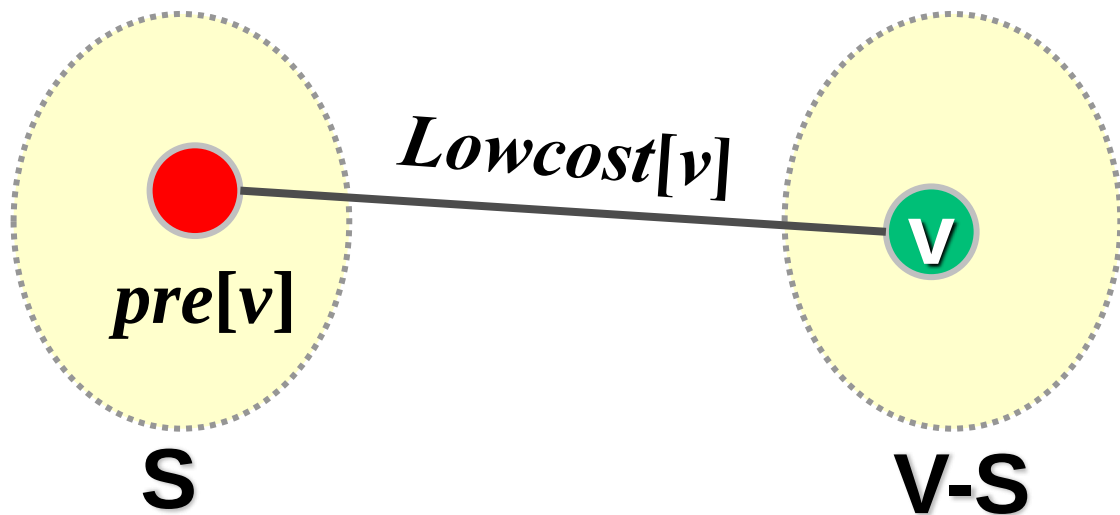
- ① 每次选择 **Lowcost** 值最小的顶点 v , 把边 $(pre[v], v)$ 加入最小支撑树, 把点 v 放入集合 S .
- ② 更新顶点 v 的邻居 w 的 $Lowcost$ 值: 若 $G[v][w] < Lowcost[w]$, 则 $Lowcost[w] \leftarrow G[v][w]$. $pre[w] \leftarrow v$.



将 v 加入集合 S 后将产生新的跨集合边 (v, w)

Prim算法实现——准备工作

- $Lowcost[v]$: 顶点 v 到集合 S 的最小跨边的权值。初始时 $Lowcost[u]=0$, $Lowcost[i]=\infty$ ($\forall i \neq u$), 其中 u 为起点;
- $pre[v]$: 顶点 v 到集合 S 的最小跨边在集合 S 中的端点, 初值-1;
- $S[v]$: 顶点 v 是否在集合 S 中, 初值0。
- 若存在 MST , 求出 MST 中的边并返回边权和, 否则 (图不连通) 求出包含起点 u 的连通子图的 MST 并返回其边权和。
- 采用邻接矩阵 G 存图。

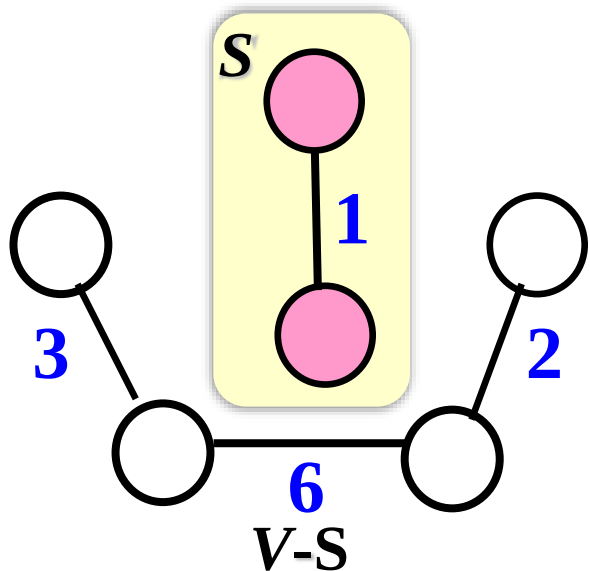


Prim算法的实现

```
int Prim(int G[N][N], int n, int u, int Lowcost[], int pre[]){
    int S[N]={0}, sum=0; // sum存MST边权之和, 作为函数返回值
    for(int i=1; i<=n; i++){ pre[i]=-1; Lowcost[i] = (i==u)? 0: INF; }
    for(int i=1; i<=n; i++){ //把n个点逐个加入集合S
        int v=FindMin(S, Lowcost, n); //从不在S中的点里选Lowcost值最小的点
        if(v==-1) return sum; //不存在跨边, 图不连通
    }
    //未完待续...
```

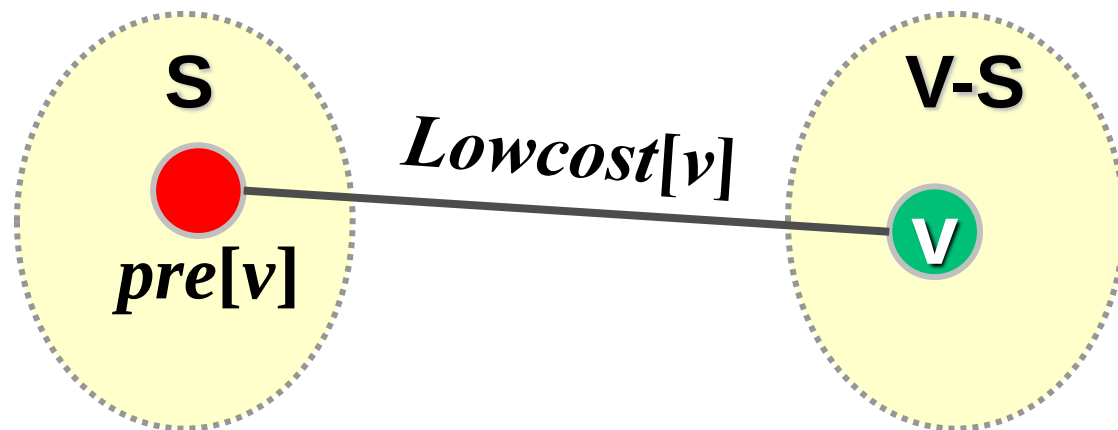
```
int FindMin(int S[], int Lowcost[], int n){
    int v=-1, min=INF;
    for(int i=1; i<=n; i++){
        if(S[i]==0 && Lowcost[i]<min){
            min=Lowcost[i]; v=i;
        }
    }
    return v;
}
```

时间复杂度
 $O(n)$



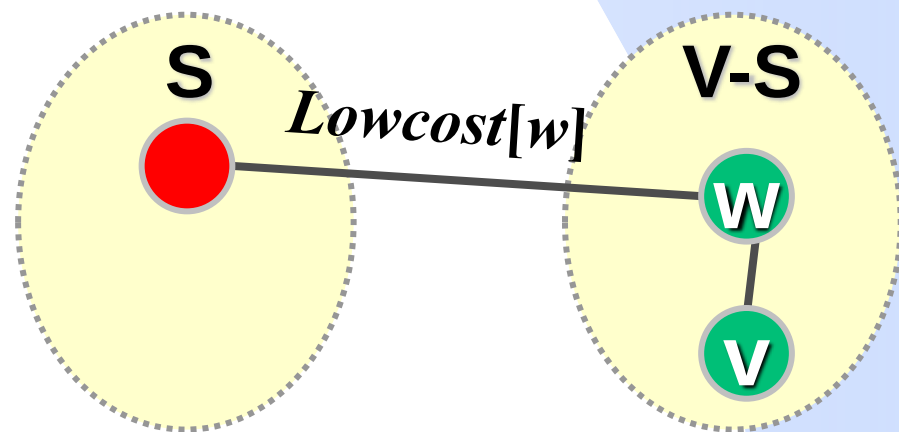
Prim算法的实现

```
int Prim(int G[N][N], int n, int u, int Lowcost[], int pre[]){
    int S[N]={0}, sum=0; // sum存MST边权之和, 作为函数返回值
    for(int i=1; i<=n; i++){pre[i]=-1; Lowcost[i] = (i==u)? 0: INF;}
    for(int i=1; i<=n; i++){ //把n个点逐个加入集合S
        int v=FindMin(S, Lowcost, n); //从不在S中的点里选Lowcost值最小的点
        if(v==-1) return sum; //不存在跨边, 图不连通
        S[v]=1; sum+=Lowcost[v]; //找到一条MST的边 (pre[v],v), v加入集合S
    }
    //未完待续...
```



Prim算法的实现

```
int Prim(int G[N][N], int n, int u, int Lowcost[], int pre[]){
    int S[N]={0}, sum=0; // sum存MST边权之和, 作为函数返回值
    for(int i=1; i<=n; i++){pre[i]=-1; Lowcost[i] = (i==u)? 0: INF;}
    for(int i=1; i<=n; i++){
        //把n个点逐个加入集合S
        int v=FindMin(S, Lowcost, n); //从不在S中的点里选Lowcost值最小的点
        if(v==-1) return sum; //不存在跨边, 图不连通
        S[v]=1; sum+=Lowcost[v]; //找到一条MST的边 (pre[v],v), v加入集合S
    }
    //未完待续...
```

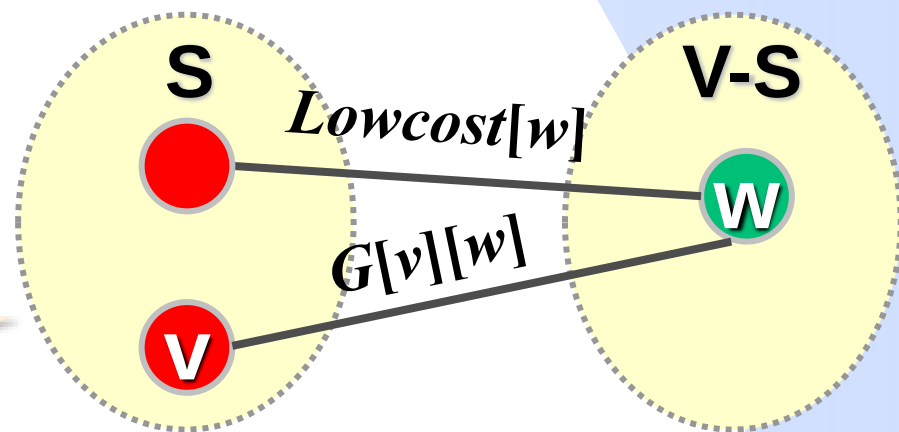


Prim算法的实现

```
int Prim(int G[N][N], int n, int u, int Lowcost[], int pre[]){
    int S[N]={0}, sum=0; // sum存MST边权之和, 作为函数返回值
    for(int i=1; i<=n; i++){pre[i]=-1; Lowcost[i] = (i==u)? 0: INF;}
    for(int i=1; i<=n; i++){ //把n个点逐个加入集合S
        int v=FindMin(S, Lowcost, n); //从不在S中的点里选Lowcost值最小的点
        if(v==-1) return sum; //不存在跨边, 图不连通
        S[v]=1; sum+=Lowcost[v]; //找到一条MST的边 (pre[v],v), v加入集合S
        for(int w=1; w<=n; w++) //更新v邻接顶点的Lowcost和pre值
            if(S[w]==0 && G[v][w]<Lowcost[w]){
                Lowcost[w]=G[v][w]; pre[w]=v;
            }
    }
    return sum;
}
```

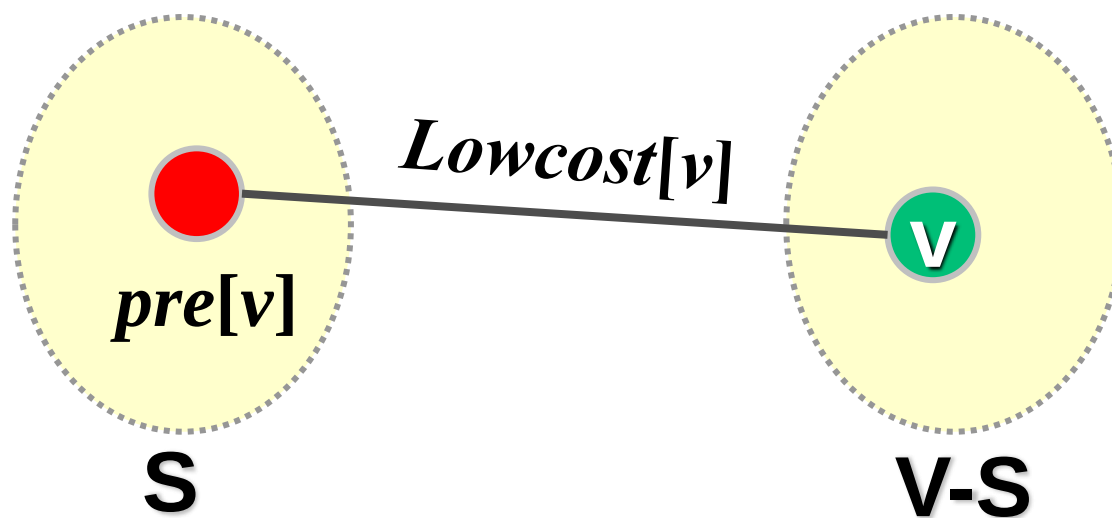
时间复杂度
 $O(n^2)$

将v加入集合S后可能
产生新跨边(v,w)



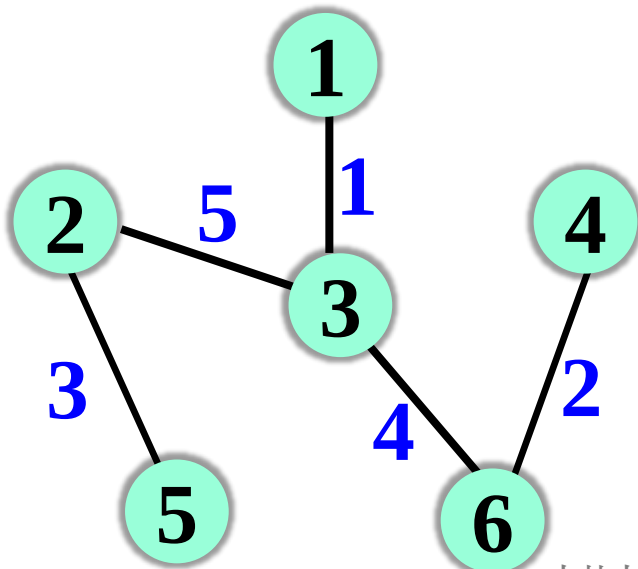
如何使用pre数组——直接输出MST的边

```
void OutputMST(int Lowcost[],int pre[],int n){ //输出MST中的边
    for(int v=1; v<=n; v++) //输出MST中的边 (pre[v],v)及权值
        if(pre[v]!=-1)
            printf("edge:%d-%d,weight:%d\n",pre[v],v,Lowcost[v]);
}
```



如何使用pre数组——构建MST的邻接表

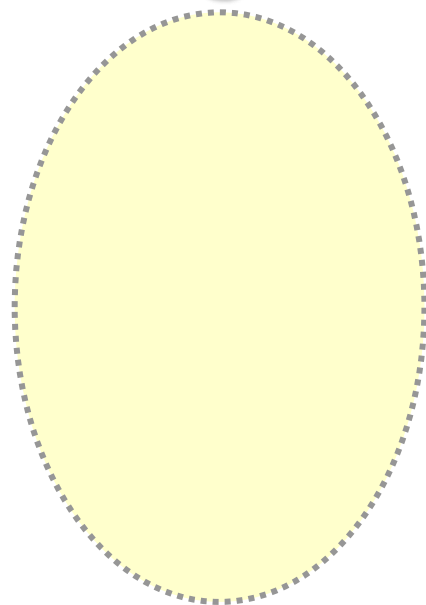
```
void BuildMST(Vertex Head[], int n, int Lowcost[], int pre[]){
    //通过Lowcost和pre数组构建MST的邻接表
    for(int v=1; v<=n; v++){
        Edge *p = Head[v].adjacent; //将v的邻接顶点pre[v]插入邻接表
        Head[v].adjacent = new Edge(pre[v], Lowcost[v], p);
        p = Head[pre[v]].adjacent; //将pre[v]的邻接顶点v插入邻接表
        Head[pre[v]].adjacent = new Edge(v, Lowcost[v], p);
    }
}
```



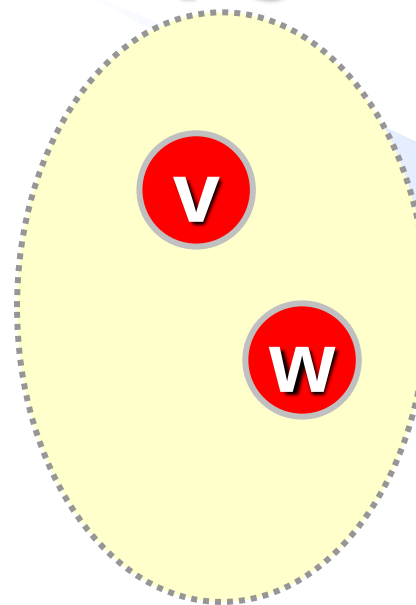
```
struct Edge{
    int VerAdj;
    int cost;
    Edge *link=NULL;
    Edge(int a, int c, Edge *p){
        VerAdj=a; cost=c; link=p;
    }
};
```

Dijkstra算法 VS Prim算法

S



V-S



Dijkstra算法

从集合 $V-S$ 中选择 $Dist$ 值最小的顶点放入集合 S ，并更新其邻居的 $Dist$ 值

Prim算法

从集合 $V-S$ 中选择 $Lowcost$ 值最小的顶点放入集合 S ，并更新其邻居的 $Lowcost$ 值

```

void Dijkstra(int G[N][N],int n,int u,int dist[],int pre[]){
    int S[N]={0};
    for(int i=1;i<=n;i++){dist[i]=(i==u)?0:INF; pre[i]=-1;}
    for(int i=1;i<=n;i++){
        int v = FindMin(S, dist, n);
        if(v==-1) return;
        S[v]=1;
        for(int w=1; w<=n; w++){
            if(S[w]==0 && dist[v]+G[v][w]<dist[w]){
                dist[w]=dist[v]+G[v][w]; pre[w]=v;
            }
        }
    }
}

```

Dijkstra算法 VS Prim算法

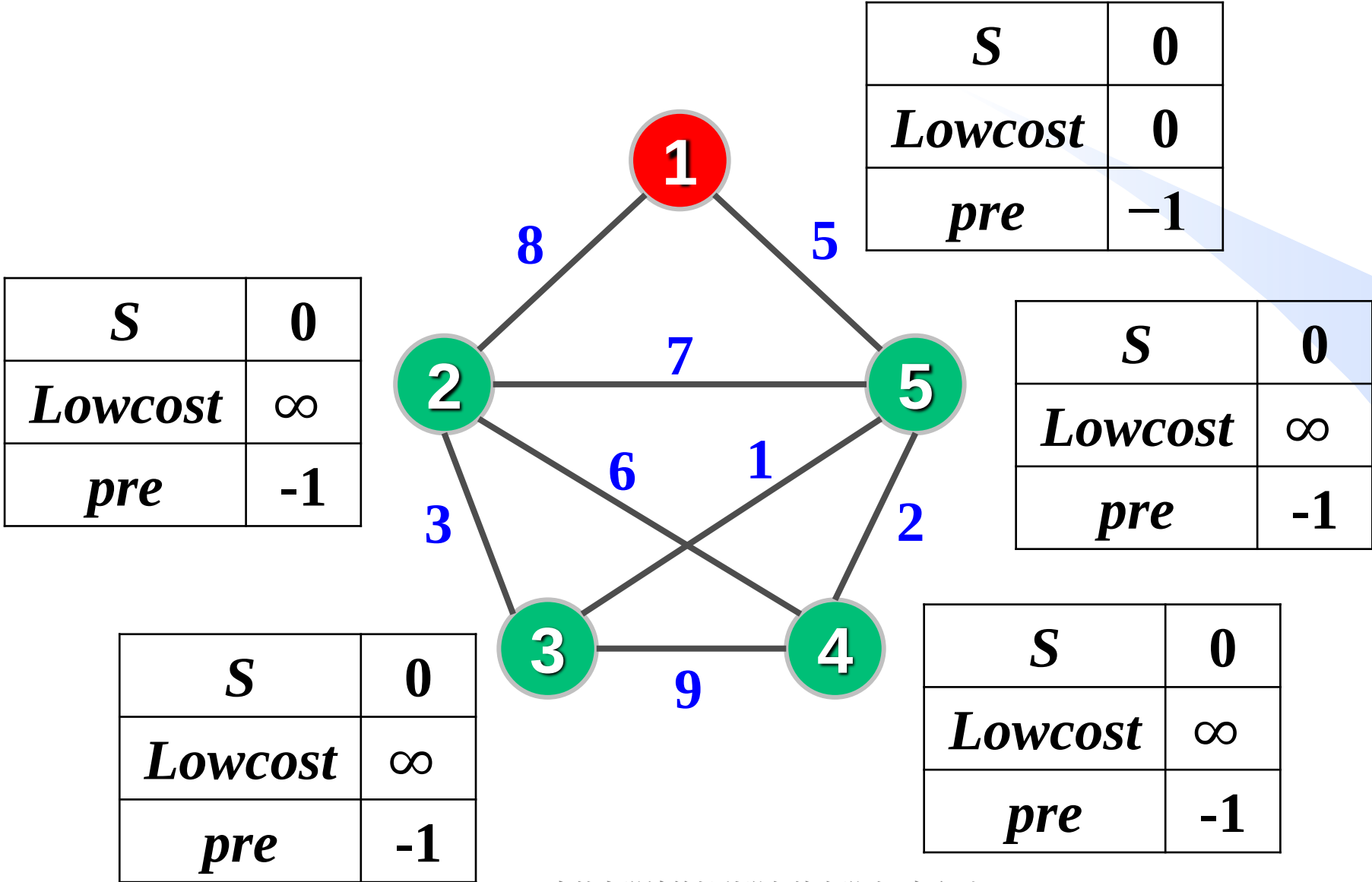
仔细体会两段代码的细微不同，
会加深你对这两个算法的理解

```

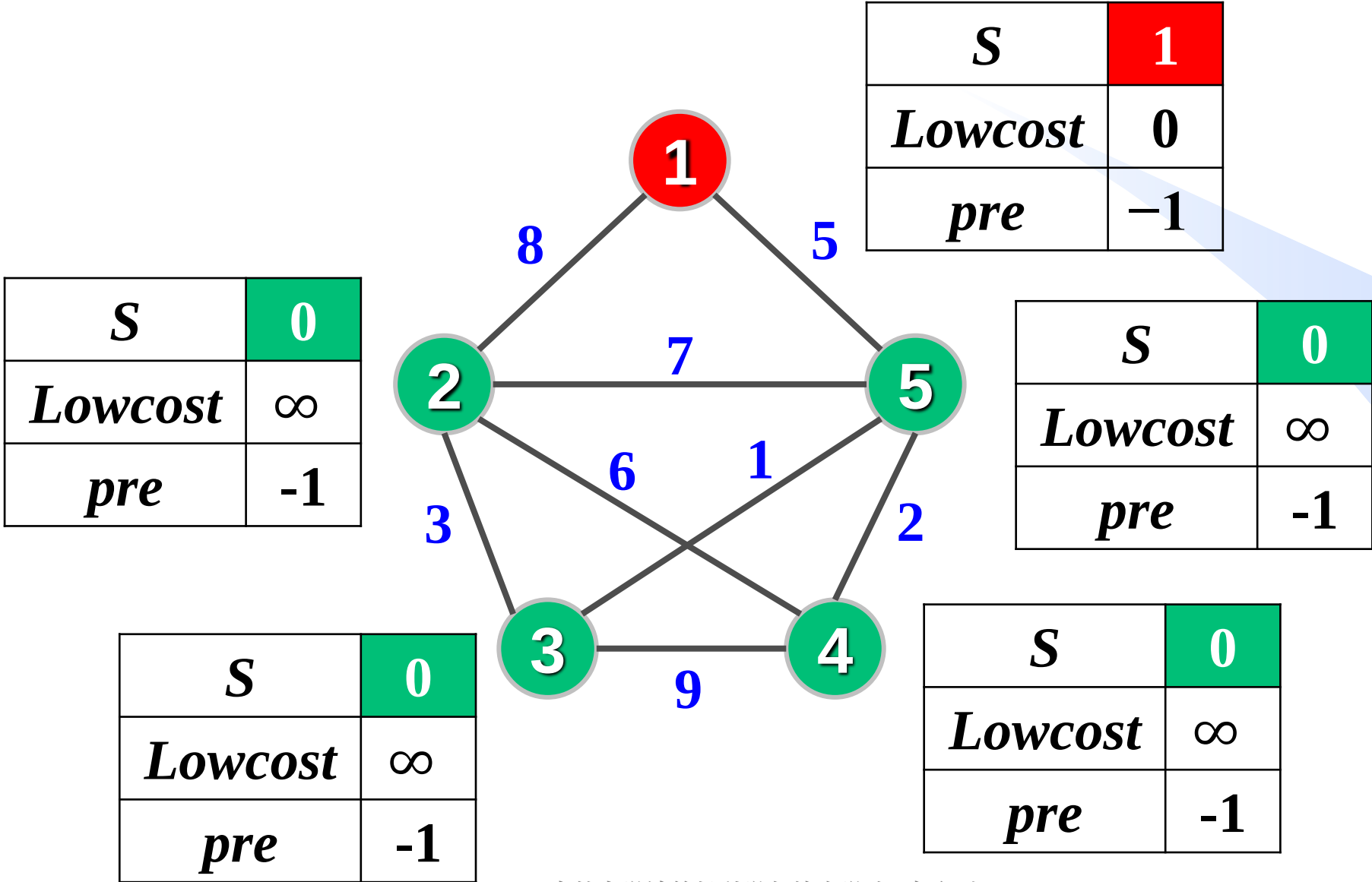
int Prim(int G[N][N],int n,int u,int Lowcost[],int pre[]){
    int S[N]={0}, sum=0;
    for(int i=1;i<=n;i++){Lowcost[i]=(i==u)?0:INF; pre[i]=-1;}
    for(int i=1;i<=n;i++){
        int v = FindMin(S, Lowcost, n);
        if(v==-1) return sum;
        S[v]=1; sum+=Lowcost[v];
        for(int w=1; w<=n; w++){
            if(S[w]==0 && G[v][w]<Lowcost[w]){
                Lowcost[w]=G[v][w]; pre[w]=v;
            }
        }
    }
    return sum;
}

```

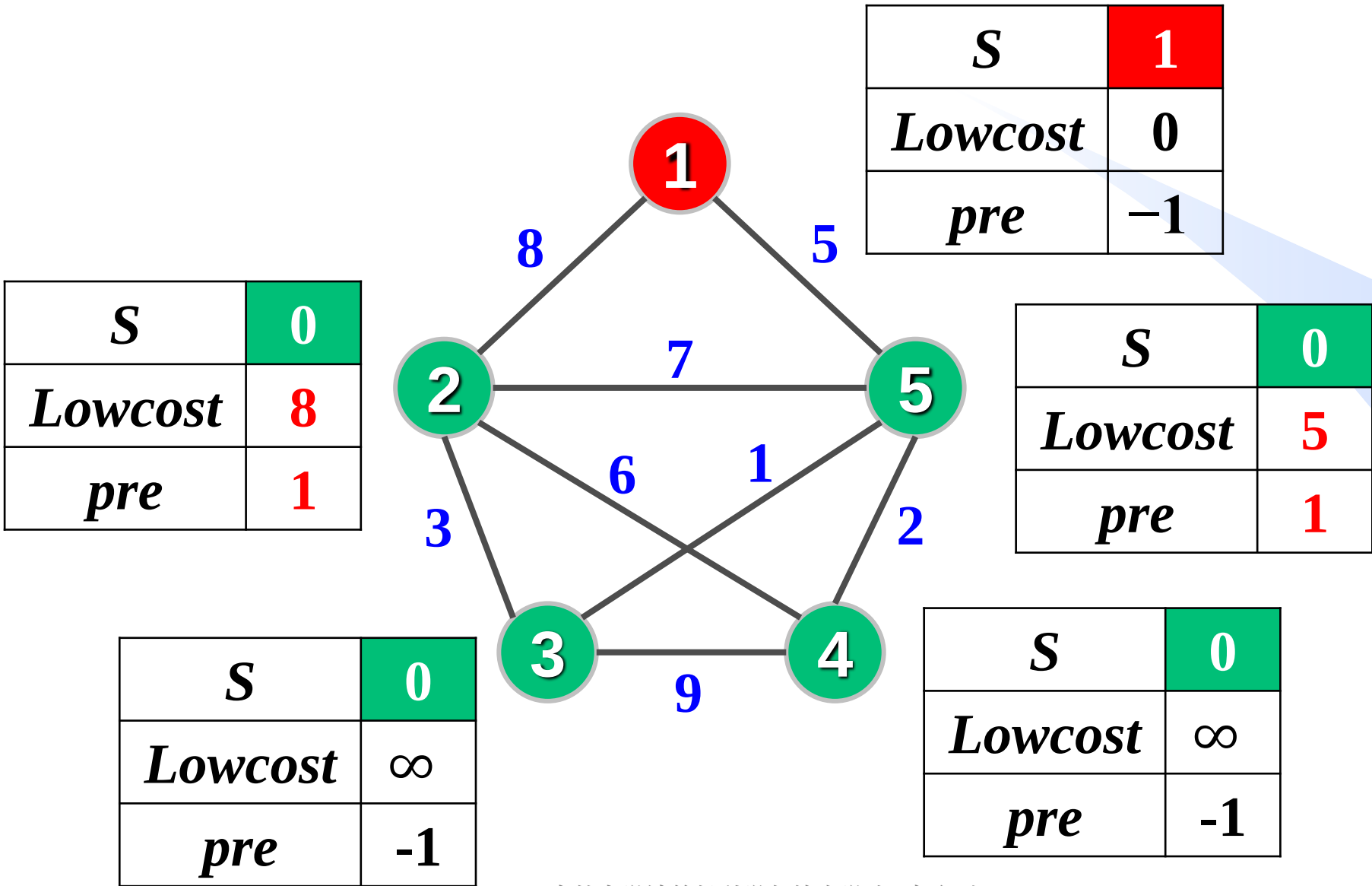
Prim算法举例



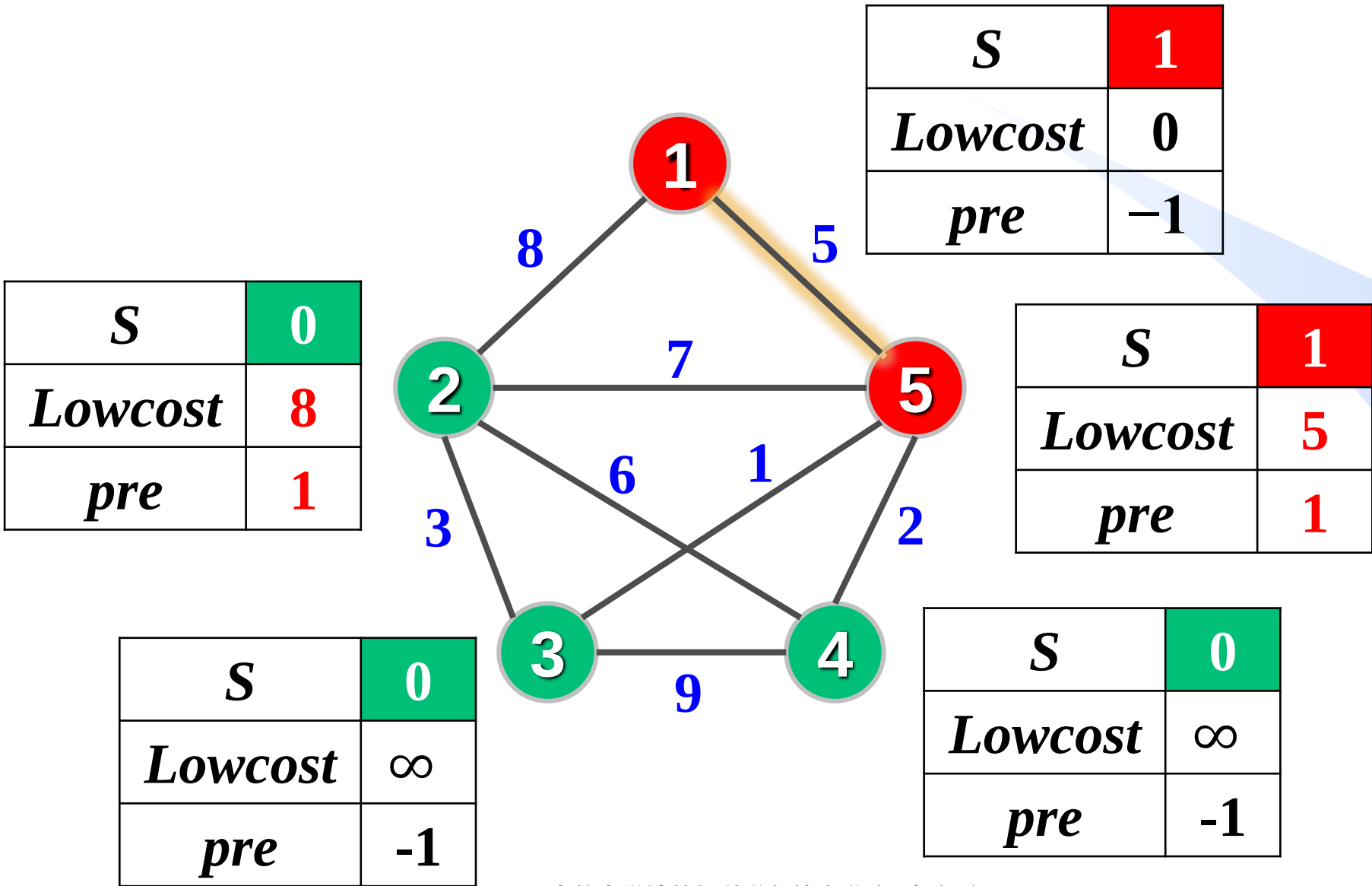
Prim算法举例



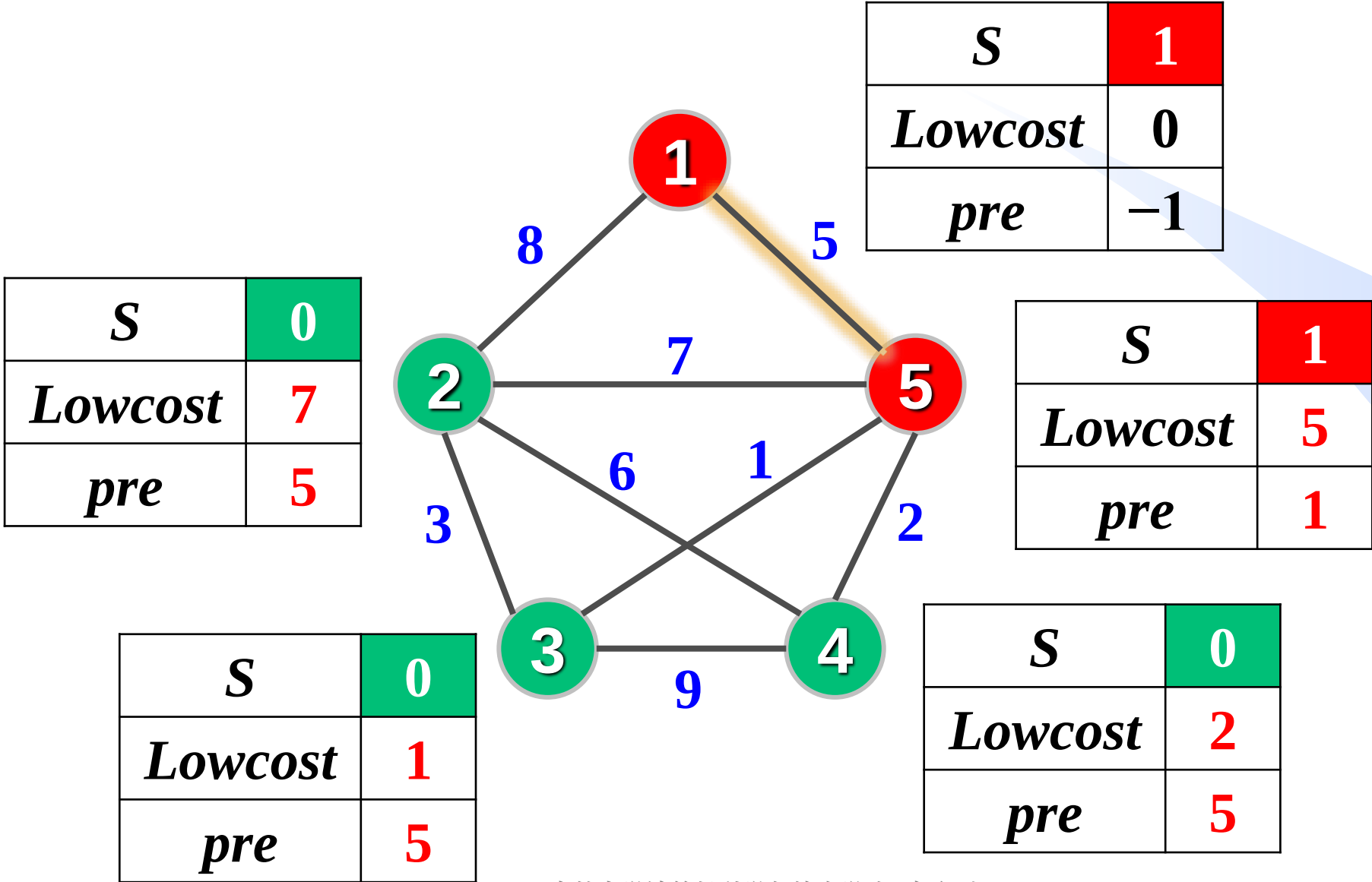
Prim算法举例



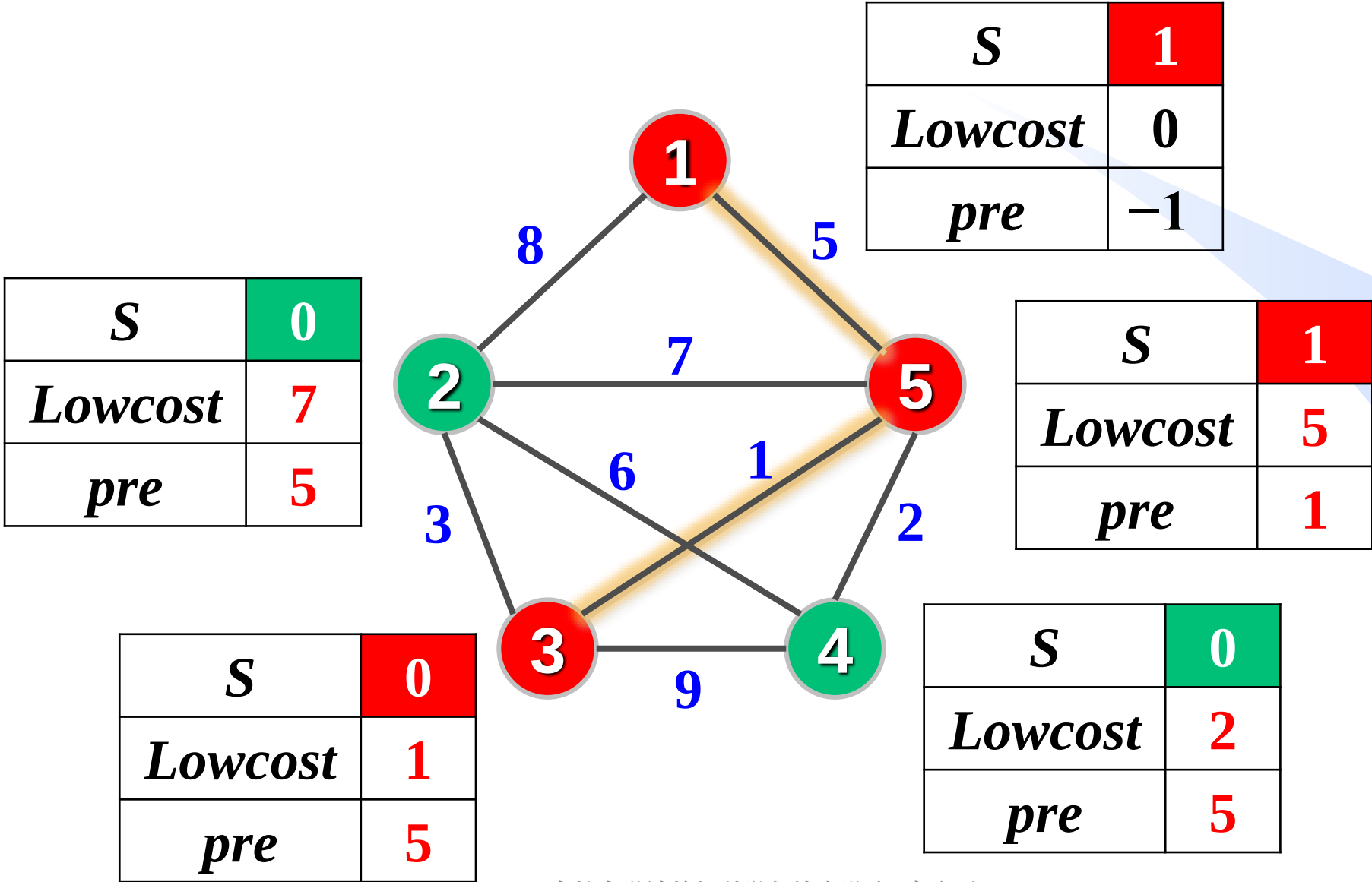
Prim算法举例



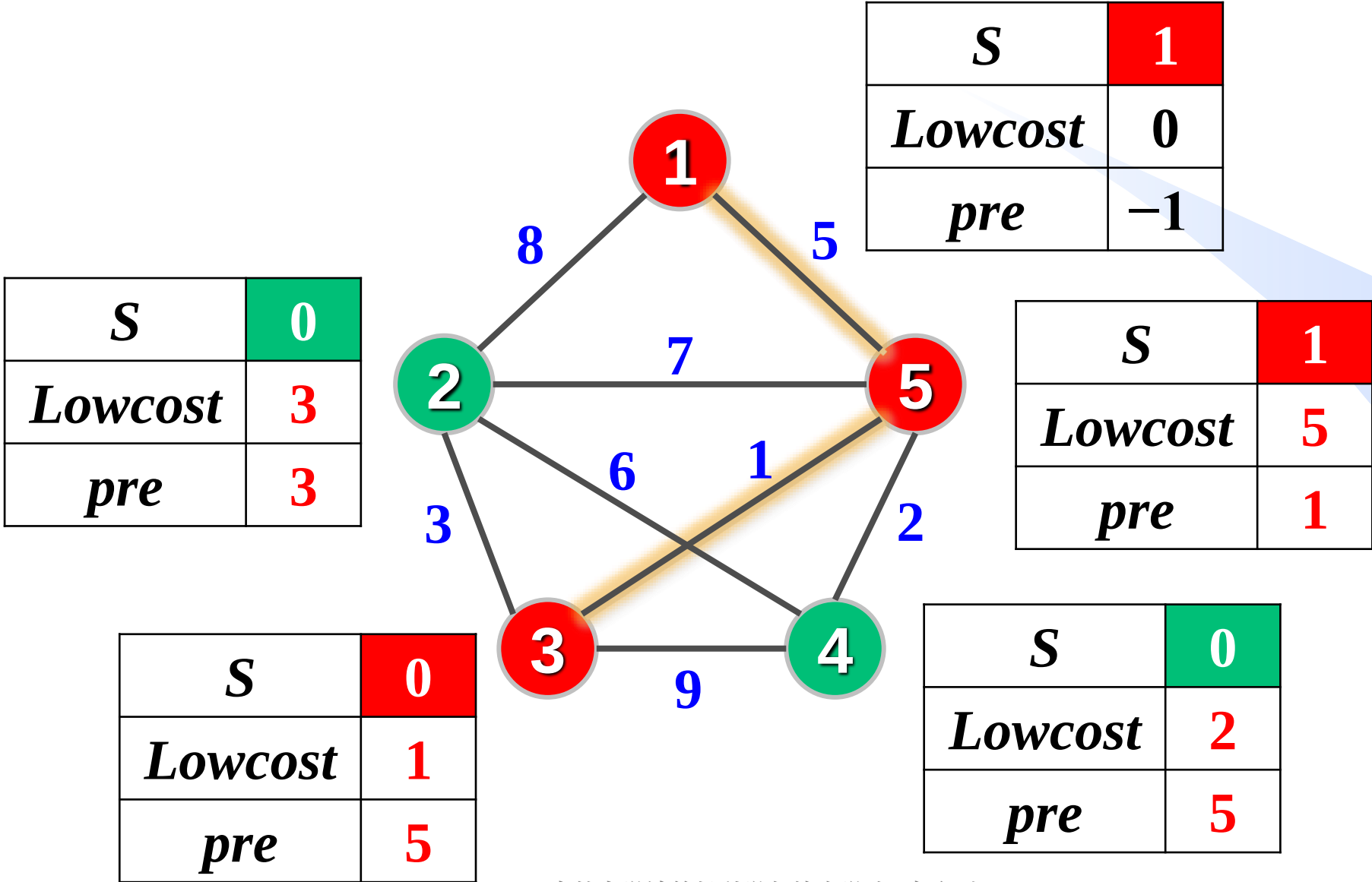
Prim算法举例



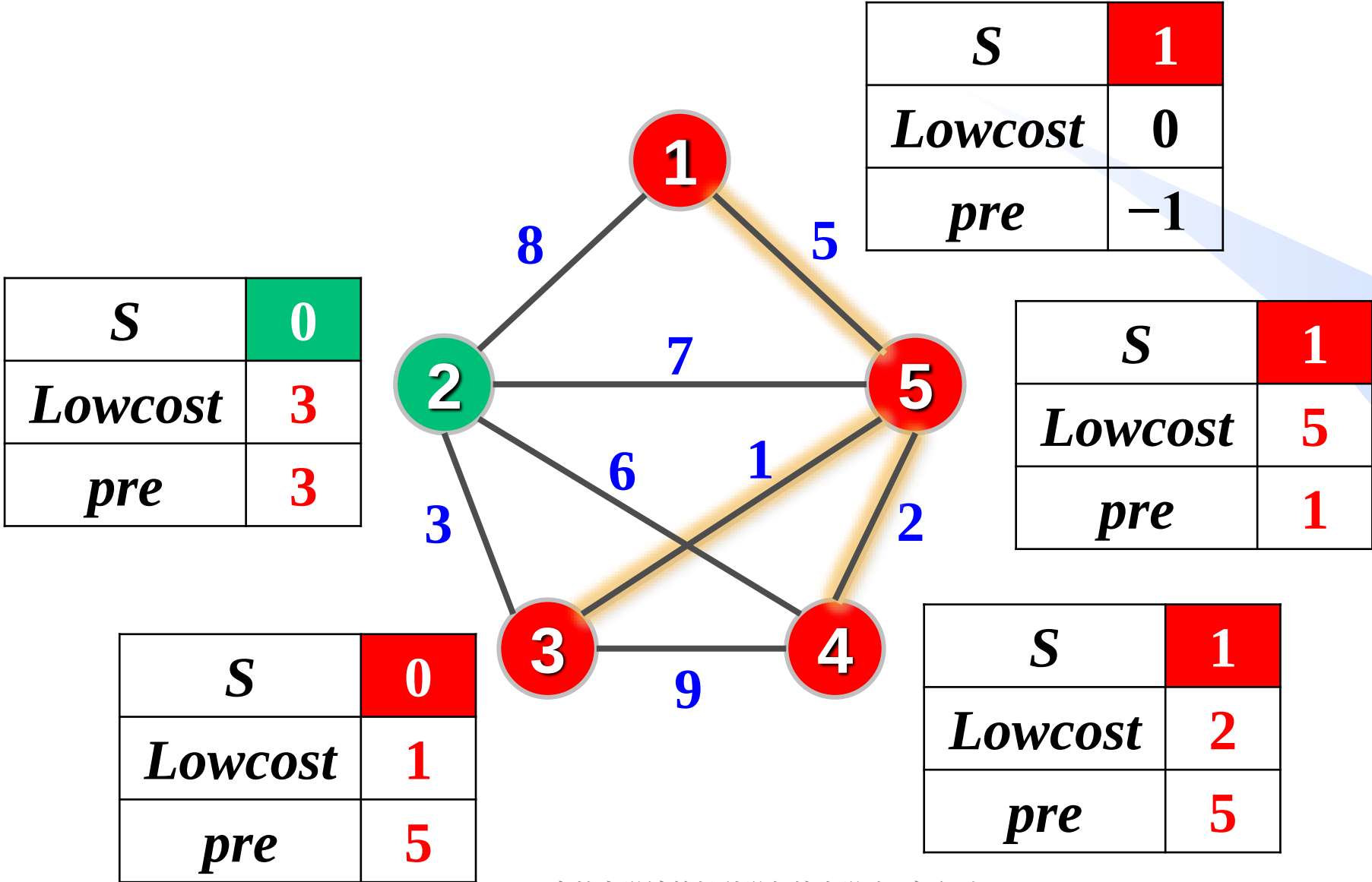
Prim算法举例



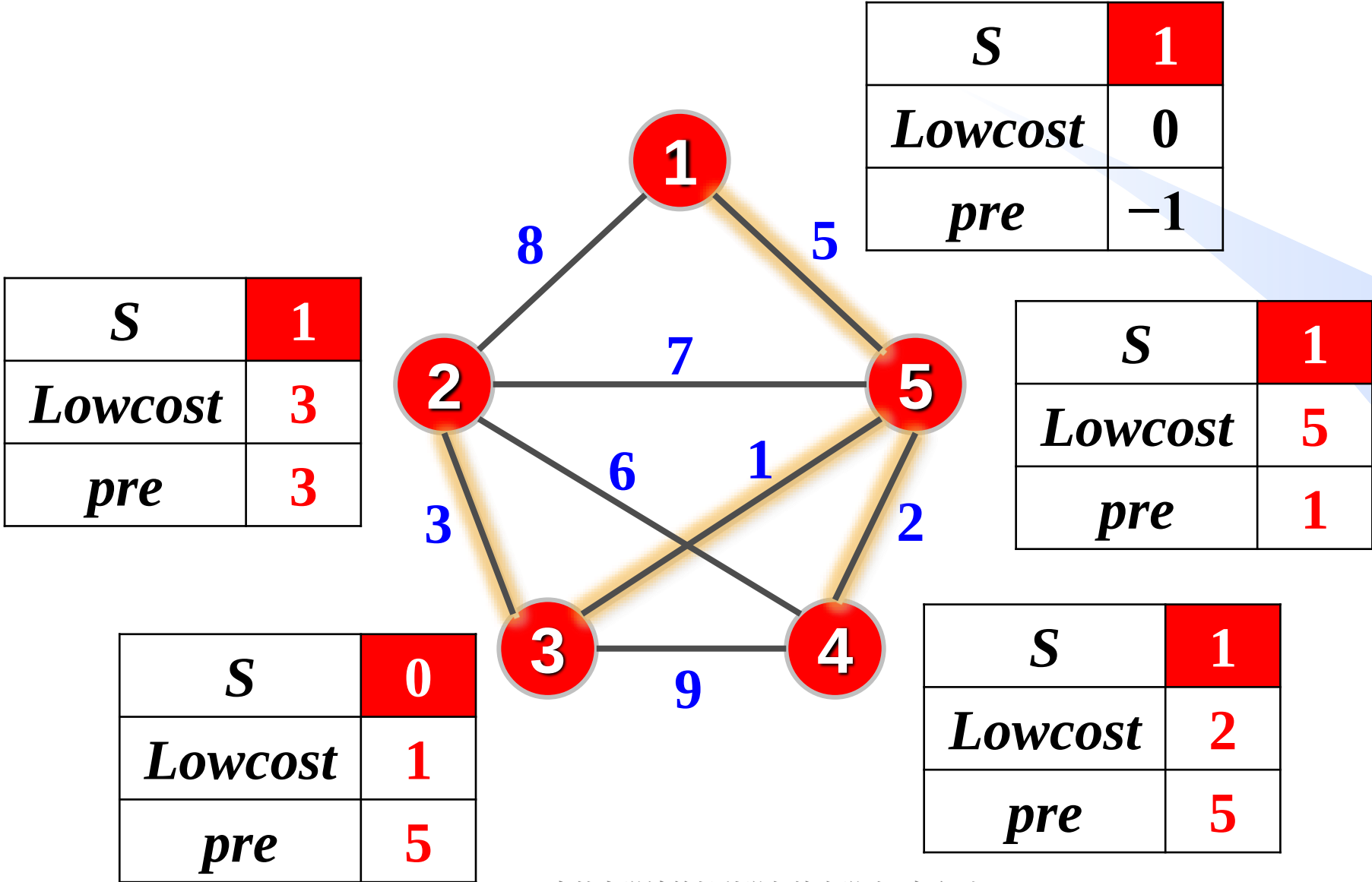
Prim算法举例

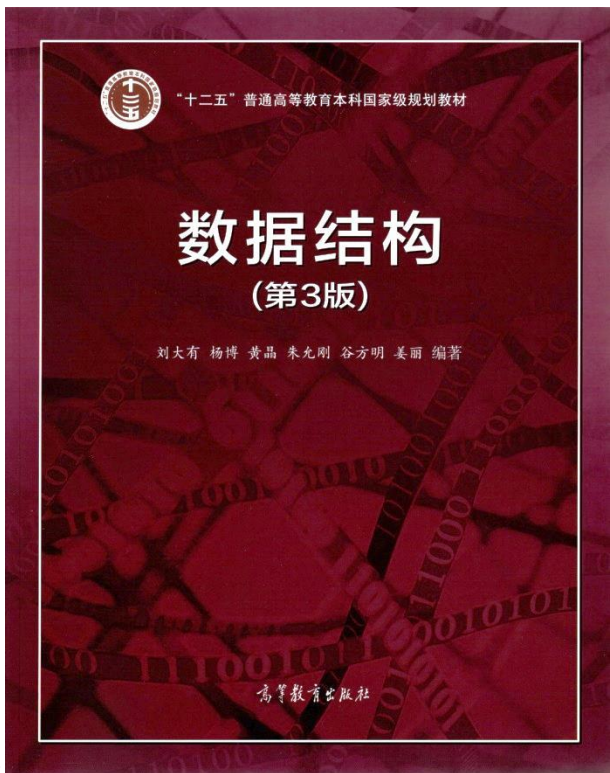


Prim算法举例



Prim算法举例





最小支撑树及图应用

- 最小支撑树的概念
- Prim算法
- **Kruskal算法**
- 强连通分量
- 图的其他应用举例

数据之美
结构之美
算法之道

zhuyungang@jlu.edu.cn

Kruskal算法（逐边加入）



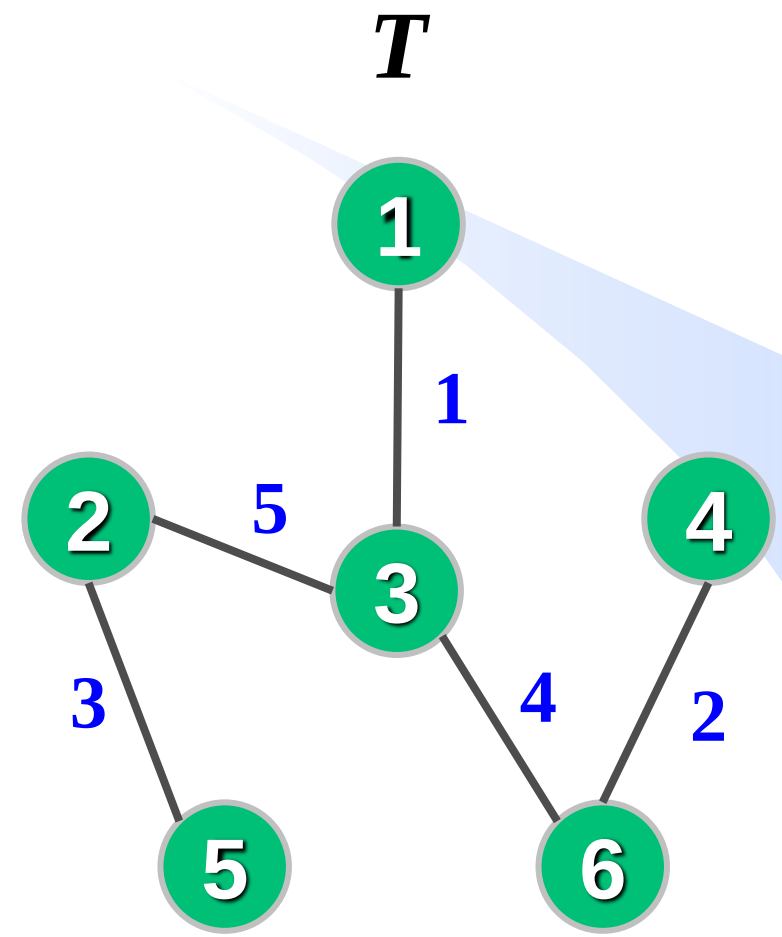
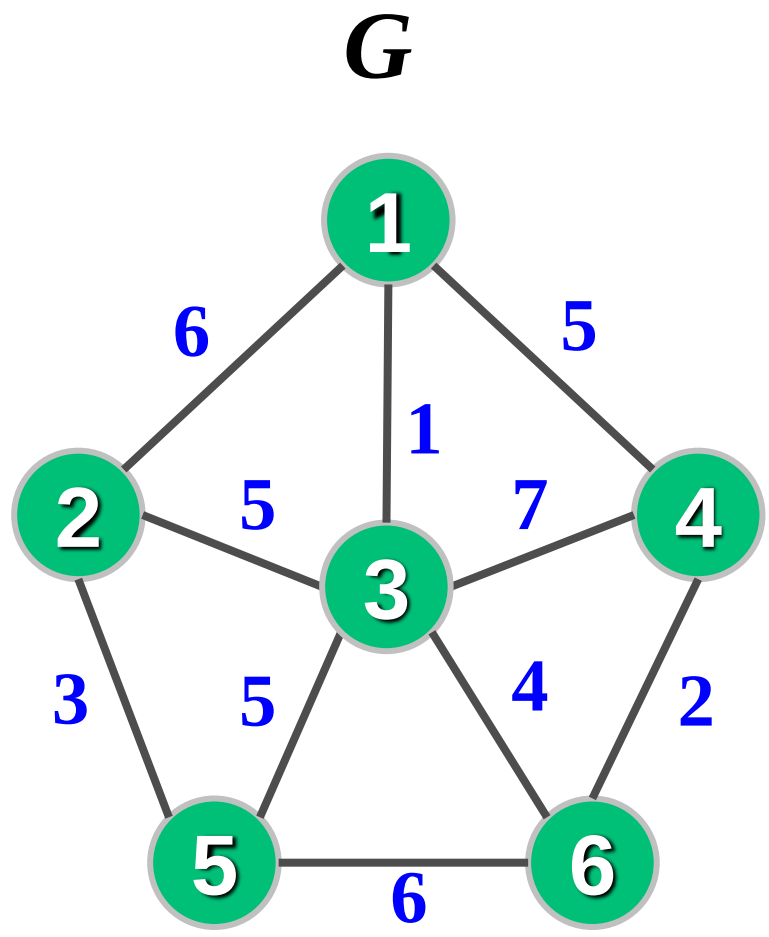
Joseph Kruskal
(1928-2010)

普林斯顿大学博士
贝尔实验室研究员
美国统计学会会士

Kruskal算法（逐边加入）

设连通图 $G=(V, E)$ ，令 T 为 G 的最小支撑树，初始时 T 中有 G 的 n 个顶点和0条边，可看成 n 个连通分量。

- ① 在 G 中选择权值最小的边，并将此边从 G 中删除；
- ② 若该边加入 T 后不产生环（即此边的两个端点在 T 的不同连通分量中），则将此边加入 T 中，从而使 T 减少一个连通分量，否则本步骤无操作。
- ③ 重复① ②直至 T 中仅剩一个连通分量。



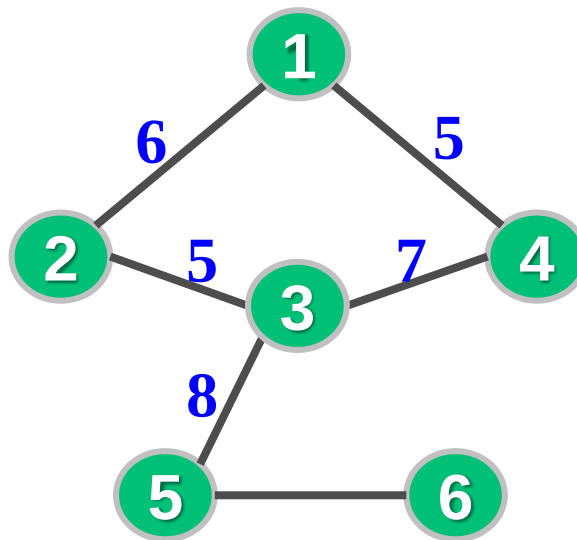
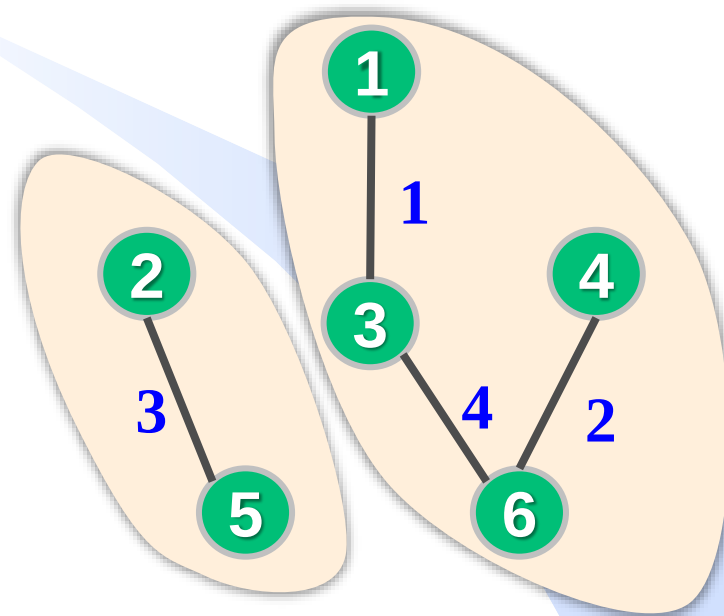
Kruskal算法的实现

- 选权值最小的边：先对所有边按权值递增排序。
- 判环：并查集（连通分量 $\Leftrightarrow$ 集合）



顶点 u 和 v 不在同一连通分量，在MST中加入边 (u,v) 不会产生环，故边 (u,v) 是MST的边

```
if (Find(u) != Find(v)) {  
    将边 $(u,v)$ 加入最小支撑树  
    Union(u, v).  
}
```

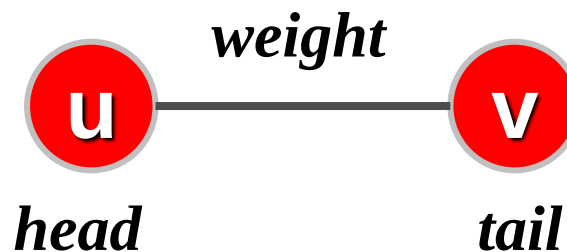
 G  T

合并 u 和 v 所在集合，使二者在同一连通分量里

Kruskal算法的实现

- 如何存图：只需存储图中边的信息，使用边集数组。

```
struct Edge{  
    int head;  
    int tail;  
    int weight;  
};  
Edge E[1010];
```



- 若图的最小支撑树存在，输出最小支撑树的边并返回边权之和，否则（图不连通）返回 ∞

Kruskal算法的实现

```

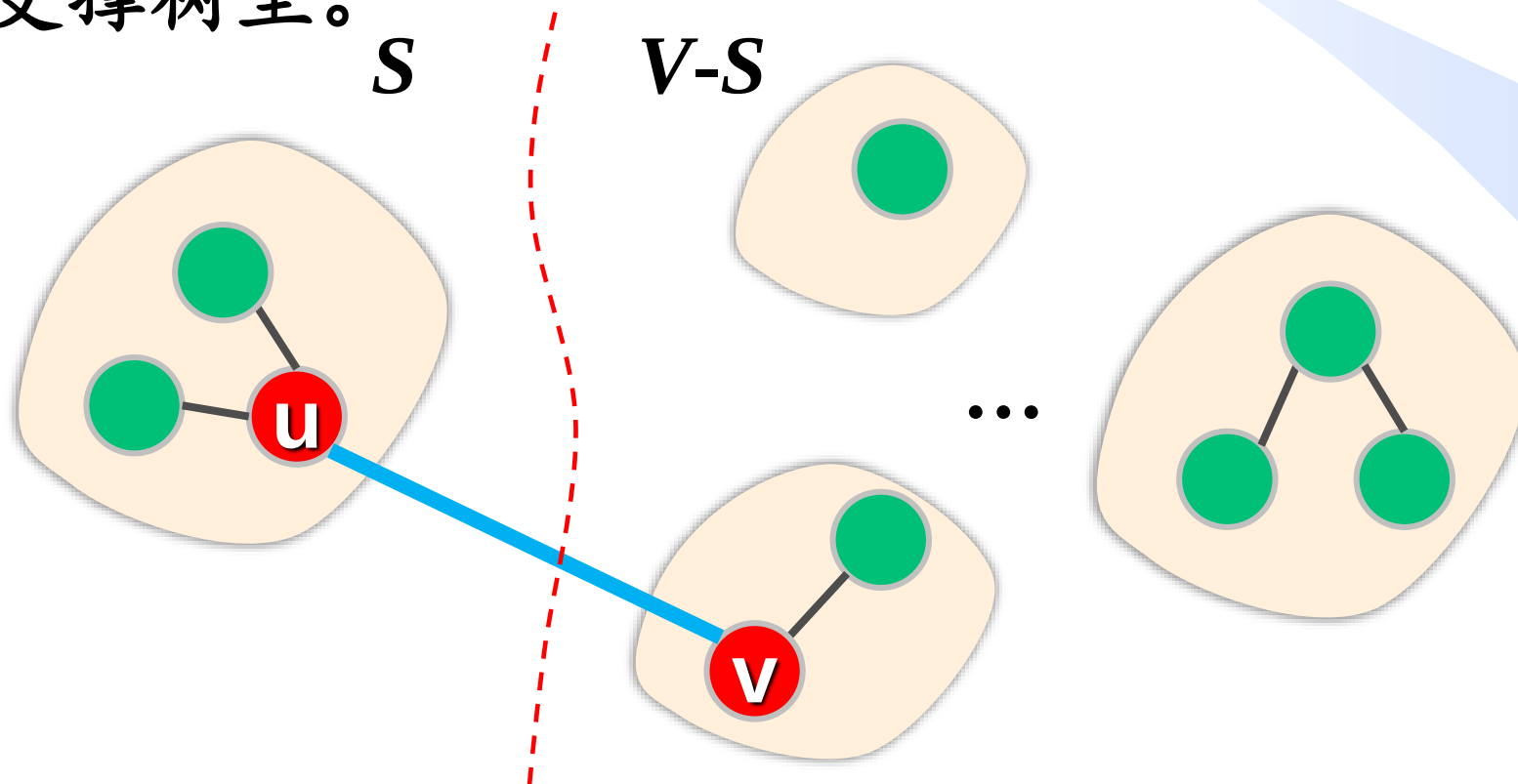
int Kruskal(Edge E[],int n,int e){ //E存图的边,n为顶点数,e为边数
    for(int i=1;i<=n;i++) Make_set(i); //初始化
    Sort(E,e); //对边按权值递增排序,时间复杂度O(eloge)
    int sum=0,k=0; //sum为mst边权和,k为已找到的mst边的条数
    for(int i=0;i<e;i++){ //从小到大依次扫描每条边
        int u=E[i].head, v=E[i].tail, w=E[i].weight;
        if(Find(u) != Find(v)){ //边(u,v)是最小生成树的一条边
            printf("%d-%d",u,v); k++; sum+=w;
            Union(u,v); //合并集合
        }
        if(k==n-1) return sum; //成功找到mst的全部n-1条边
    }
    return INF;
}

```

时间复杂度
 $O(e \log e)$

Kruskal算法的正确性依据

设Kruskal算法某次选出的最小边为 (u,v) ，其中 u 所在的连通分量的顶点集合为 S 。则边 (u,v) 是集合 S 和 $V-S$ 的最小跨边，故必在最小支撑树里。



Prim算法与Kruskal算法

贪心算法

算法	时间复杂度	适用场合
Prim	$O(n^2)$	稠密图
Kruskal	$O(e \log e)$	稀疏图

判断：相对于 Kruskal 算法， Prim 算法更适宜于稀疏图。
【2023年清华大学考研题】

主要研究进展举例

时间	时间复杂度	提出者
1975	$O(e \log \log n)$	姚期智
2002	$O(e \cdot \alpha(n, e))$	Pettie Ramachandran



姚期智 (1946-)

图灵奖获得者 (迄今唯一华人获奖者)
原Stanford, Princeton, UC Berkeley教授

现清华大学教授

中国科学院院士

美国科学院外籍院士

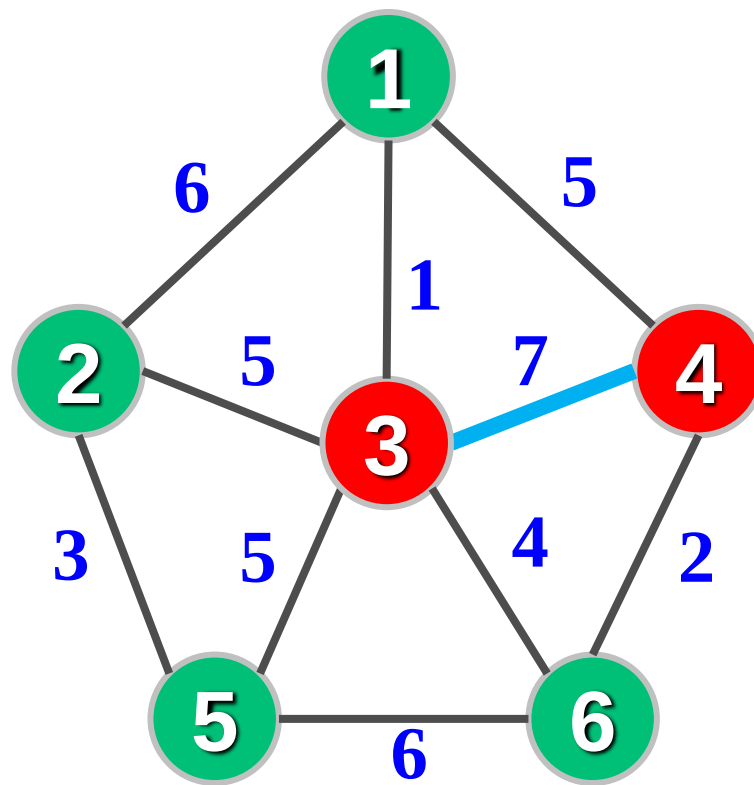
2016年放弃美国国籍成为中国公民

课下思考

- 图 G 的最小支撑树一定包含 G 中的最小边么？一定包含 G 中第2小的边么？一定包含 G 中第3小的边么？
- 若图中边权互异，其最小支撑树是否唯一？
- 若图中有权值相同的边，其最小支撑树一定不唯一么？

拓展问题

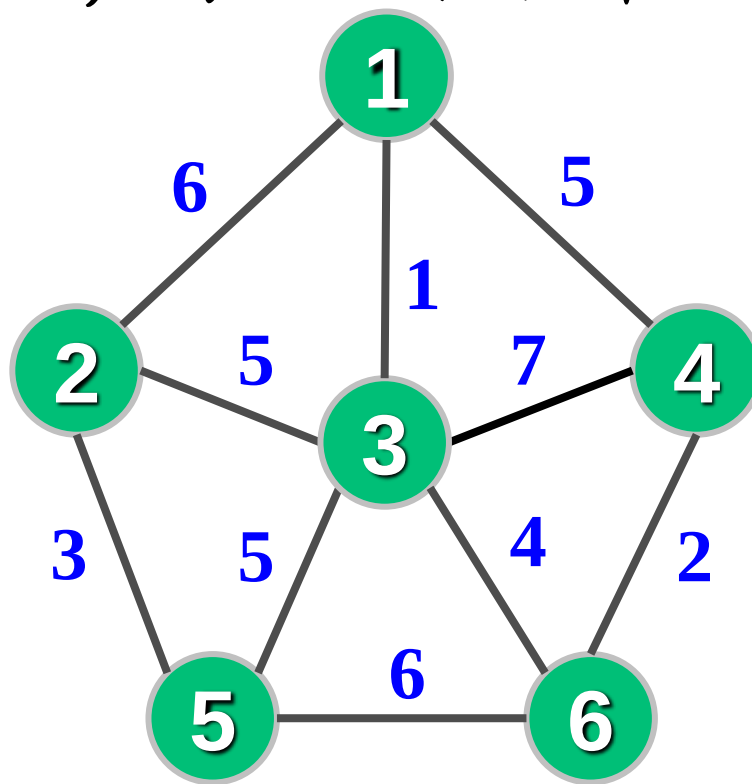
➤ 求图的必须包含某些边的最小支撑树。



拓展问题

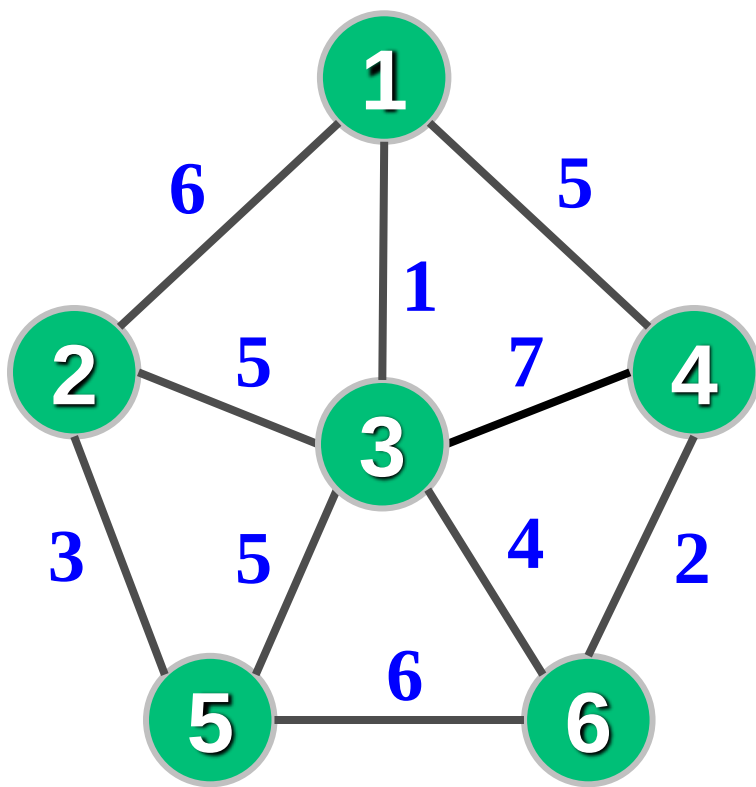
➤ 最大支撑树

- ✓ 边权值取反，求最小支撑树
- ✓ 修改Kruskal算法，每次选权值最大的边



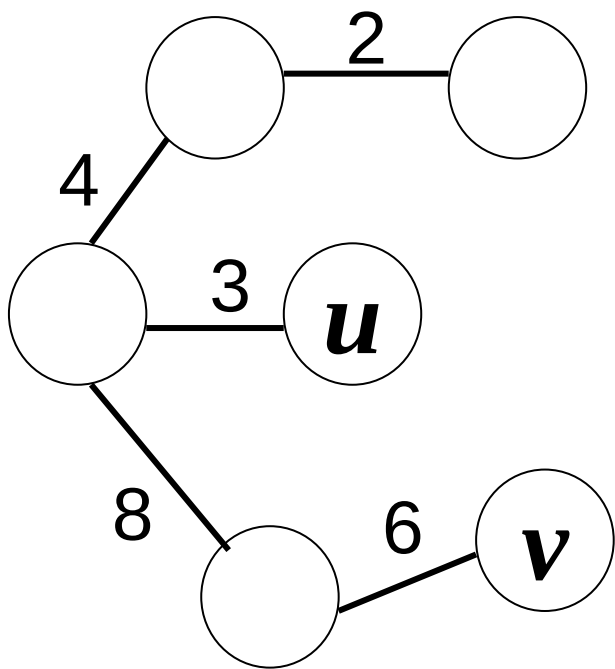
拓展问题

- 最小乘积支撑树：给定正权图，求图的一棵支撑树，其边的权值乘积最小。



拓展问题

增量最小支撑树：假设已经获得某个无向带权连通图 G 的最小支撑树 T ，将一条新边 e 加入 G （图 G 的顶点集不变），给出一个寻找新图 $G+e$ 的最小支撑树的高效算法，时间复杂度要求 $O(n)$ ， n 为图 G 的顶点数目。【2018级期末考试题，14分】



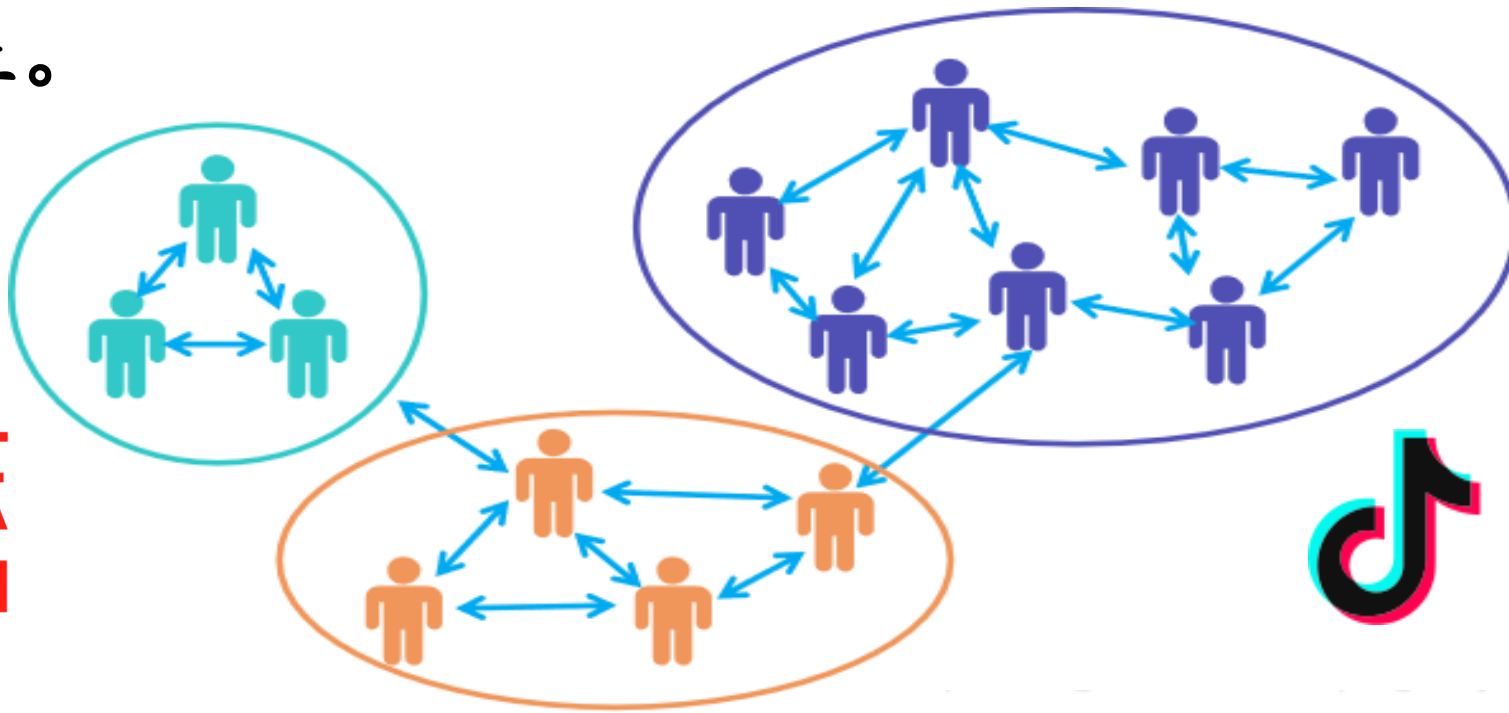
算法思想：设边 $e=(u, v)$ ，加入该边后，会使最小支撑树 T 中产生环，去掉该环中权值最大的边，就是新图的最小支撑树。具体实现：在最小支撑树 T 中求顶点 u 到 v 的路径（可以 u 为起点进行DFS，遍历到顶点 v 停止），记录该路径中权值最大的边，若该边权值大于 e ，则删去此边并加入边 e 作为新MST，否则MST不变。

MST的其他应用举例——聚类

- 用户聚类：根据不同用户的兴趣，对所有用户聚类，相似的用户分到一组（簇），对不同组的用户推荐不同商品。
- 每个用户对应图中一个顶点，边权反映用户购买习惯的相似性。设置参数 k ，执行Kruskal算法直至剩下 k 个连通分量（ k 组用户）。或者设置一个阈值，选出的最小边权高于阈值算法就终止。

淘宝网
Taobao.com

京东
JD.COM



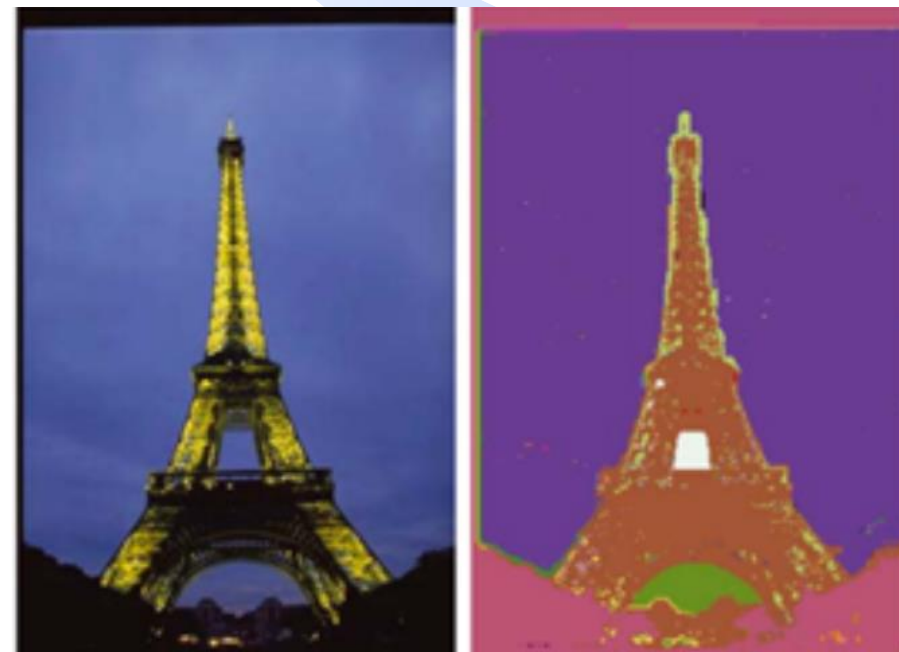
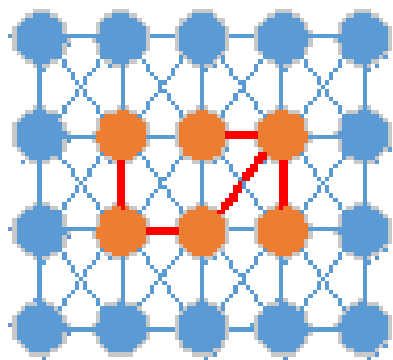
Tmall
理想生活上天猫

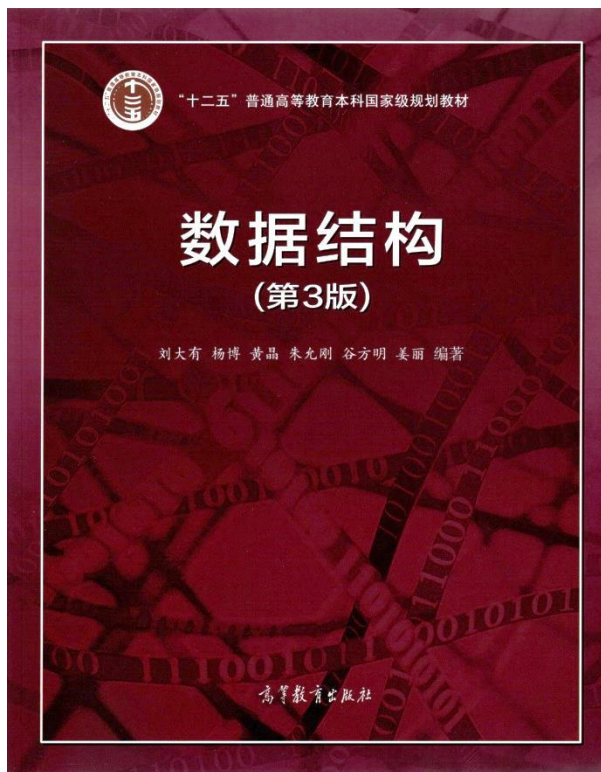
抖音

MST的其他应用举例——图像分割

D

- 每个像素对应一个顶点，边权表示两个像素颜色、亮度等的相似度，用Kruskal算法把颜色、亮度相似的像素分到一组（簇）。





最小支撑树及图应用

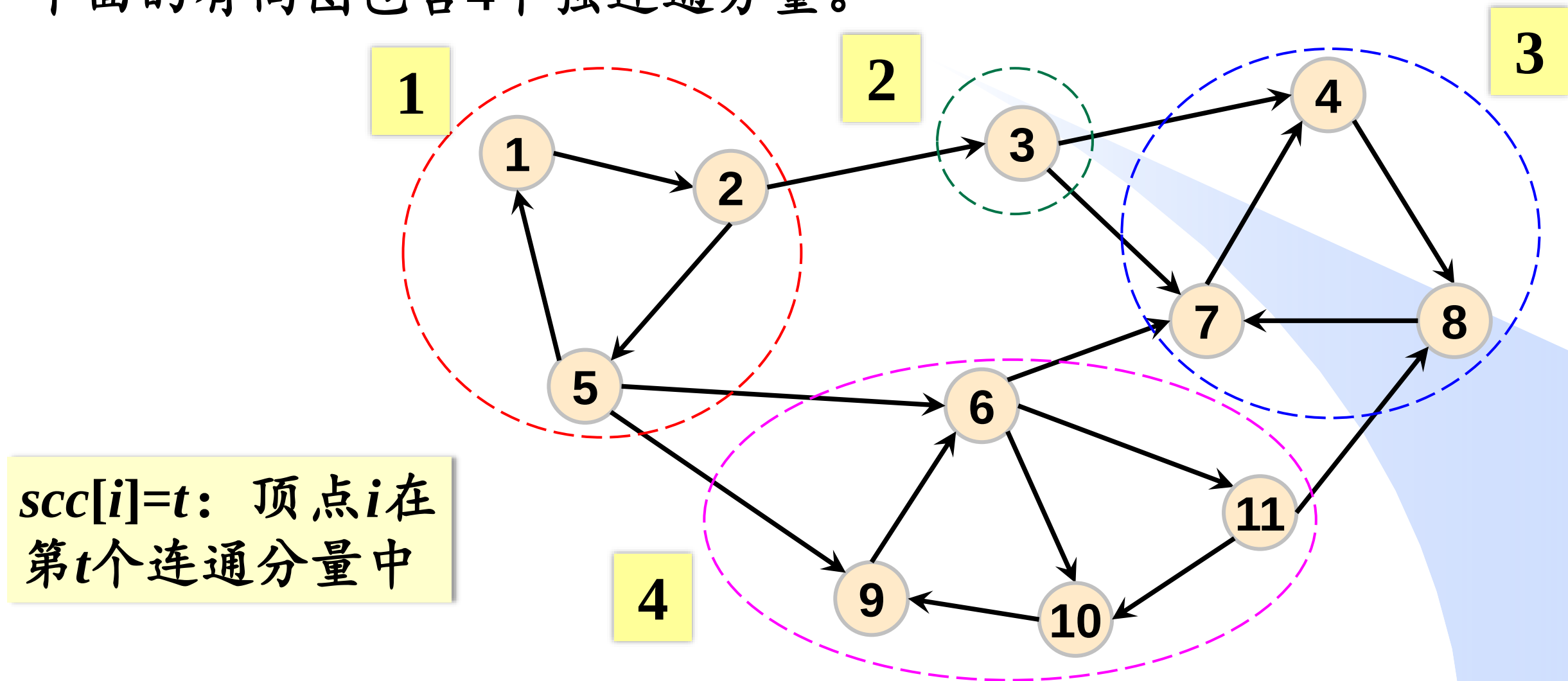
- 最小支撑树的概念
- Prim算法
- Kruskal算法
- **强连通分量**
- 图的其他应用举例

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

强连通分量 (Strongly Connected Components, SCC) B

下面的有向图包含4个强连通分量。



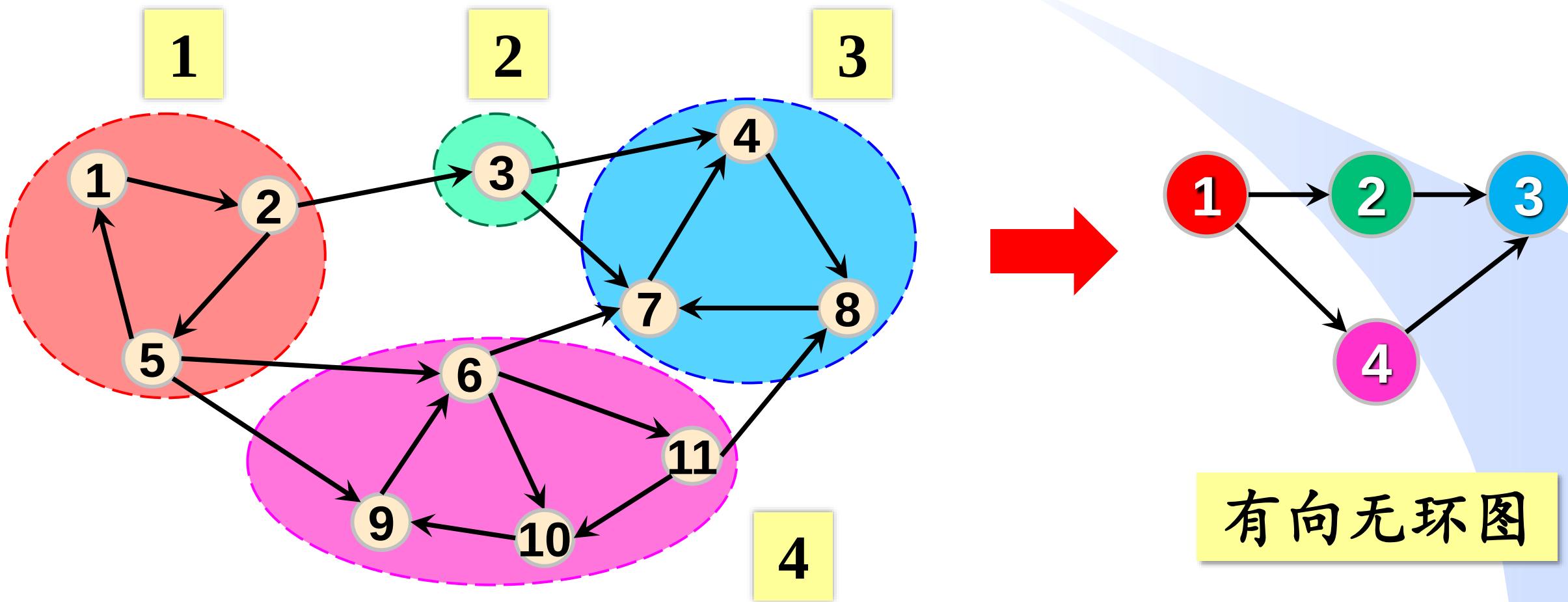

```
int SCC(int G[N][N], int n, int scc[]){//返回强连通分量个数
    int R[N][N]={0};
    for(int i=1; i<=n; i++) scc[i]=0;
    Warshall(G, n, R); //R为可及矩阵
    int t=0; //t为强连通分量的编号
    for(int i=1; i<=n; i++) //求顶点i所在的强连通分量
        if(scc[i]==0){ //顶点i还没被放入某个强连通分量里
            scc[i]=++t; //将顶点i放入强连通分量t
            for(int j=1; j<=n; j++) //与i相互可及的点放入强连通分量t
                if(scc[j]==0 && R[i][j]==1 && R[j][i]==1)
                    scc[j]=t; //顶点j放入强连通分量t里
        }
    return t;
}
```

$scc[i]=t$ 表示顶点 i 在个连通分量 t 里

时间复杂度 $O(n^3)$

强连通分量——缩点

把连通分量缩成一个点，该点的编号就是连通分量的编号



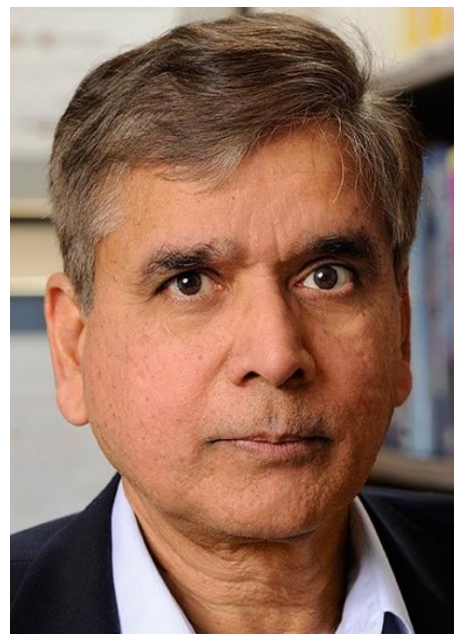
有向无环图

求有向图强连通分量的更高效算法

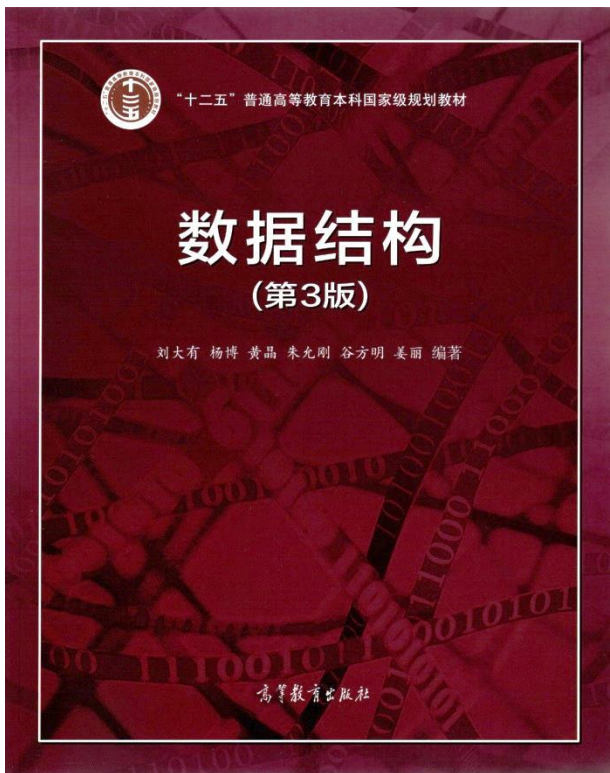
算法	时间复杂度	提出者
Tarjan算法	$O(n+e)$	Robert Tarjan
Kosaraju算法	$O(n+e)$	S. Rao Kosaraju



Robert Tarjan
图灵奖获得者
普林斯顿大学教授
美国科学院院士
美国工程院院士



S. Rao Kosaraju
约翰霍普金斯大学教授
ACM Fellow
IEEE Fellow



最小支撑树及图应用

- 最小支撑树的概念
- Prim算法
- Kruskal算法
- 强连通分量
- **图的其他应用举例**

数据之美
结构之美
算法之道

zhuyungang@jlu.edu.cn

PageRank算法简介

8 直播吧

首页 论坛 NBA视频 足球视频 **NBA资讯** 足球资讯 电竞 综合 比分 数据 资料

- ⚡ 国足世预赛取消包机待遇，搭乘普通航班出行，足...
 - 黄日华向中超联队道歉：对不起，我错了！
 - 梅西和马拉多纳在阿根廷人心中的地位！贝隆总统 圣马丁将...
 - 弗格森：“世界上任何门将都扑不出这个球”
 - 要扣钱了！马凡舒女足节目中口播失误及时纠正🙄
 - 霍伊伦这一扣让我们看到了红魔复兴的火苗
 - 这简直是人间轰炸机！什么东西飞过去了🤯
 - 身在曹营心在汉？帕尔默绝平后科瓦契奇的反应🙄
 - 拜仁签下17岁澳大利亚边锋伊昆昆达，曾对国少轰入不讲理...
 - 国外点击量很高的 C罗滑跪防守梅西的视频 [查看更多](#)
- 今日趣图：对韩国拿1分，对泰国拿4分，对新加坡...
 - 最老联赛🤔全球联赛年龄：中超28.41岁遥遥领先，日本第2...
 - 净年薪2000万欧！意媒：尤文愿付部分工资租桑乔，曼联仍...
 - 罗马诺：那不勒斯主帅西亚下课，马扎里回归签约至明年6月
 - ⚡西媒主编爆：瓜迪奥拉在世界杯上赞扬梅西，哈兰德很生气
 - 马赫雷斯：瓜帅想留我但我决定离开 喜欢的盘带高手梅西本...
 - 莱切总监：不同意VAR的判罚，本赛季的意甲最佳进球被取消了
 - 媒体人：当年为了舔恒大，有些头头把划健的合同给改了
 - 危🚨武汉足球博主：武汉三镇股改陷入僵局，明年大概率体...
 - RMC：皇马内部越来越欣赏哈兰德 他合同中有“皇... [查看更多](#)
- 加时变成被绝杀！最后0.6秒左联年自杀式犯规，...
 - 考神要来打CBA？外媒记者：中国联赛球队对考辛...
 - 一副老大姿态！暂停时教练布置战术 普尔被多次提醒后完美...
 - 表现相当低迷！多项数据创生涯新低！本赛季的维金斯到底...

1

2

3

互联网可以看成是一个有向图

- ✓ 顶点：网页
- ✓ 有向边：网页间的超链接

英超 曼联 切尔西 阿森纳 曼城 利物浦 热刺 +更多

真敢要？尤文有意租桑乔 但他已3个月没上场&两年来...
时光机 | 利物浦嘉士伯冠名球衣发布
全球球队年龄：青岛海牛31.48岁无解居首🐼成都蓉城2...
BBC：博比-查尔顿的世界杯半决赛签名球衣被拍出5...
本赛季五大联赛点射进球榜：恰尔汗奥卢5球居首，帕...
9.64球！阿森纳是本赛季英超至今，预期丢球数最少的...
本赛季英超仅3队未因失误导致丢球：曼联、热刺、西...
再坐半年冷板凳？罗马诺：切尔西不会签拉姆斯代尔，...
英媒：热刺有意弗鲁米嫩塞中场安德烈，考虑出售霍伊...

湖人 詹姆斯 戴维斯 科比 +更多

场均得分最高二人组：帝西61分居首 东欧55.6分第2 詹...
美媒晒命中率对比：唐斯45.1%&三分34.7% 浓眉52.7%...
彩经：步行者胜76人 森林狼双杀勇士 快船难敌掘金 湖...
🦈鲨鱼生涯打哪队场均得分最高？仅一队低于20分 第...
【直播吧评选】11月14日NBA最佳球员：西亚卡姆
进击的巨人——但是是NBA球星版
美记排现役TOP50：约基奇第1 库里大帝第2档 老詹KD...
湖人球员到访UCLA Health 与患者及其家属进行交流💛
湖人本赛季至今场均命中9记三分&命中率30% 均排名...

重大 英超 西甲 意甲 德甲 中超 法甲

马德里竞技 3-1 比利亚雷亚尔 [集锦](#) [录像](#)
国际米兰 2-0 弗洛西诺内 [集锦](#) [录像](#)
拉齐奥 0-0 罗马 [集锦](#) [录像](#)
切尔西 4-4 曼城 [集锦](#) [录像](#)
巴塞罗那 2-1 阿拉维斯 [集锦](#) [录像](#)
利物浦 3-0 布伦特福德 [集锦](#) [录像](#)
那不勒斯 0-1 恩波利 [集锦](#) [录像](#)
皇马 5-1 瓦伦西亚 [集锦](#) [录像](#)
尤文图斯 2-1 卡利亚里 [集锦](#) [录像](#)

勇士 库里 克莱 维金斯 保罗 +更多

表现相当低迷！多项数据创生涯新低！这赛季...
累了？库里过去5场送出17助攻但失误21次...
彩经：步行者胜76人 森林狼双杀勇士 快船...
🦈鲨鱼生涯打哪队场均得分最高？仅一队低...
到底咋啦？维金斯本季得分助攻抢断盖帽等...
保罗出席威少35岁生日趴并晒出合照：祝我...
【直播吧评选】11月14日NBA最佳球员：西...
85投14中！赛季至今保罗、维金斯&库明加...
进击的巨人——但是是NBA球星版

搜索引擎的网页排序

当搜索引擎接收到用户查询请求时，执行如下操作：

- ① 从网页数据库中检索出包含指定关键字的所有网页。
- ② 按某种标准对返回的网页排序，将排序结果返回给用户。

Google™ Google™ Google™

Larry Page
Google联合创始人
斯坦福大学硕士
美国工程院院士

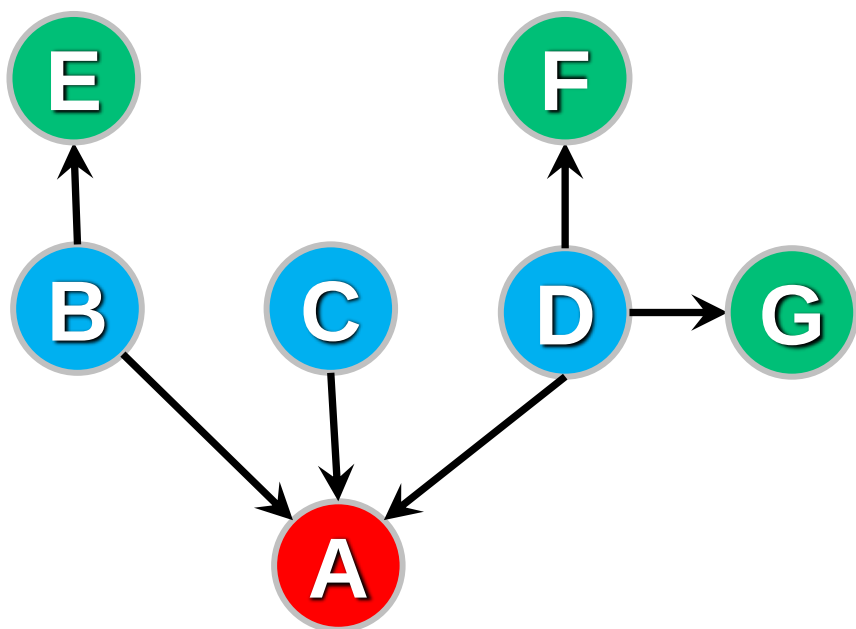
Sergey Brin
Google联合创始人
斯坦福大学硕士

PageRank算法——如何衡量网页的重要性

D

一个具有较高重要性的网页应该满足如下两个条件：

- ① 指向该网页的超链接很多，即在图结构中该页面所对应的顶点具有较大的入度。
- ② 指向该网页的网页都很重要。

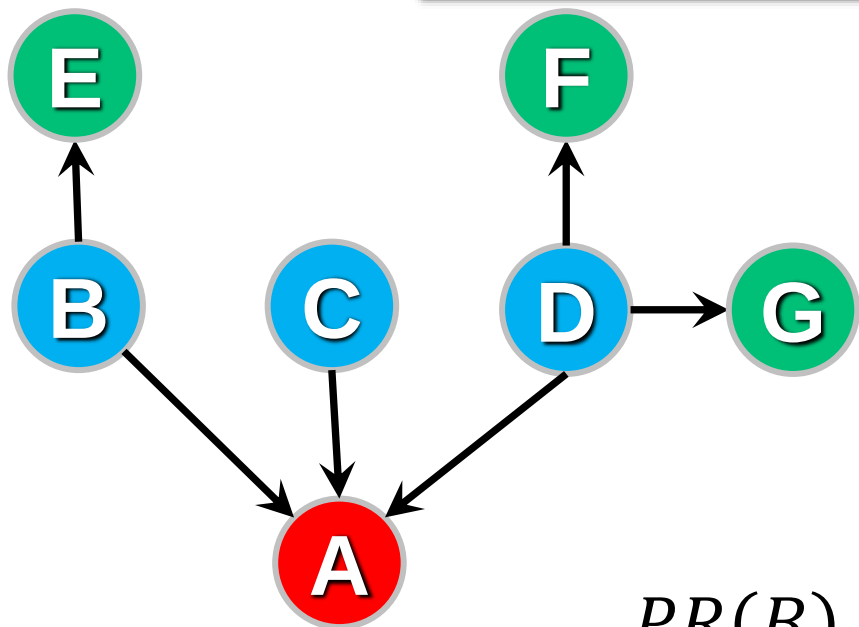


指向 v 的页面越多，指向 v 的页面越重要， v 就越重要。

PageRank算法——如何衡量网页的重要性

$PR(v)$: 表示页面 v 重要程度的PageRank值, 其公式如下, 其中 $v_1, \dots, v_k$ 为指向 v 的页面。

$$PR(v) = \frac{PR(v_1)}{\text{outdegree}(v_1)} + \dots + \frac{PR(v_k)}{\text{outdegree}(v_k)}$$



$$PR(A) = \frac{PR(B)}{2} + \frac{PR(C)}{1} + \frac{PR(D)}{3}$$

United States Patent Page

(10) Patent No.: US 7,058,628 B1
(45) Date of Patent: *Jun. 6, 2006

(54) METHOD FOR NODE RANKING IN A LINKED DATABASE	5,754,939 A	5/1998 Herz et al.	455/4.2
(75) Inventor: Lawrence Page, Stanford, CA (US)	5,832,494 A	11/1998 Egger et al.	
(73) Assignee: The Board of Trustees of the Leland Stanford Junior University, Palo Alto, CA (US)	5,848,407 A	12/1998 Ishikawa et al.	
	5,915,249 A	6/1999 Spencer	
	5,920,854 A	7/1999 Kirsch et al.	
	5,920,859 A	7/1999 Li	707/5
	6,014,678 A	1/2000 Inoue et al.	
	6,112,202 A	8/2000 Kleinberg	707/5
	6,163,778 A	12/2000 Fogg et al.	707/10
(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 622 days.	6,269,368 B1	7/2001 Diamond	707/6
	6,285,999 B1	9/2001 Page	707/5
	6,389,436 B1	5/2002 Chakrabarti et al.	707/513
	2001/0002466 A1	5/2001 Krasle	704/270.1

This patent is subject to a terminal disclaimer.

OTHER PUBLICATIONS

(21) Appl. No.: 09/895,174

(22) Filed: Jul. 2, 2001

Related U.S. Application Data

(63) Continuation of application No. 09/004,827, filed on Jan. 9, 1998.

(60) Provisional application No. 60/035,205, filed on Jan. 10, 1997.

(51) Int. Cl. G06F 17/00 (2006.01)

(52) U.S. Cl. 707/5; 707/10; 715/501.1

(58) Field of Classification Search 707/1-3, 707/100, 104.1; 715/501.1

See application file for complete search history.

Yuwono et al., "Search and Ranking Algorithms for Locating Resources on the World Wide Web", IEEE 1996, pp. 164-171.

L. Katz, "A new status index derived from sociometric analysis", 1953, Psychometrika, vol. 18, pp. 39-43.

C.H. Hubbell, "An input-output approach to clique identification sociometry", 1965, pp. 377-399.

(Continued)

Primary Examiner—Uyen Le

(74) Attorney, Agent, or Firm—Harry Snyder, LLP

(57) ABSTRACT

A method assigns importance ranks to nodes in a linked database, such as any database of documents containing citations, the world wide web or any other hypermedia database. The rank assigned to a document is calculated from the ranks of documents citing it. In addition, the rank

搜索引擎的网页排序

Google计算并保存网页数据库中所有页面的PageRank值，当接收到用户查询请求时，执行如下操作：

- ① 从网页数据库中检索出包含指定关键字的所有页面。
- ② 将这些页面按照PageRank值进行排序，并将排序结果返回给用户。

智能拼音输入法问题

shu	ju	jie	gou	ji	qi	ying	yong
树	局	结	够	及	期	应	勇
书	据	节	构	计	器	英	用
数	具	借	狗	机	其	营	永
...	...	...	...	...	...	...	...

shu'ju'jie'gou'ji'qi'ying'yong|

工具箱(分号)



1. 数据结构及其应用
2. 数据结构即
3. 数据结构及
4. 数据结构
5. 数据结



智能拼音输入法问题

$$P(w_i | w_{i-1}) = \frac{w_i w_{i-1} \text{同现的次数}}{w_{i-1} \text{出现的次数}}$$

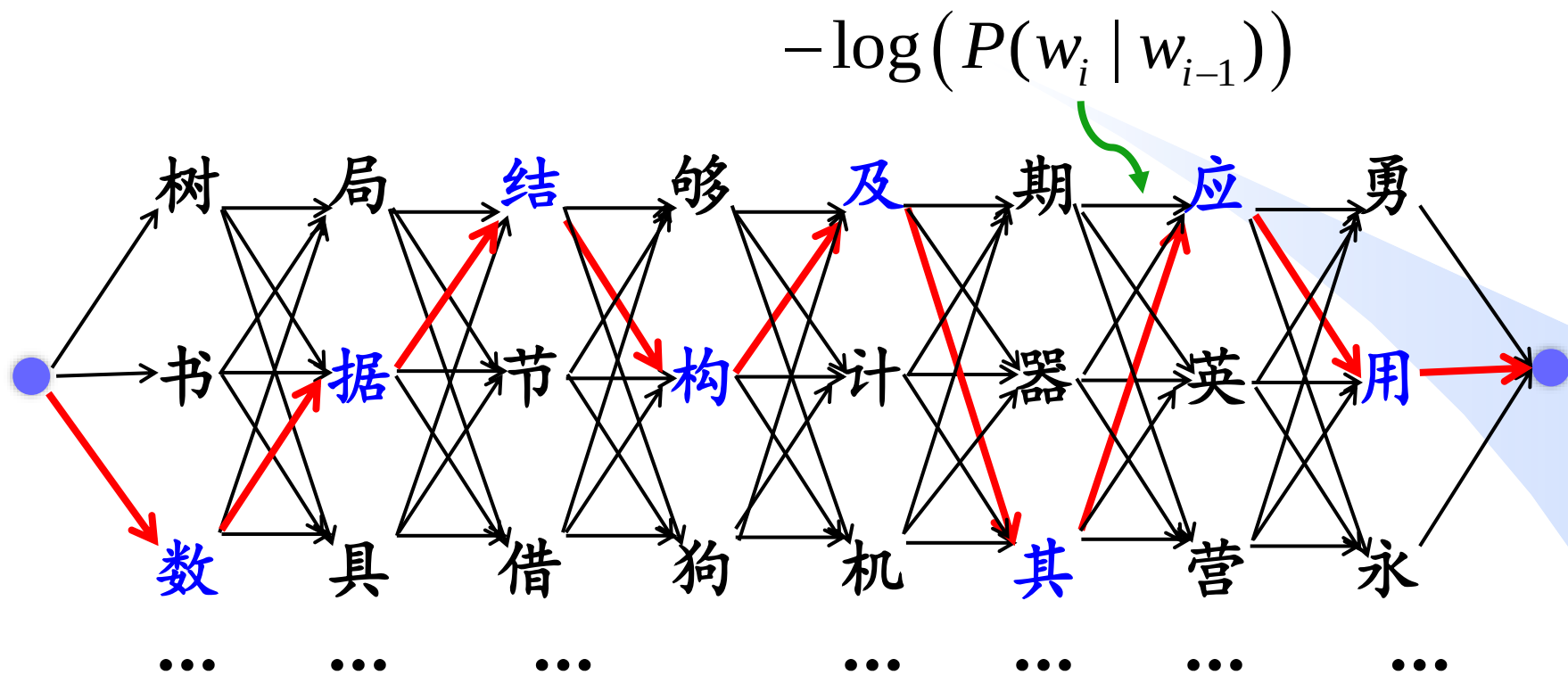
$$P(\text{据} | \text{数}) = \frac{\text{“数据”同时出现的次数}}{\text{“数”出现的次数}}$$

求 $\max \left(\prod_{i=1}^n P(w_i | w_{i-1}) \right)$ 所对应的句子



求 $\min \left(-\sum_{i=1}^n \log(P(w_i | w_{i-1})) \right)$ 所对应的句子

智能拼音输入法问题

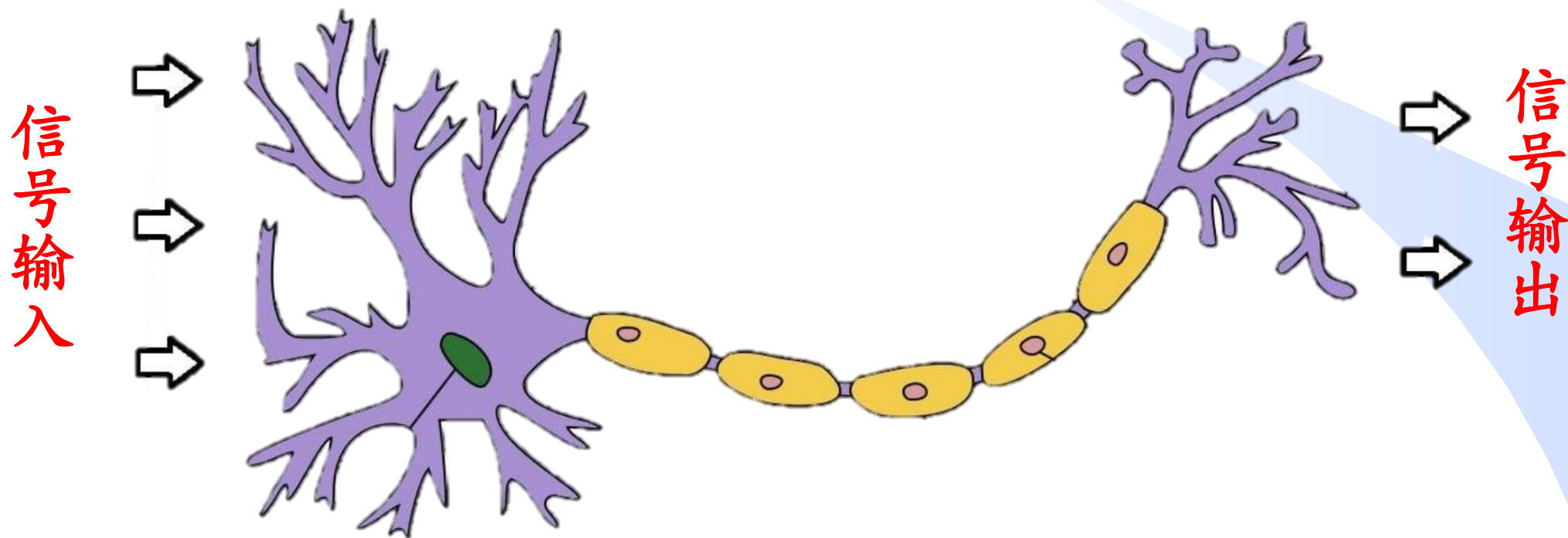


求使 $-\sum_{i=1}^n \log(P(w_i | w_{i-1}))$ 最小的顶点序列

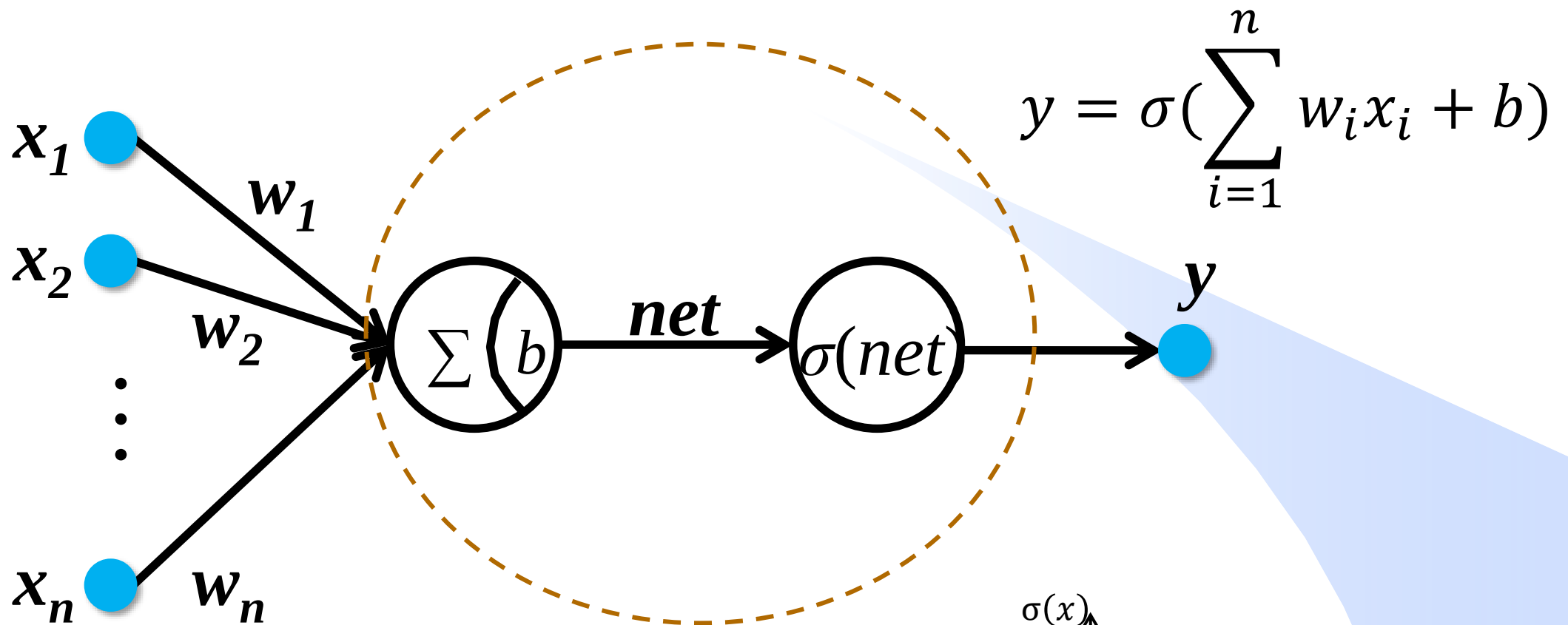
源点到汇点的最短路径

神经网络初探——人类的神经元

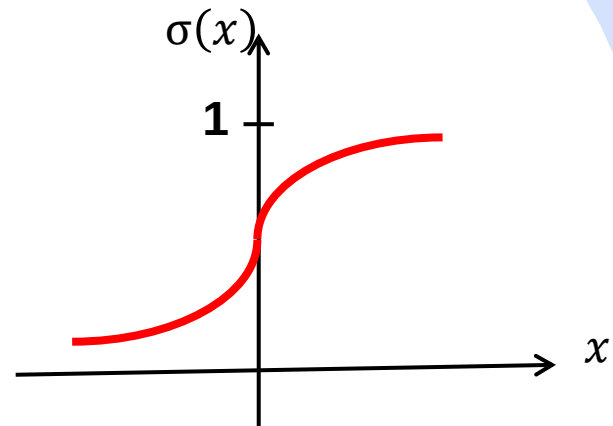
D



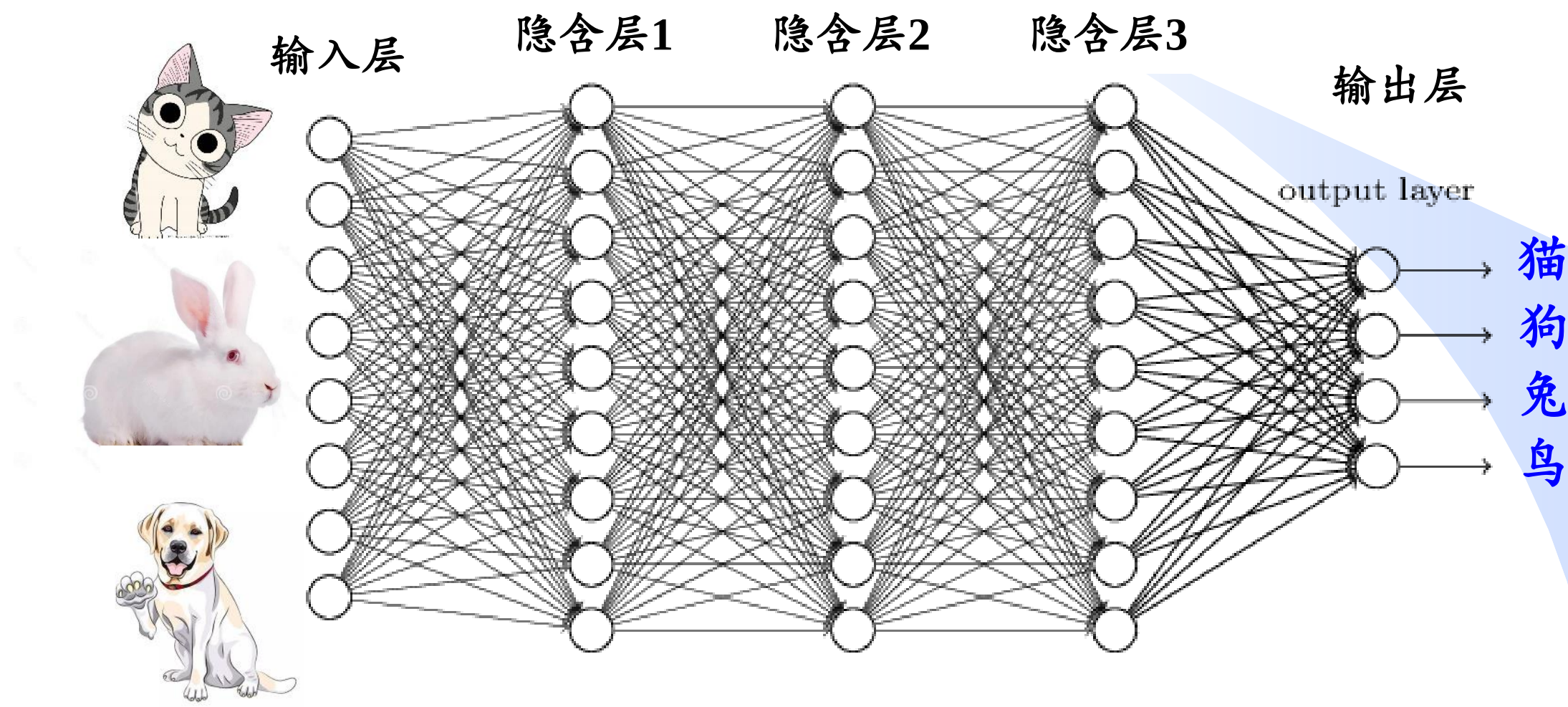
人工神经元



Sigmoid函数 $\sigma(x) = \frac{1}{1 + e^{-x}}$



人工神经网络



2018年图灵奖获得者——深度神经网络先驱



Yann LeCun

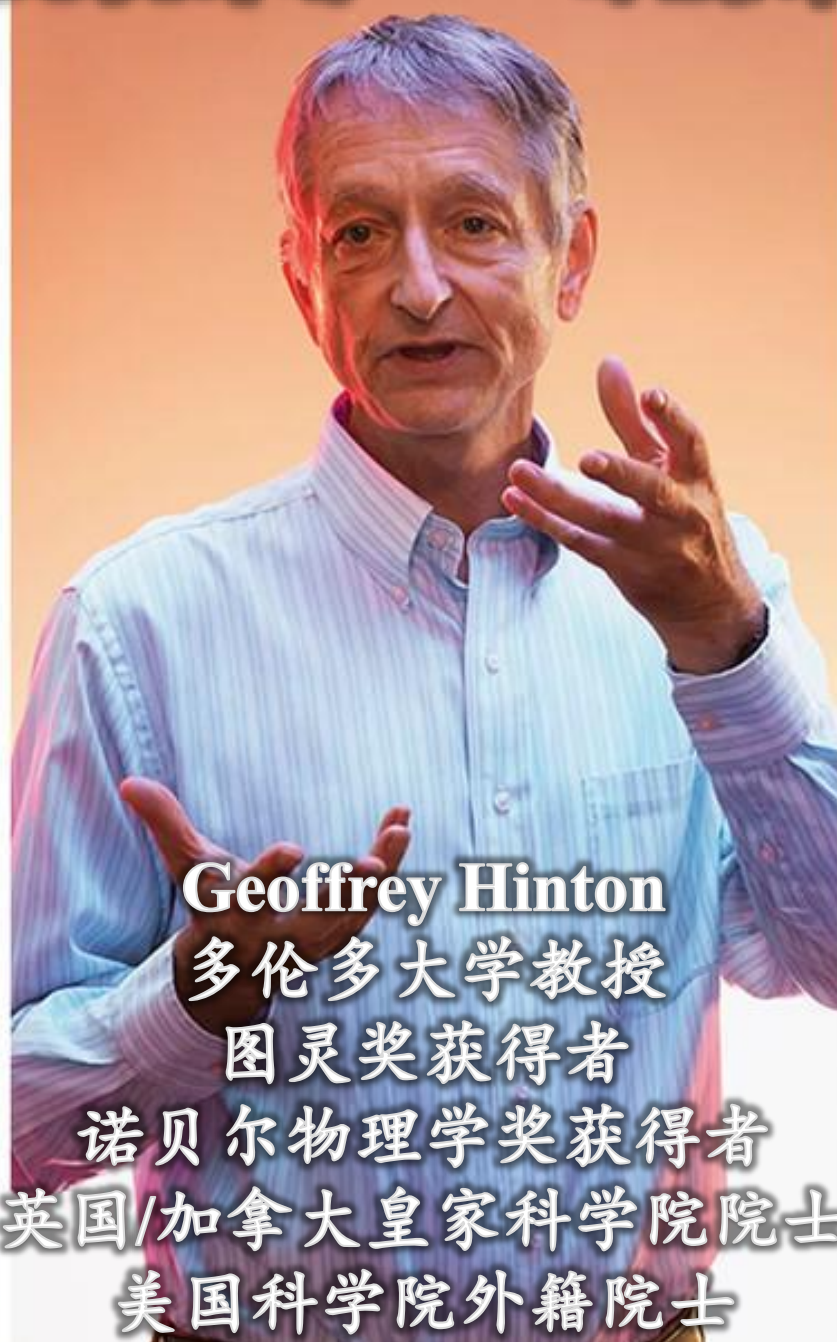
纽约大学教授

图灵奖获得者

美国科学院/工程院院士

法国科学院院士

Facebook 首席AI科学家



Geoffrey Hinton

多伦多大学教授

图灵奖获得者

诺贝尔物理学奖获得者

英国/加拿大皇家科学院院士

美国科学院外籍院士



Yoshua Bengio

蒙特利尔大学教授

图灵奖获得者

英国皇家科学院院士

加拿大皇家科学院院士

贝叶斯网络



Judea Pearl

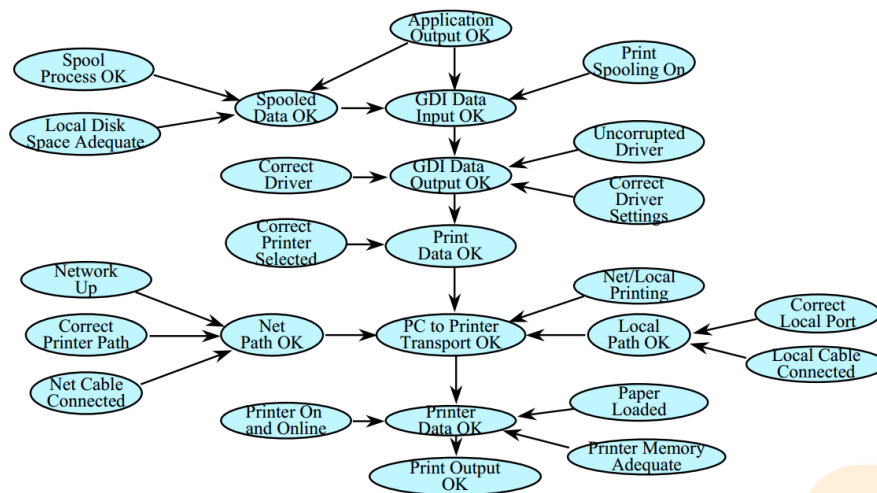
加州大学洛杉矶分校教授

图灵奖获得者

美国科学院院士

美国工程院院士

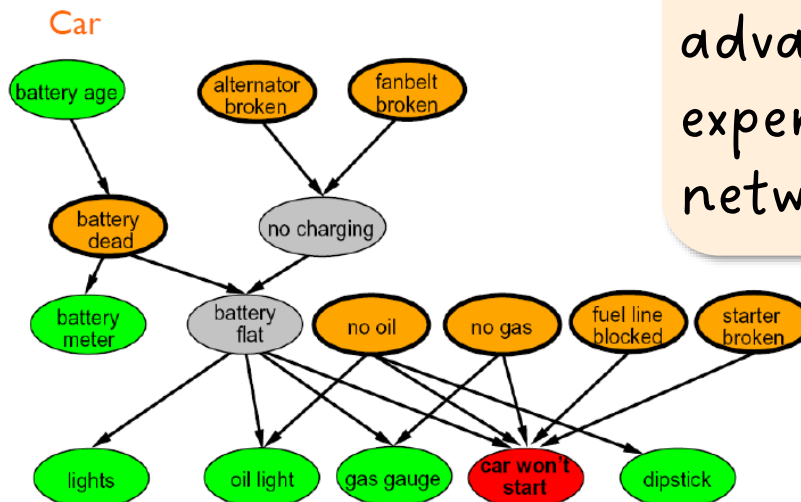
贝叶斯网络提出者



➤ 有向无环图

➤ 顶点：变量

➤ 边：变量间的因果关系



Microsoft's competitive advantage lies in its expertise in Bayesian networks.

